MAST387_Final_Project_TessaSkelly_40265993

March 17, 2026

1 MAST 387 Final Project

1.1 Student: Tessa Skelly (40265993)

1.2 Question A

[1]:
```
import numpy as np
import matplotlib.pyplot as plt
```

1.2.1 Question A-1

[2]:
```
def sparse_data(s, N, d):
    # create N-dimensional matrix with d entries
    data = np.zeros((d,N))

    for i in range(d):
        # get s different elements from 0 to N-1
        index = np.random.choice(N, s, replace=False)

        # add random standard normal variables to each index
        data[i, index] = np.random.standard_normal(s)

    return data
```

1.2.2 Question A-2

[3]:
```
print(sparse_data(4,10,5))
```

```
[[ 0.         -1.32129755  0.          0.          0.          0.
  -0.14678762 -1.7778823   1.13835623  0.        ]
 [ 0.          0.          0.42157883  0.          0.          0.
  -0.95618644  0.          0.78344496  1.75195612]
 [ 0.          0.          0.          0.          1.93503529  0.00352838
  -1.16115801  0.         -0.70758618  0.        ]
 [ 0.          0.65751539  0.          1.20148756 -0.62307274  0.
   0.          0.61838876  0.          0.        ]
 [ 0.          0.28858953  0.         -0.11202562  0.          0.
   0.         -0.89997561  0.13406547  0.        ]]
```

1.2.3 Question A-3

[4]:
```
def MP(A, y, K):
    # number of columns
    N = A.shape[1]
    # array of approximations 0 to k
    X = np.zeros((N, K+1))

    # initial approximation
    x_k = np.zeros(N)

    for k in range(K):
        r_k = y - A @ x_k

        # |A^T r_k|
        q = np.abs(A.T @ r_k)

        # j that maximizes quantity q
        j_k = np.argmax(q)

        # get jth column of A
        a_j = A[:, j_k]
        q_j = a_j.T @ r_k

        # update current column of x
        x_k[j_k] += q_j


        # store approximation in k+1th column of X
        X[:, k+1] = x_k

    return X
```

1.2.4 Question A-4

[7]:
```
# load data from numpy files
A = np.load("C:\\Users\\tessa\\Downloads\\sem 5 concordia\\mast 387\\sparse_A.
 ↪npy")
y = np.load("C:\\Users\\tessa\\Downloads\\sem 5 concordia\\mast 387\\sparse_y.
 ↪npy")
x = np.load("C:\\Users\\tessa\\Downloads\\sem 5 concordia\\mast 387\\sparse_x.
 ↪npy")

# take only first column
y = y[:,0]
x = x[:,0]
```

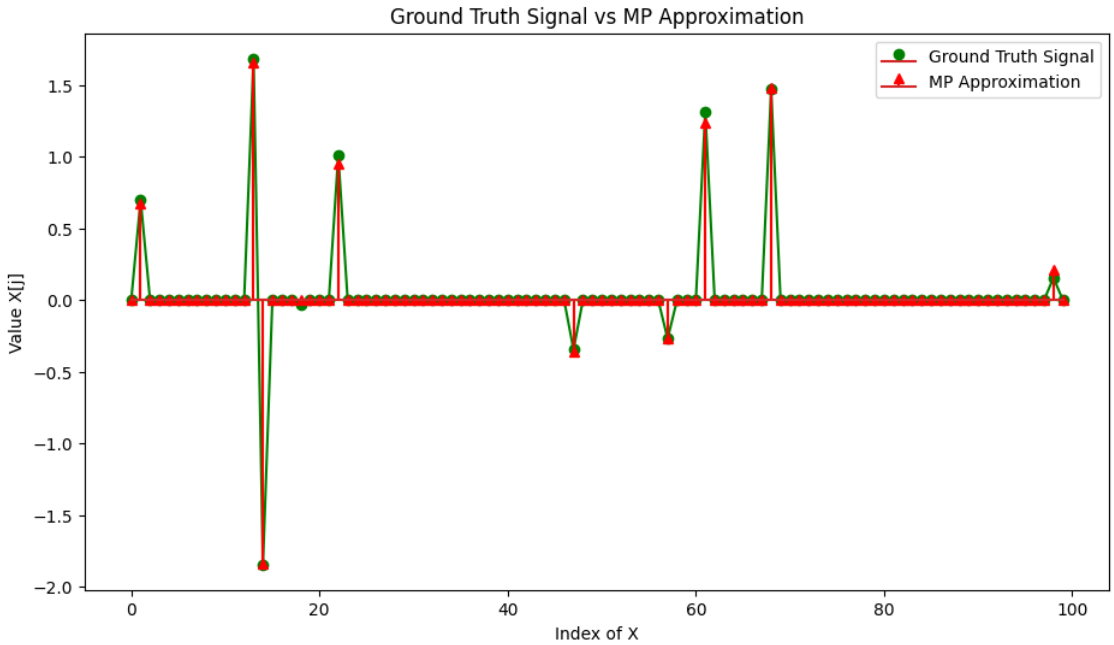
```
# get s
index_s = np.where(x != 0)[0]
s = len(index_s)
```

[8]:
```
# get X with MP()
X1 = MP(A, y, 2*s)
# get final approximation
x_app1 = X1[:,-1]
```

[9]:
```
x_app1
```

[9]:
```
array([ 0.        ,  0.67536399,  0.        ,  0.        ,  0.        ,
        0.        ,  0.        ,  0.        ,  0.        ,  0.        ,
        0.        ,  0.        ,  0.        ,  1.65652775, -1.83963965,
        0.        ,  0.        ,  0.        ,  0.        ,  0.        ,
        0.        ,  0.        ,  0.95207837,  0.        ,  0.        ,
        0.        ,  0.        ,  0.        ,  0.        ,  0.        ,
        0.        ,  0.        ,  0.        ,  0.        ,  0.        ,
        0.        ,  0.        ,  0.        ,  0.        ,  0.        ,
        0.        ,  0.        ,  0.        ,  0.        ,  0.        ,
        0.        ,  0.        , -0.35761913,  0.        ,  0.        ,
        0.        ,  0.        ,  0.        ,  0.        ,  0.        ,
        0.        ,  0.        , -0.2672645 ,  0.        ,  0.        ,
        0.        ,  1.24053829,  0.        ,  0.        ,  0.        ,
        0.        ,  0.        ,  0.        ,  1.48252945,  0.        ,
        0.        ,  0.        ,  0.        ,  0.        ,  0.        ,
        0.        ,  0.        ,  0.        ,  0.        ,  0.        ,
        0.        ,  0.        ,  0.        ,  0.        ,  0.        ,
        0.        ,  0.        ,  0.        ,  0.        ,  0.        ,
        0.        ,  0.        ,  0.        ,  0.        ,  0.        ,
        0.        ,  0.        ,  0.        ,  0.21615454,  0.        ])
```

[10]:
```
plt.figure(figsize = (11, 6))
plt.stem(x, linefmt='g-', markerfmt='go-',label = "Ground Truth Signal")
plt.stem(x_app1, linefmt='r-', markerfmt='r^', label = "MP Approximation")
plt.title("Ground Truth Signal vs MP Approximation")
plt.xlabel("Index of X")
plt.ylabel("Value X[j]")
plt.legend()
plt.show()
```



[11]:
```
# 2-norm value
two_norm = np.linalg.norm(x)

# error
rel_e1 = np.linalg.norm(x-x_app1)/two_norm
print("Relative Approximation Error:", rel_e1)
```

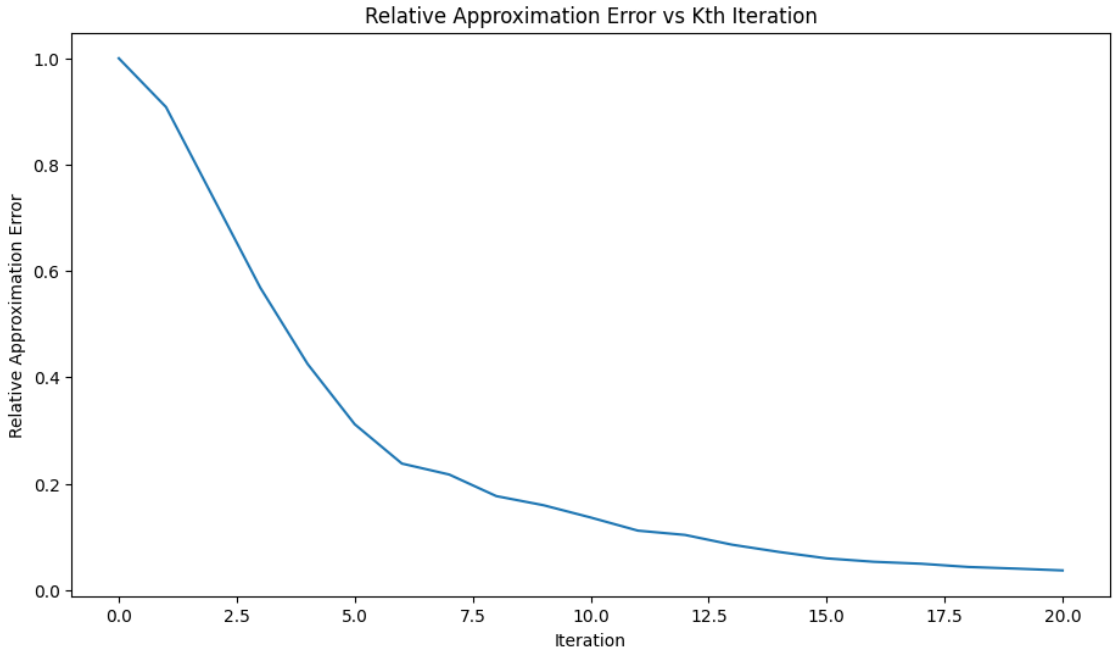
```
Relative Approximation Error: 0.03666076133480485
```

1.2.5 Question A-5

[12]:
```
# number of iterations is the number of columns in X
K1 = X1.shape[1]

# store relative approximation errors
rel_errors1 = np.zeros(K1)

for k in range(K1):
    rel_errors1[k] = np.linalg.norm(x-X1[:,k])/np.linalg.norm(x)


# plot
plt.figure(figsize=(11,6))
plt.plot(range(K1), rel_errors1)
plt.xlabel("Iteration")
plt.ylabel("Relative Approximation Error")
```

```
plt.title("Relative Approximation Error vs Kth Iteration")
```

[12]: Text(0.5, 1.0, 'Relative Approximation Error vs Kth Iteration')



The relative error seems to follow a negative exponential curve, showing that the error approaches zero very quickly as the iterations increase comes very close to 0 around the 16th iteration.

1.2.6 Question A-6

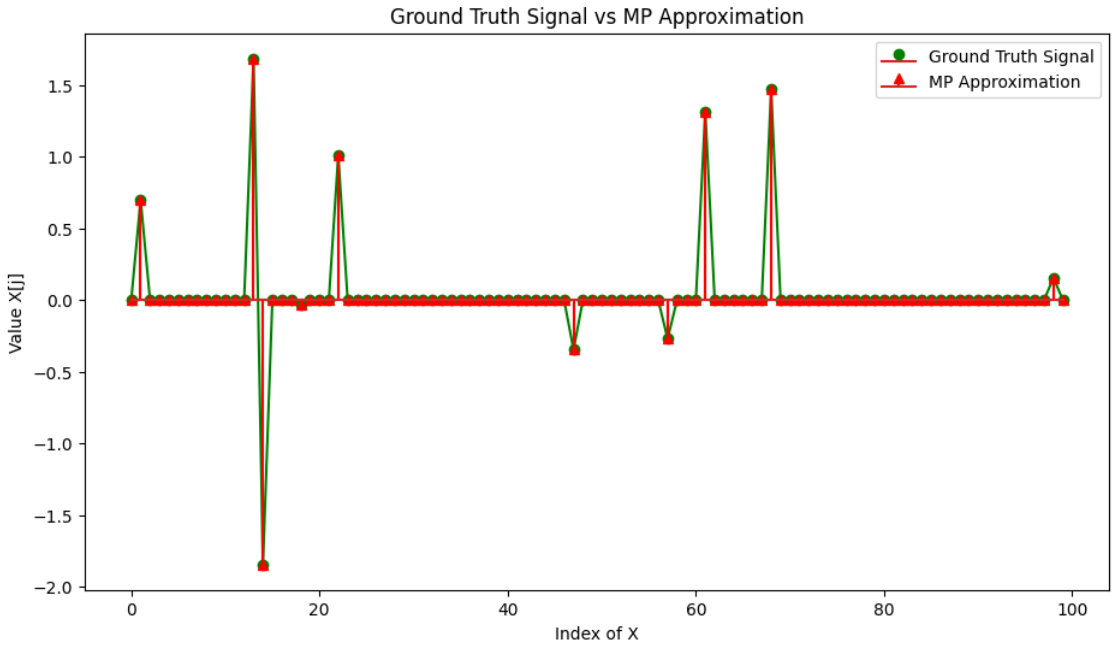
[13]:
```
def OMP(A, y, K):
    # number of columns
    N = A.shape[1]

    # array of approximations 0 to k
    X = np.zeros((N, K+1))

    # initial approximation
    x_k = np.zeros(N)

    # S list
    S = []

    for k in range(K+1):
        r_k = y - A @ x_k

        # |A^T r_k|
```

```
        q = np.abs(A.T @ r_k)

        # set q[S] to 0 so it is not picked again
        if len(S) > 0:
            q[S] = 0

        # jth index that maximizes q
        j_k = np.argmax(q)

        # add j to S
        S.append(j_k)

        # z solution to least squares problem
        z = np.linalg.lstsq(A[:,S], y)[0]

        # update x_k
        x_k = np.zeros(N)
        x_k[S] = z

        # add to array
        X[:, k] = x_k

    return X
```

1.2.7 Question A-7

[14]:
```
# OMP on data from files
X2 = OMP(A,y,2*s)
# final approximation
x_app2 = X2[:,-1]
```

[15]:
```
# visualize ground truth signal
plt.figure(figsize = (11, 6))
plt.stem(x, linefmt='g-', markerfmt='go-',label = "Ground Truth Signal")
plt.stem(x_app2, linefmt='r-', markerfmt='r^', label = "MP Approximation")
plt.title("Ground Truth Signal vs MP Approximation")
plt.xlabel("Index of X")
plt.ylabel("Value X[j]")
plt.legend()
plt.show()
```



[16]:
```
# 2-norm value
two_norm = np.linalg.norm(x)

# error
rel_e2 = np.linalg.norm(x-x_app2)/two_norm
print("Relative Approximation Error:", rel_e2)
```

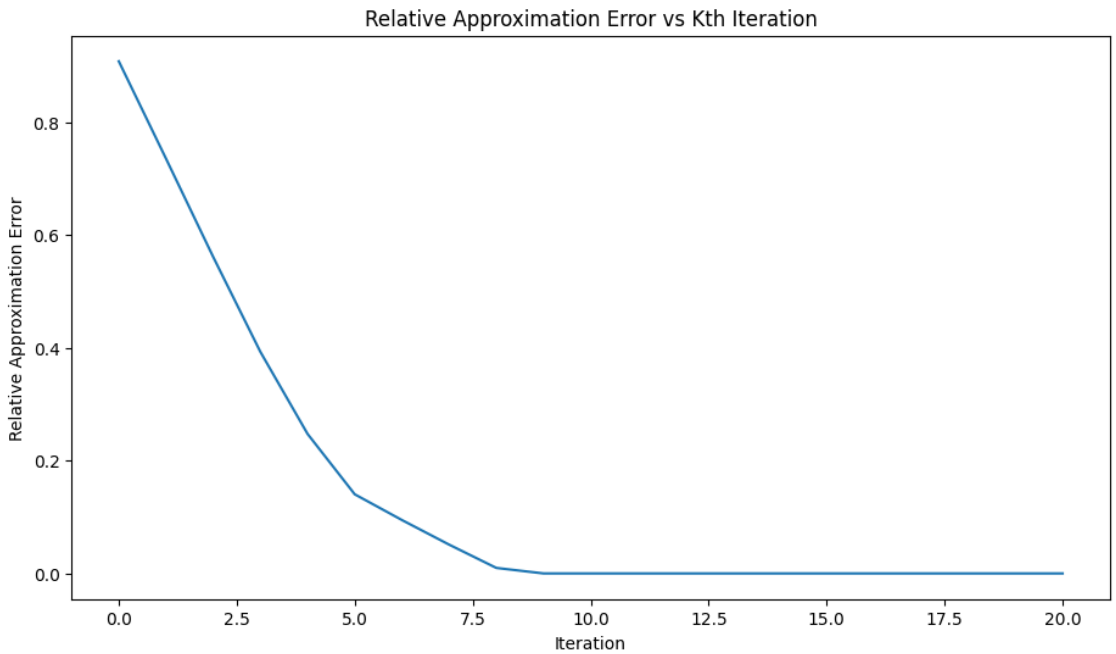
```
Relative Approximation Error: 1.1313740959022178e-15
```

1.2.8 Question A-8

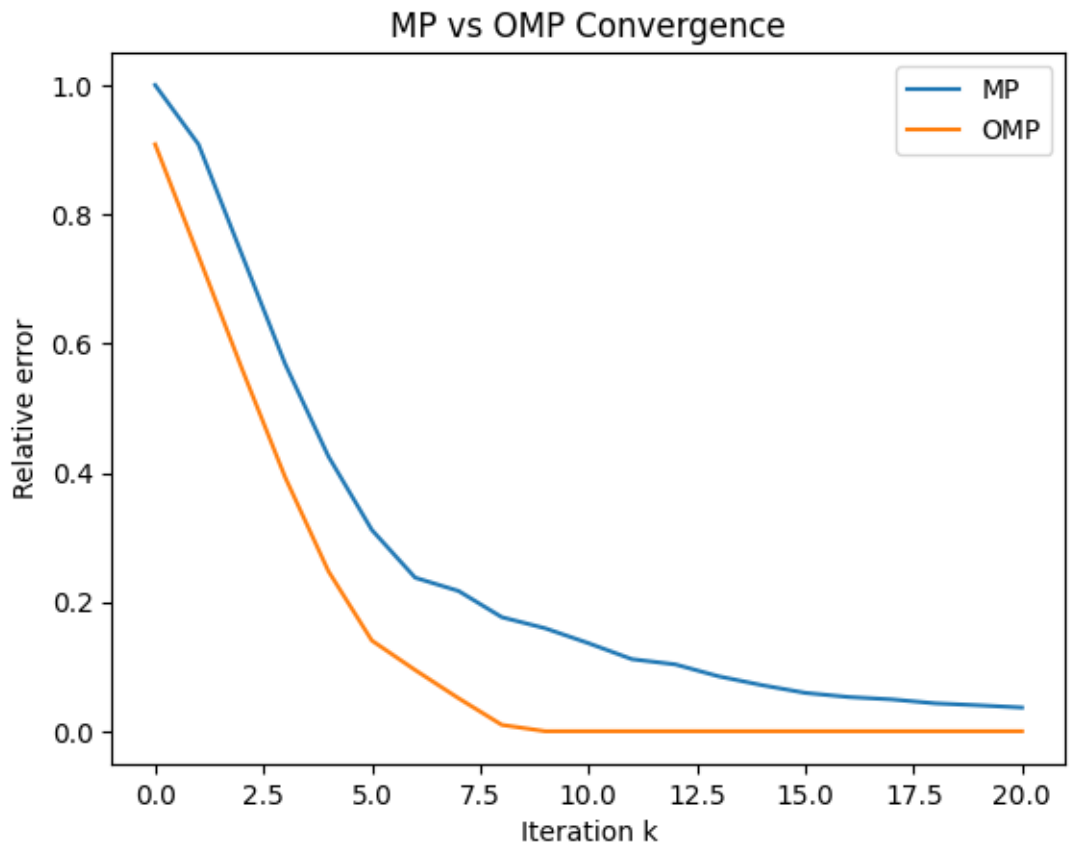
[17]:
```
# visualize convergence
# number of iterations is the number of columns in X
K2 = X2.shape[1]

# store relative approximation errors
rel_errors2 = np.zeros(K2)

for k in range(K2):
    rel_errors2[k] = np.linalg.norm(x-X2[:,k])/np.linalg.norm(x)


# plot
plt.figure(figsize=(11,6))
plt.plot(range(K2), rel_errors2)
plt.xlabel("Iteration")
```

```
plt.ylabel("Relative Approximation Error")
plt.title("Relative Approximation Error vs Kth Iteration")
```

[17]: Text(0.5, 1.0, 'Relative Approximation Error vs Kth Iteration')



[19]:
```
# plot comparing OMP to MP
plt.plot(range(K1), rel_errors1, label="MP")
plt.plot(range(K2), rel_errors2, label="OMP")
plt.xlabel("Iteration k")
plt.ylabel("Relative error")
plt.title("MP vs OMP Convergence")
plt.legend()
plt.show()
```



OMP converges to 0 faster than MP, using this data. The relative approximation error reaches about 0 around the 8th iteration compared to a slower and more gradual descent for MP. The calculated relative errors for the same number of iterations also have a significant difference with MP being 0.03666 and OMP being around 1.13 e-15 which is practically 0. OMP performs much better.

1.3 Question B

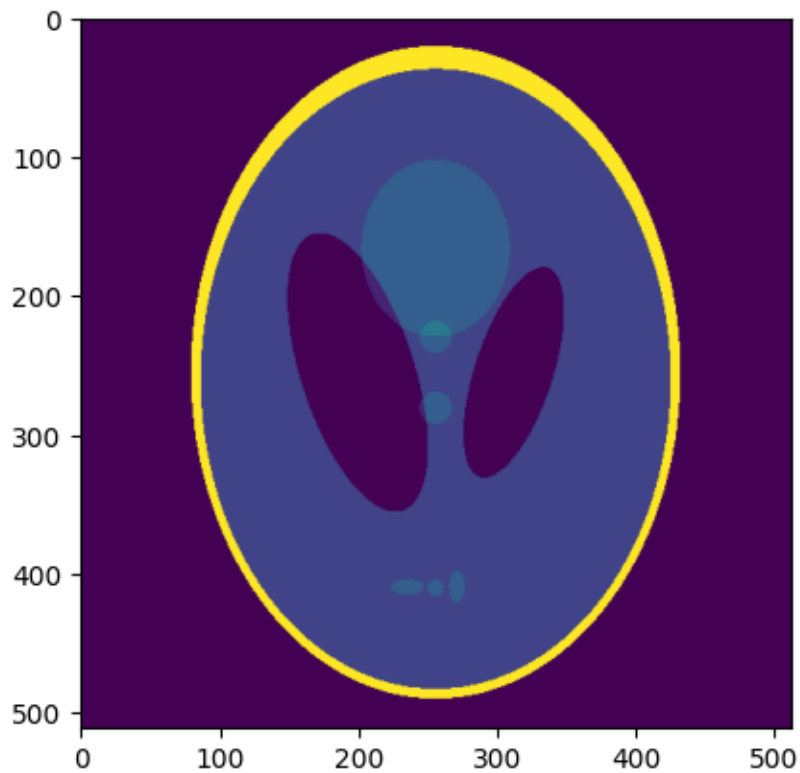
[20]:
```
from skimage import io, transform, data
import matplotlib.pyplot as plt
import numpy as np
import pywt as pw
```

1.3.1 Question B-1

[22]:
```
phantom_image = io.imread('C:\\Users\\tessa\\Downloads\\sem 5 concordia\\mast␣
↪387\\image_phantom.bmp')
```

[23]:
```
plt.imshow(phantom_image)
```

[23]: <matplotlib.image.AxesImage at 0x2001e31d940>

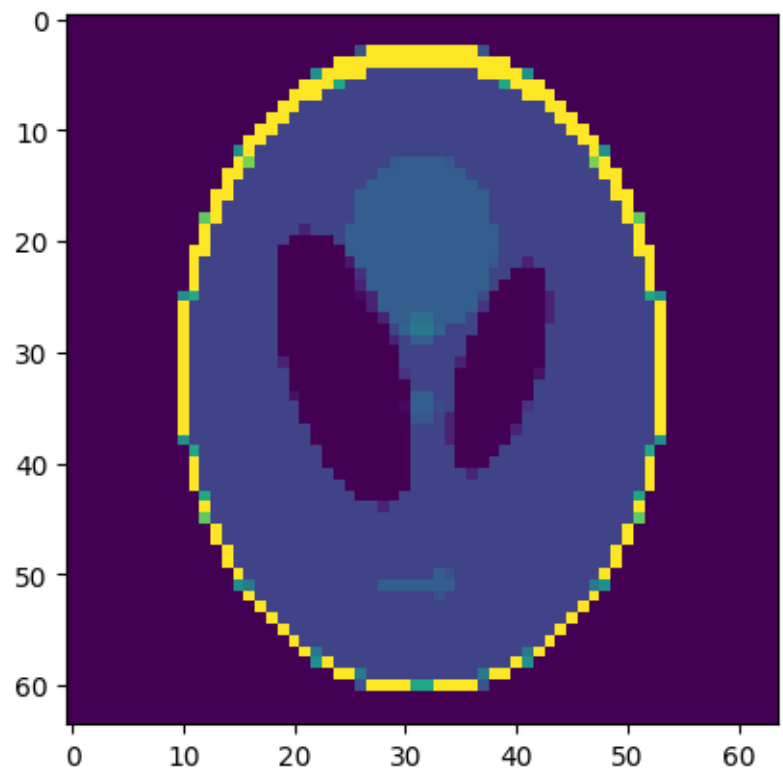


1.3.2 Question B-2

[24]:
```
img = transform.resize(phantom_image, (64, 64), anti_aliasing=False)
```

[25]:
```
plt.imshow(img)
```

[25]: <matplotlib.image.AxesImage at 0x2001eb57390>



1.3.3 Question B-3

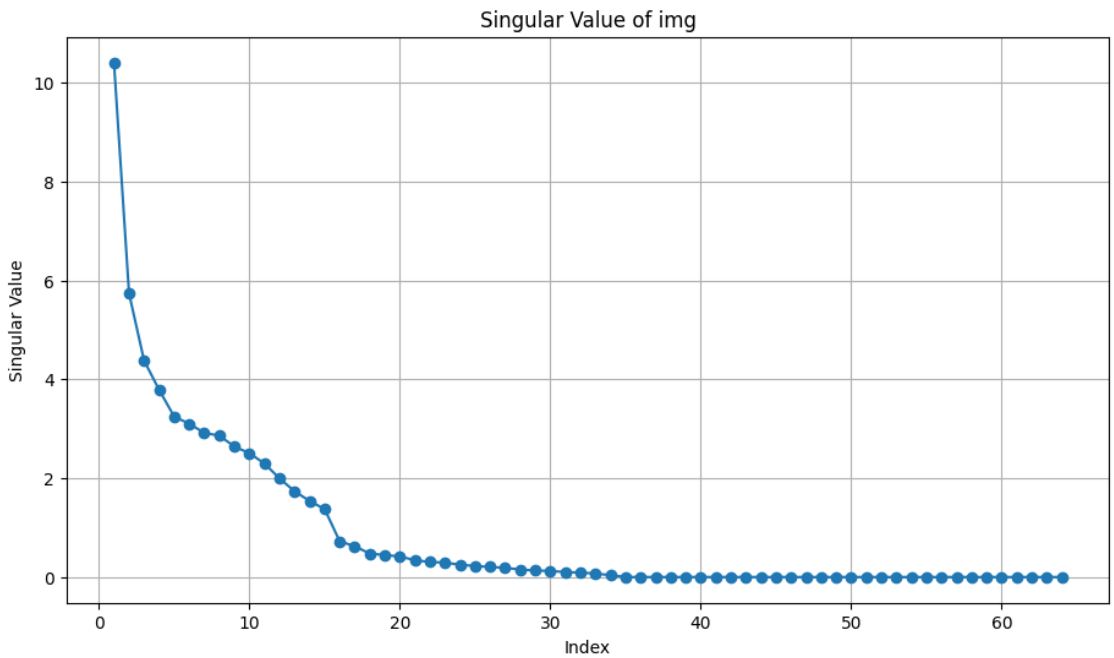
[26]:
```
# SVD
U,S,VT = np.linalg.svd(img)

print(U.shape)
print(S.shape)
print(VT.shape)
```

```
(64, 64)
(64,)
(64, 64)
```

[27]:
```
# first and last 10 singular values
S[:10], S[-10:]
```

[27]:
```
(array([10.41097601,  5.7416842 ,  4.38092928,  3.78795948,  3.24747128,
         3.10215507,  2.91698958,  2.86077654,  2.64147912,  2.5054743 ]),
 array([6.77482453e-16, 6.77482453e-16, 6.77482453e-16, 6.77482453e-16,
        6.77482453e-16, 6.77482453e-16, 6.77482453e-16, 6.77482453e-16,
        6.77482453e-16, 6.77482453e-16]))
```

[30]:
```
plt.figure(figsize=(11,6))
plt.plot(range(1,len(S)+1), S, marker='o')
plt.xlabel("Index")
plt.ylabel("Singular Value")
plt.title("Singular Value of img")
plt.grid(True)
plt.show()
```



The last 10 singular values have very little significance on the image and the first 10 decrease quickly from approximately 10 to around 2. Since there's 64 singular values. This suggests that most of the values have minimal impact on the image with just a few larger values having a bigger effect, so the image can be compressed using a relatively small number of singular values.

1.3.4 Question B-4

[31]:
```
# variance of image explained
image_variance = np.cumsum(S**2)/np.sum(S**2)

# indexes of singular values that reach 95%
s_95 = np.where(image_variance >= 0.95)
print(s_95)
```

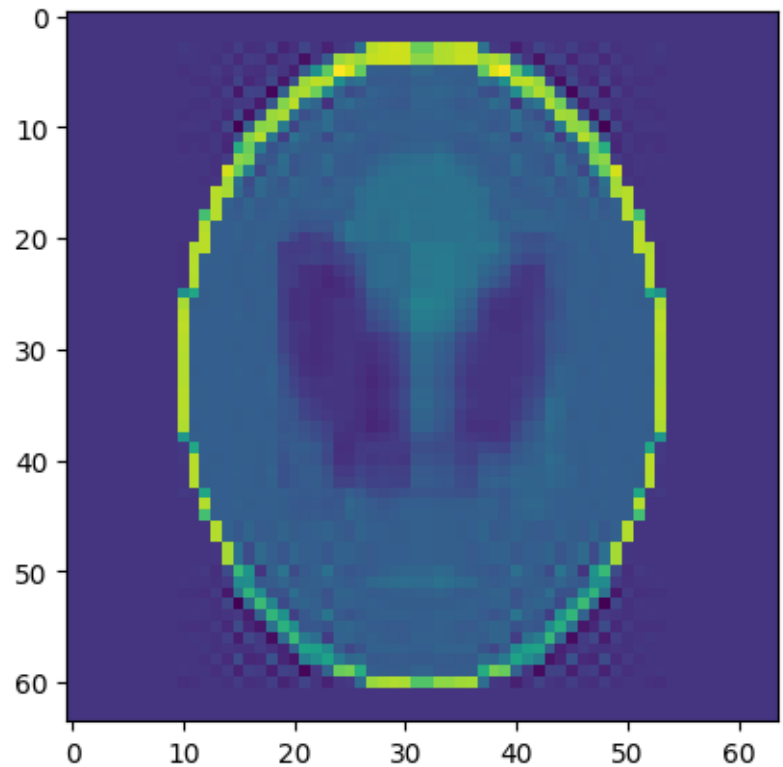
```
(array([11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27,
       28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44,
       45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61,
       62, 63]),)
```

Because at index 11 the singular values explain at least 95% of the image, then it takes 12 singular values to get there.

[32]:
```
# compressed SVD matrices
U_c = U[:, :12]
S_c = S[:12]
VT_c = VT[:12, :]
U_S = U_c * S_c

# compressed image
compressed_img = U_S @ VT_c
```

[33]:
```
plt.imshow(compressed_img)
```

[33]:
```
<matplotlib.image.AxesImage at 0x2001ec80a50>
```



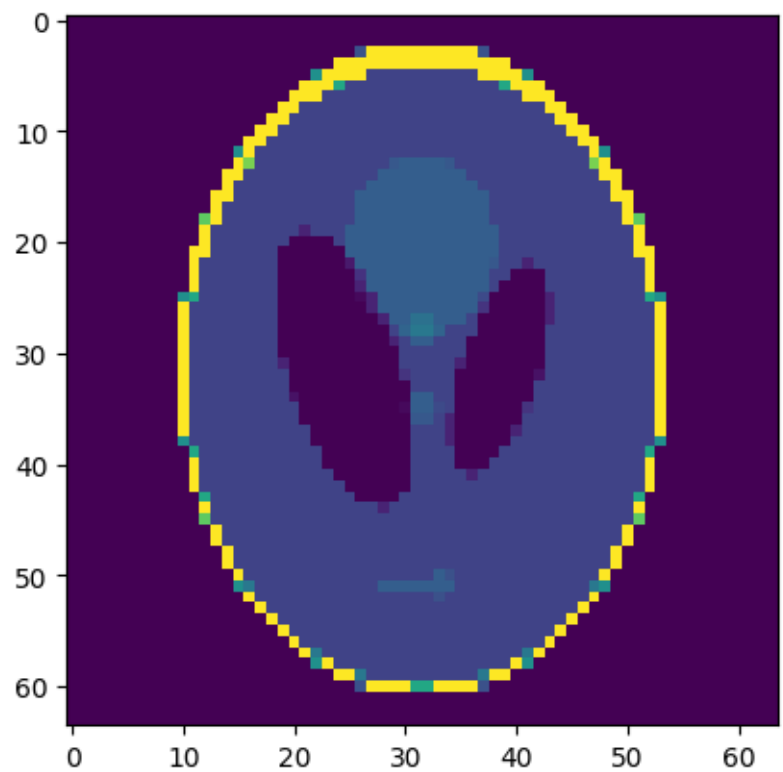
1.3.5 Question B-5

[34]:
```
# wavelet decomposition of img
wave_deco = pw.wavedec2(img, 'haar', mode='symmetric', axes=(-2,-1))
```

[35]:
```
# wavelet recomposition
img_reco = pw.waverec2(wave_deco, 'haar', mode='symmetric', axes=(-2,-1))
```

[36]:
```
# plot image
plt.imshow(img_reco)
```

[36]: <matplotlib.image.AxesImage at 0x2001ecdb750>



[37]:
```
# reconstruction error using frobenius norm
reco_err = np.linalg.norm(img-img_reco)
print("Reconstruction error:", reco_err)
```

```
Reconstruction error: 1.6234108501105212e-14
```

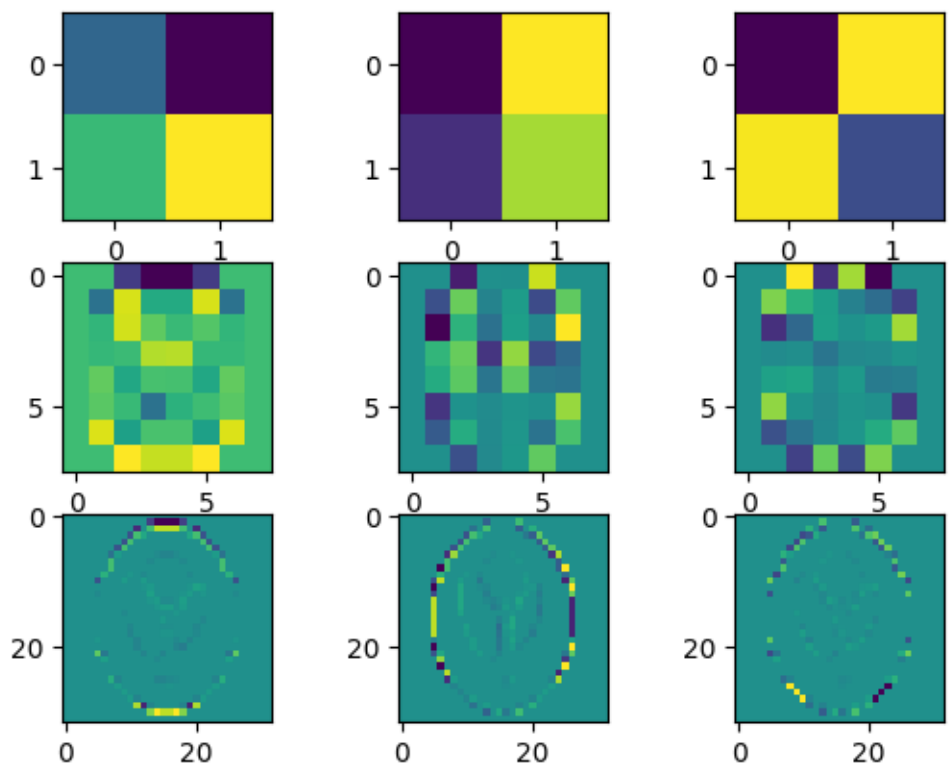
The reconstruction error is acceptable as it is practically 0 (10^{-14}).

1.3.6 Question B-6

[38]:
```
fig, ax = plt.subplots(3,3)

# level 2
H2, V2, D2 = wave_deco[2]
ax[0][0].imshow(H2)
```

```
ax[0][1].imshow(V2)
ax[0][2].imshow(D2)

# level 4
H4, V4, D4 = wave_deco[4]
ax[1][0].imshow(H4)
ax[1][1].imshow(V4)
ax[1][2].imshow(D4)

# level 6
H6, V6, D6 = wave_deco[6]
ax[2][0].imshow(H6)
ax[2][1].imshow(V6)
ax[2][2].imshow(D6)
```

[38]: <matplotlib.image.AxesImage at 0x2001ee85090>



1.3.7 Question B-7

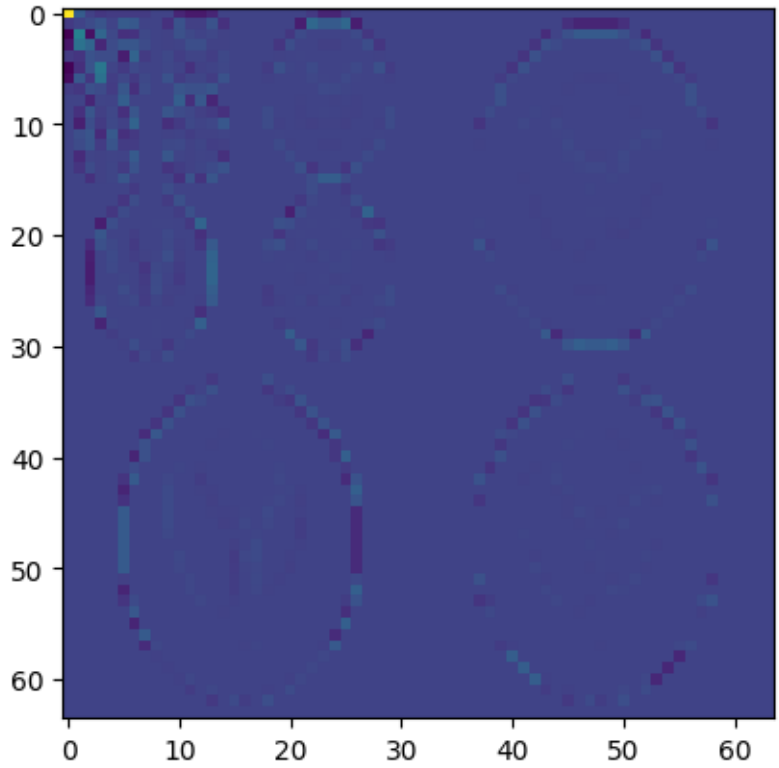
[39]:
```
def from_deco_to_img(wave_deco):

    # levels index
    lvl = len(wave_deco) - 1

    # biggest matrix from decomposition
    n = 2**lvl

    # create array
    deco_img = np.zeros((n,n))

    # scaling coefficient
    deco_img[0,0] = wave_deco[0][0,0]

    for i in range(1, lvl+1):
        # separate horizontal, vertical, and diagonal at every level i > 0
        H, V, D = wave_deco[lvl - (i - 1)]

        # size of submatrix
        size = 2**(lvl-i)

        # block of horizontal values
        deco_img[0:size, size:2*size] = H

        # block of vertical values
        deco_img[size:2*size, 0:size] = V

        # block of diagonal values
        deco_img[size:2*size, size:2*size] = D


    return deco_img
```

1.3.8 Question B-8

[40]:
```
# test function
deco_img = from_deco_to_img(wave_deco)
```

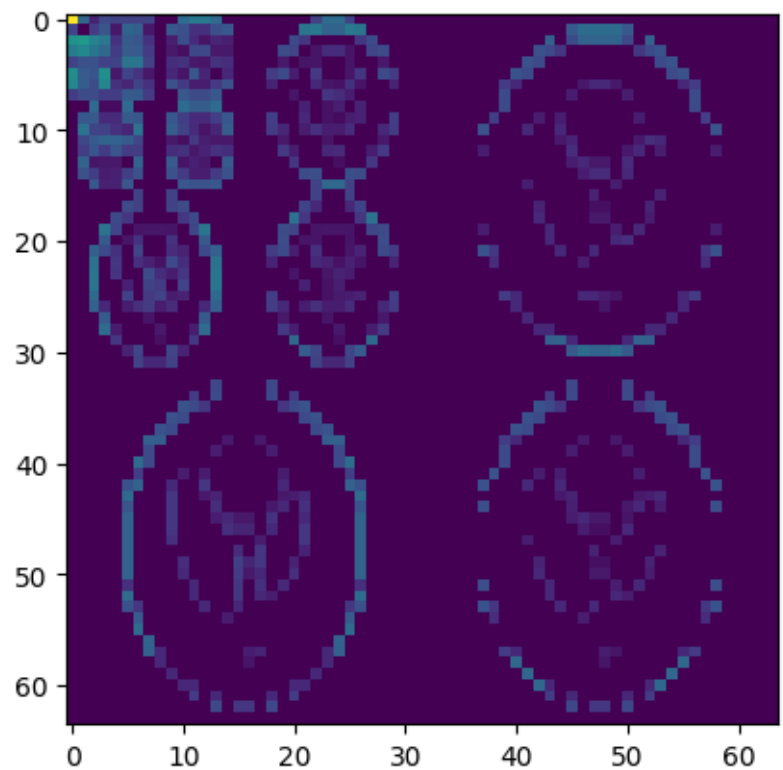
[41]:
```
# visualize
plt.imshow(deco_img)
```

[41]:
```
<matplotlib.image.AxesImage at 0x2001efa5950>
```



[42]:
```
# square root transformed visualization
plt.imshow(np.sqrt(np.abs(deco_img)))
```

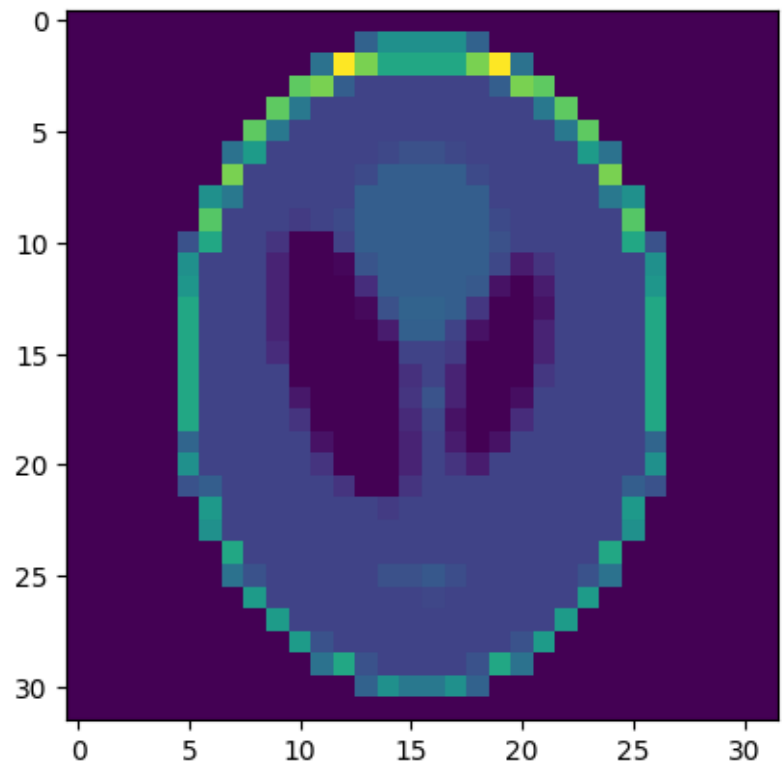
[42]: <matplotlib.image.AxesImage at 0x2001f028690>



1.3.9 Question B-9

[43]:
```
# resize
img_32 = transform.resize(img, (32, 32), anti_aliasing=False)

# visualize 32 x 32 image
plt.imshow(img_32)
```

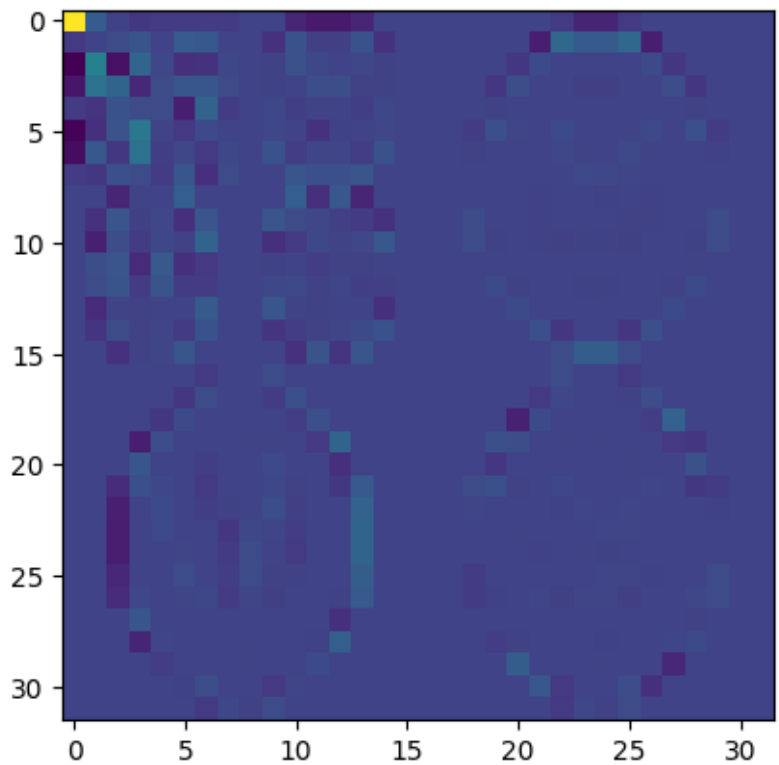
[43]: <matplotlib.image.AxesImage at 0x2001f07b390>



[44]:
```
# wavelet decomposition of img
wave_deco_32 = pw.wavedec2(img_32, 'haar', mode='symmetric', axes=(-2,-1))

# test function
deco_img_32 = from_deco_to_img(wave_deco_32)
```

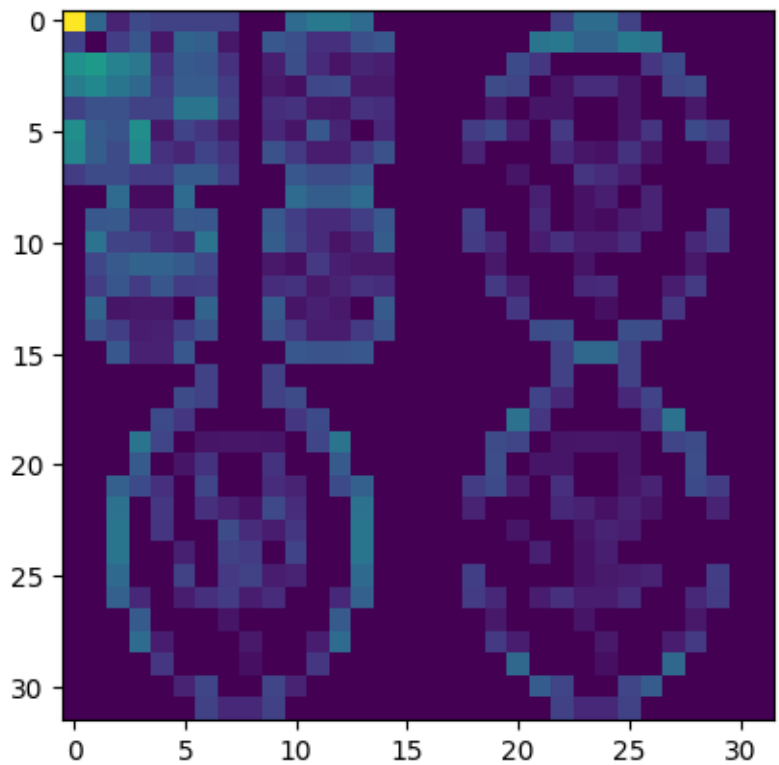
[45]:
```
# visualize
plt.imshow(deco_img_32)
```

[45]: <matplotlib.image.AxesImage at 0x2001f10a0d0>



[46]:
```
# visualize with square root
plt.imshow(np.sqrt(np.abs(deco_img_32)))
```

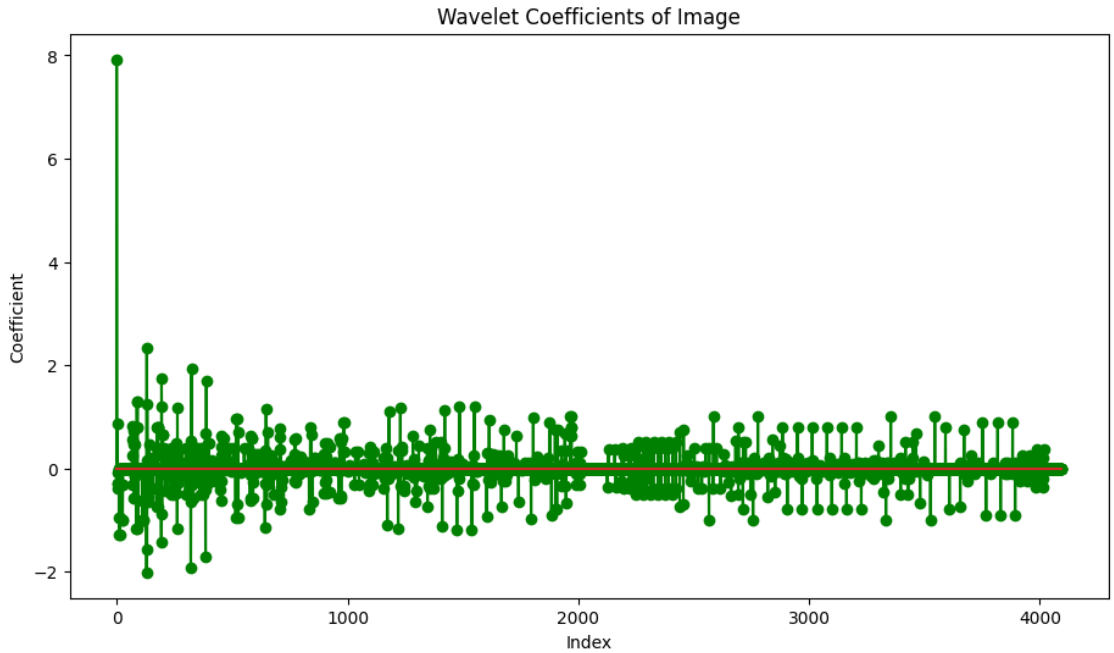
[46]: <matplotlib.image.AxesImage at 0x2001f18ce10>



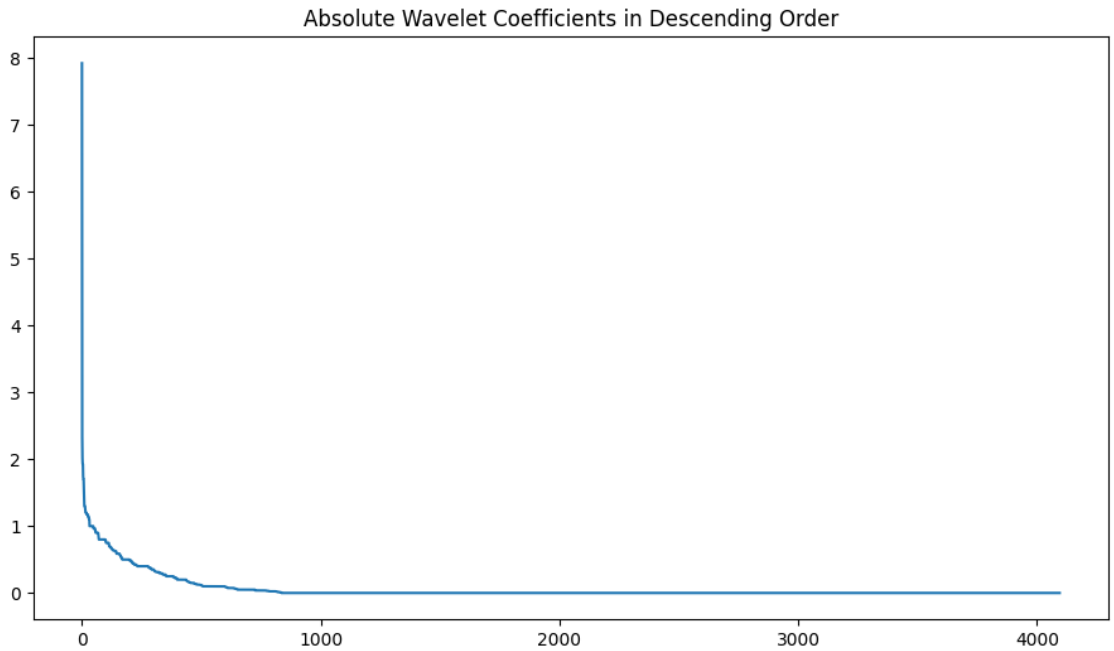
1.3.10 Question B-10

[47]:
```
# reshape
deco_vec = deco_img.flatten()
```

[48]:
```
# plot wavelet coefficients
plt.figure(figsize = (11, 6))
plt.stem(deco_vec, linefmt='g-', markerfmt='go-')
plt.title("Wavelet Coefficients of Image")
plt.xlabel("Index")
plt.ylabel("Coefficient")
plt.show()
```



[49]:
```
# get sorted coefficients
sorted_deco_vec = sorted(np.abs(deco_vec), reverse=True)

# plot absolute value
plt.figure(figsize=(11,6))
plt.plot(sorted_deco_vec)
plt.title("Absolute Wavelet Coefficients in Descending Order")
plt.show()
```



They seem to decay very fast since out of over 4000 values, they reach zero it seems before the 1000th biggest absolute value.

1.3.11 Question B-11

[50]:
```
# scalars used for wavelet coefficients
print(sorted_deco_vec[800])
print(sorted_deco_vec[900])
```

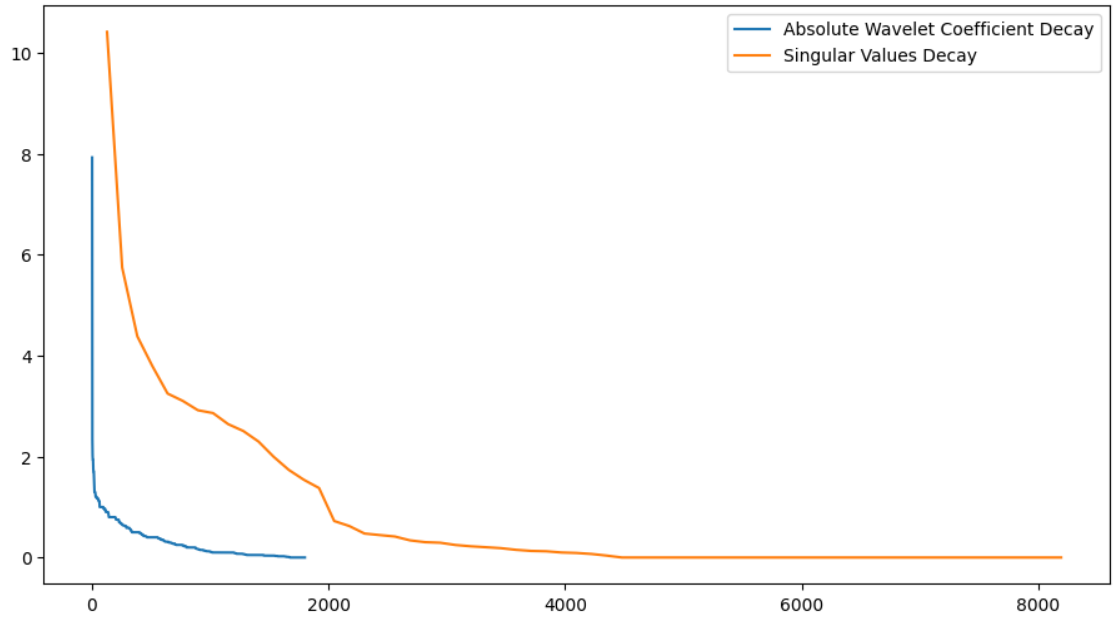
```
0.024509803921568644
0.0
```

[51]:
```
# use k=900 for absolute wavelet coefficients
k_wave = np.arange(1, 901)
wav_coeff_mem = 2*k_wave

# SVD memory using shapes of SVD matrices
n=64
k_svd = np.arange(1, len(S)+1)
svd_mem = k_svd*2*n
```

[52]:
```
# plot
plt.figure(figsize=(11,6))
plt.plot(wav_coeff_mem, sorted_deco_vec[:900], label = "Absolute Wavelet␣
↪Coefficient Decay")
plt.plot(svd_mem, S, label = "Singular Values Decay")
```

```
plt.legend()
```

[52]: <matplotlib.legend.Legend at 0x2001f8a4410>



Absolute wavelet coefficients seem to be the most memory efficient as I was able to cut k down to 900 which is less than a quarter of the original length of the vector (4096) and it was able to decay to zero. Meanwhile, the singular values seem to use between 4000 and 5000 memory scalars to decay to zero, which takes up more memory.

1.4 Question C

[129]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn.objects as so
from sklearn.decomposition import PCA
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import r2_score, mean_squared_error
from mpl_toolkits.mplot3d import Axes3D
```

1.4.1 Question C-1

[55]:
```
df_train = pd.read_csv("C:\\Users\\tessa\\Downloads\\sem 5 concordia\\mast␣
↪387\\house_train.csv")
```

1.4.2 Question C-2

[56]:
```
# mean of sale price
df_train['SalePrice'].mean()
```

[56]:
```
np.float64(181003.61007591442)
```

[57]:
```
# number of rows and columns
print("Number of columns:",len(df_train.columns))
print("Number of rows:", len(df_train))
```

```
Number of columns: 81
Number of rows: 1449
```

[58]:
```
# number of numerical features
numerical = [var for var in df_train.columns if df_train[var].dtype!='O']
print("Number of numerical features:",len(numerical))
```

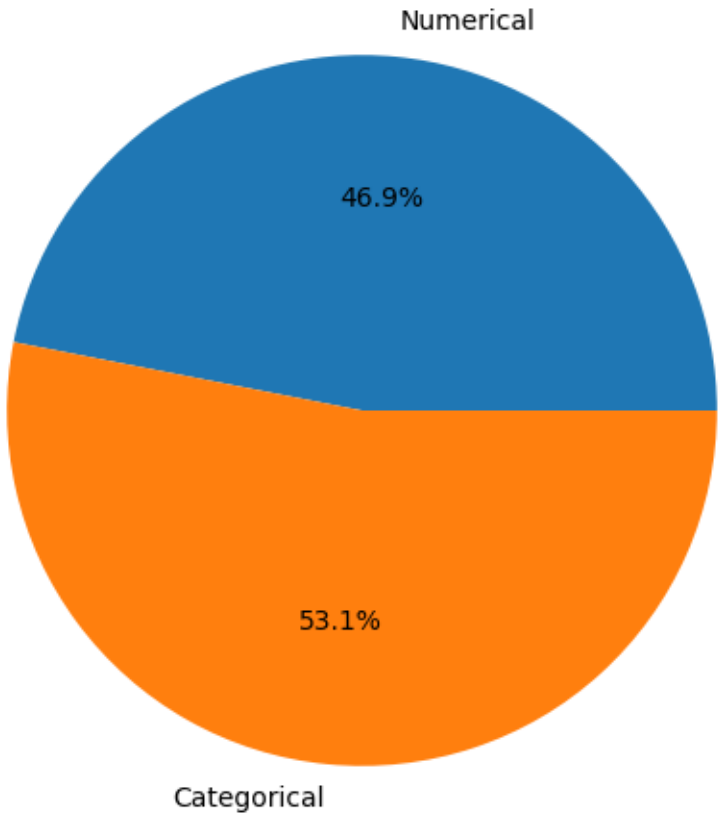
```
Number of numerical features: 38
```

[59]:
```
# number of categorical features
categorical = [var for var in df_train.columns if df_train[var].dtype=='O']
print("Number of categorical features:",len(categorical))
```

```
Number of categorical features: 43
```

[60]:
```
# visualize distribution of features
feat_labels = ["Numerical", "Categorical"]
feat_nums = [38, 43]

plt.figure(figsize=(11,6))
plt.pie(feat_nums, labels=feat_labels, autopct=lambda x: f"{x:.1f}%")
```

[60]:
```
([<matplotlib.patches.Wedge at 0x200242230e0>,
  <matplotlib.patches.Wedge at 0x20024a15810>],
 [Text(0.10649199021272866, 1.0948330722171906, 'Numerical'),
  Text(-0.10649215552792206, -1.094833056137335, 'Categorical')],
 [Text(0.05808654011603381, 0.5971816757548312, '46.9%'),
  Text(-0.05808663028795748, -0.5971816669840009, '53.1%')])
```



[61]:
```
# info of data set
df_train.describe()
```

[61]:
```
                Id   MSSubClass  LotFrontage        LotArea  OverallQual  \
count  1449.000000  1449.000000  1190.000000    1449.000000  1449.000000
mean    725.000000    56.846101    70.107563   10529.050380     6.101449
std     418.434583    42.136377    24.317431   10012.121164     1.385124
min       1.000000    20.000000    21.000000    1300.000000     1.000000
25%     363.000000    20.000000    59.000000    7558.000000     5.000000
50%     725.000000    50.000000    70.000000    9490.000000     6.000000
75%    1087.000000    70.000000    80.000000   11606.000000     7.000000
max    1449.000000   190.000000   313.000000  215245.000000    10.000000

       OverallCond    YearBuilt  YearRemodAdd   MasVnrArea   BsmtFinSF1  …  \
count  1449.000000  1449.000000   1449.000000  1441.000000  1449.000000  …
mean      5.573499  1971.187716   1984.803313   104.204025   444.623879  …
std       1.111971    30.237514     20.667135   181.574227   456.871490  …
min       1.000000  1872.000000   1950.000000     0.000000     0.000000  …
```

```
25%       5.000000  1954.000000   1966.000000     0.000000     0.000000  …
50%       5.000000  1972.000000   1994.000000     0.000000   384.000000  …
75%       6.000000  2000.000000   2004.000000   166.000000   713.000000  …
max       9.000000  2010.000000   2010.000000  1600.000000  5644.000000  …

        WoodDeckSF  OpenPorchSF  EnclosedPorch    3SsnPorch  ScreenPorch  \
count  1449.000000  1449.000000    1449.000000  1449.000000  1449.000000
mean     93.911663    46.706694      22.043478     3.435473    15.175293
std     124.124828    66.444023      61.277989    29.426965    55.953287
min       0.000000     0.000000       0.000000     0.000000     0.000000
25%       0.000000     0.000000       0.000000     0.000000     0.000000
50%       0.000000    24.000000       0.000000     0.000000     0.000000
75%     168.000000    68.000000       0.000000     0.000000     0.000000
max     857.000000   547.000000     552.000000   508.000000   480.000000

          PoolArea       MiscVal       MoSold       YrSold      SalePrice
count  1449.000000   1449.000000  1449.000000  1449.000000    1449.000000
mean      2.779848     42.093858     6.322291  2007.812974  181003.610076
std      40.328903    493.786841     2.706715     1.325566   79558.841800
min       0.000000      0.000000     1.000000  2006.000000   34900.000000
25%       0.000000      0.000000     5.000000  2007.000000  129900.000000
50%       0.000000      0.000000     6.000000  2008.000000  163000.000000
75%       0.000000      0.000000     8.000000  2009.000000  214000.000000
max     738.000000  15500.000000    12.000000  2010.000000  755000.000000

[8 rows x 38 columns]
```

[62]:
```python
# find columns with missing values
missing = df_train.isna().sum()
missing_vals = missing[missing > 0]
missing_vals
```

[62]:
```
LotFrontage      259
Alley           1359
MasVnrType       864
MasVnrArea         8
BsmtQual          37
BsmtCond          37
BsmtExposure      38
BsmtFinType1      37
BsmtFinType2      38
Electrical         1
FireplaceQu      683
GarageType        78
GarageYrBlt       78
GarageFinish      78
GarageQual        78
```

```
GarageCond        78
PoolQC          1442
Fence           1170
MiscFeature     1396
dtype: int64
```

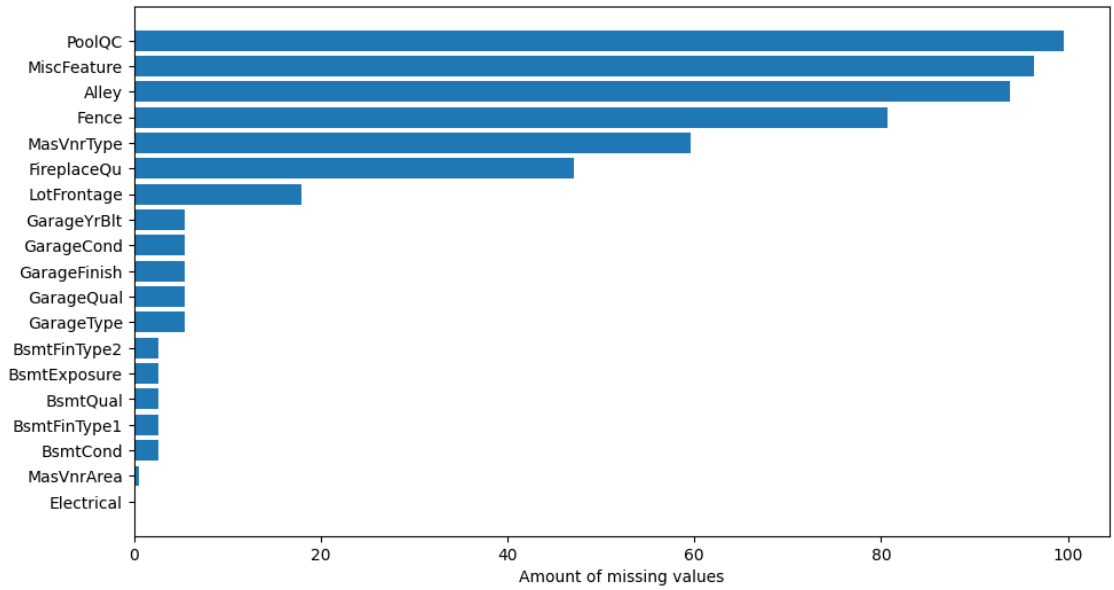
[63]:
```
# percentages of values missing in each column
missing_vals_p = (missing_vals/1449)*100
missing_vals_p
```

[63]:
```
LotFrontage     17.874396
Alley           93.788820
MasVnrType      59.627329
MasVnrArea       0.552105
BsmtQual         2.553485
BsmtCond         2.553485
BsmtExposure     2.622498
BsmtFinType1     2.553485
BsmtFinType2     2.622498
Electrical       0.069013
FireplaceQu     47.135956
GarageType       5.383023
GarageYrBlt      5.383023
GarageFinish     5.383023
GarageQual       5.383023
GarageCond       5.383023
PoolQC          99.516908
Fence           80.745342
MiscFeature     96.342305
dtype: float64
```

While some columns are only missing a small amount of data, a few columns such as 'Alley', 'PoolQC', 'Fence', and 'MiscFeature' are missing a very large amount of values. There are some missing close to or more than half the data such as MasVnrType and FirePlaceQu.

[64]:
```
# visualize missing data with a bar plot
sorted_missing = missing_vals_p.sort_values()

plt.figure(figsize=(11,6))
plt.barh(sorted_missing.index, sorted_missing.values)
plt.xlabel('Amount of missing values')
```

[64]:
```
Text(0.5, 0, 'Amount of missing values')
```

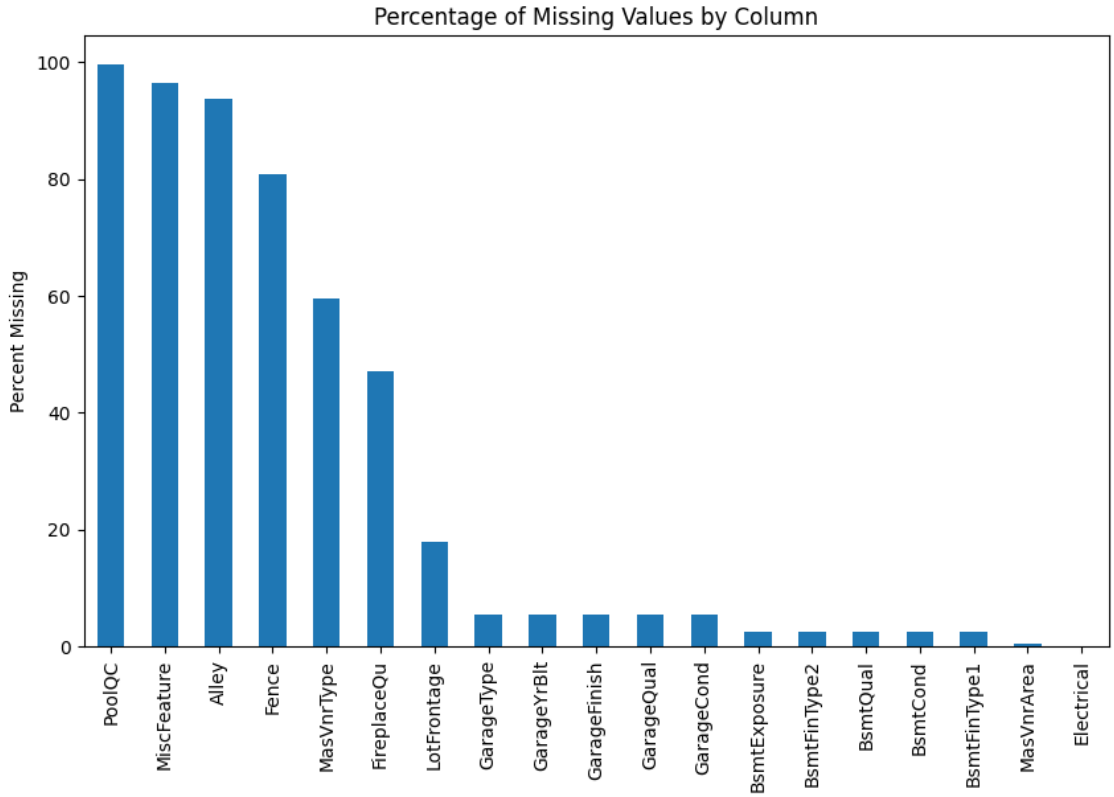


[65]:
```
# check top three categories missing values
print(df_train["PoolQC"].value_counts())
print("\n",df_train["MiscFeature"].value_counts())
print("\n", df_train["Alley"].value_counts())
```

```
PoolQC
Gd    3
Ex    2
Fa    2
Name: count, dtype: int64

 MiscFeature
Shed    48
Gar2     2
Othr     2
TenC     1
Name: count, dtype: int64

 Alley
Grvl    50
Pave    40
Name: count, dtype: int64
```

[196]:
```
missing_percent = df_train.isna().mean()*100
missing_percent = missing_percent[missing_percent > 0].
 ↪sort_values(ascending=False)

plt.figure(figsize=(10,6))
```

```
missing_percent.plot(kind='bar')
plt.ylabel('Percent Missing')
plt.title('Percentage of Missing Values by Column')
plt.show()
```



In addition to the missingness itself, the percentage of missing values is hihgly uneven across variables. This suggests different filling strategies are needed depending on the meaning of each feature.

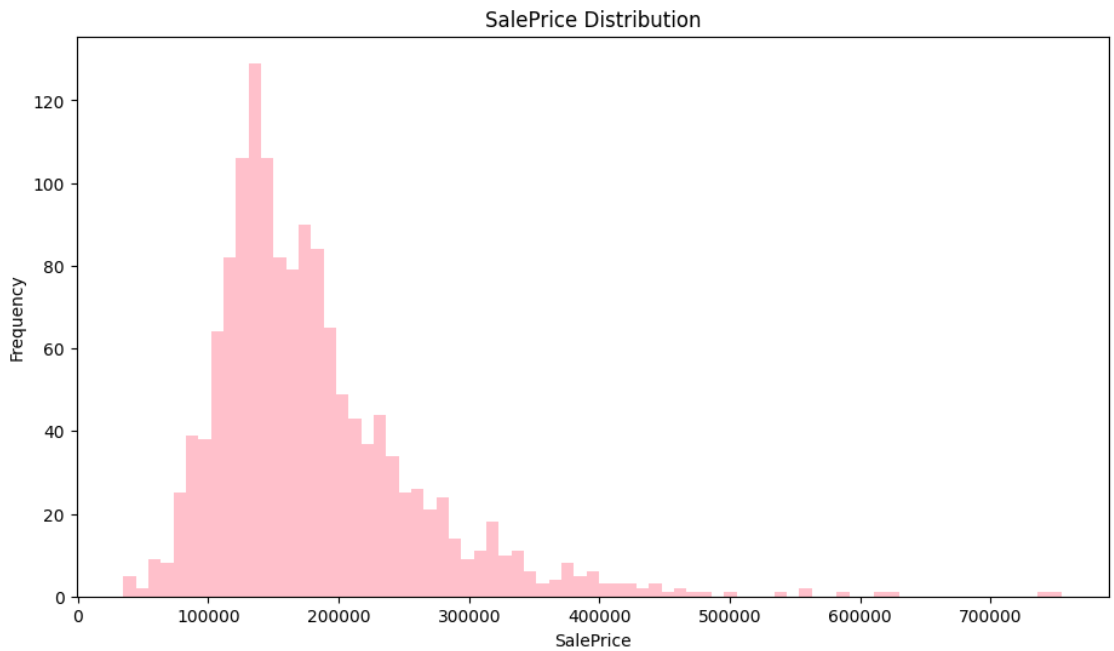
1.4.3 Question C-3

[67]:
```
# value counts SalePrice
df_train['SalePrice'].value_counts()
```

[67]:
```
SalePrice
140000    20
135000    17
155000    14
110000    13
190000    13
          ..
121000     1
```

```
181500     1
156500     1
72500      1
307000     1
Name: count, Length: 658, dtype: int64
```

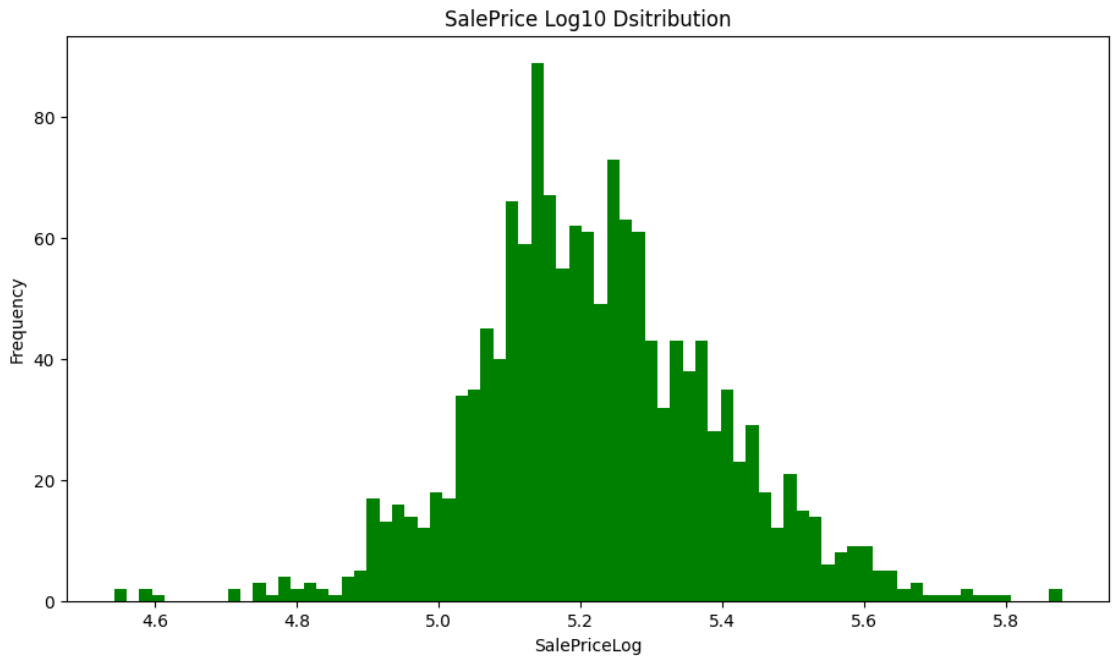
[68]:
```
# visualize with histogram
plt.figure(figsize=(11,6))
plt.hist(df_train['SalePrice'], bins=75, color = 'pink')
plt.title('SalePrice Distribution')
plt.xlabel('SalePrice')
plt.ylabel("Frequency")
plt.show()
```



In addition to the missing values and the distribution of numerical and categorical values, this distribution of the frequency of sales prices is right-skewed, with a long upper tail and quite a few expensive outliers. This suggests a log transformation might make the data more symmetric.

[69]:
```
# new sale price log column
df_train['SalePriceLog'] = np.log10(df_train['SalePrice'])

# histogram
plt.figure(figsize=(11,6))
plt.hist(df_train['SalePriceLog'], bins=75, color='green')
plt.title('SalePrice Log10 Dsitribution')
plt.xlabel('SalePriceLog')
```

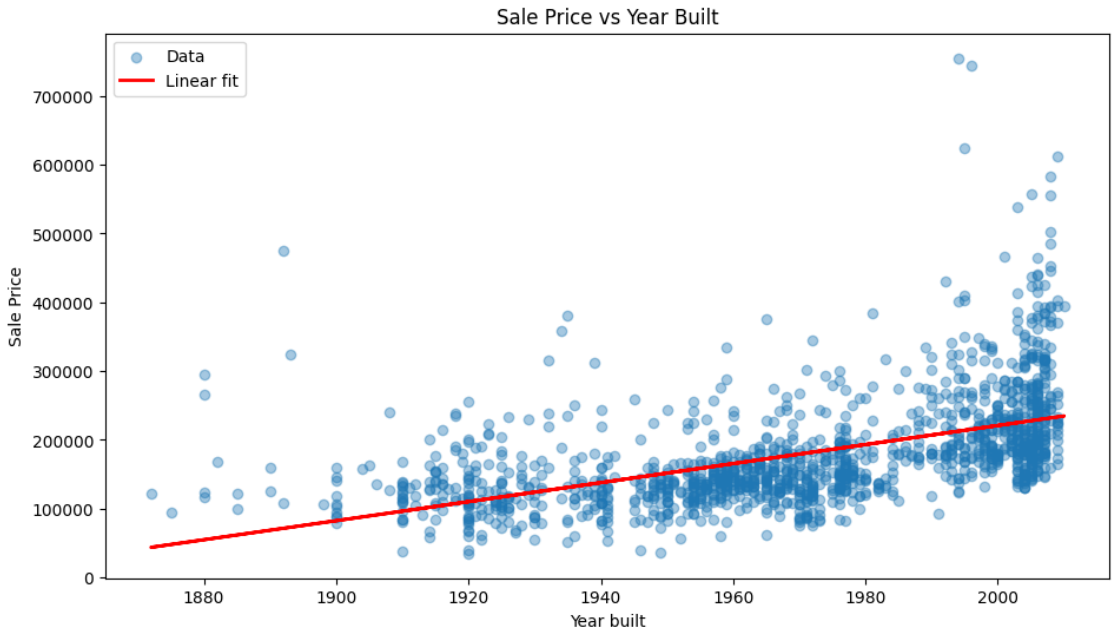
```
plt.ylabel("Frequency")
plt.show()
```



This transformation reduces the right-skewness seen in the graph before and makes the distribution more symmetric.

1.4.4 Question C-4

[71]:
```
# visualize with scatter plot
plt.figure(figsize=(11,6))
plt.scatter(df_train['YearBuilt'], df_train['SalePrice'], alpha=0.4,␣
 ↪label='Data')
z = np.polyfit(df_train['YearBuilt'], df_train['SalePrice'], 1)
p = np.poly1d(z)
plt.plot(df_train['YearBuilt'], p(df_train['YearBuilt']), color='red',␣
 ↪linewidth=2, label='Linear fit')
plt.xlabel('Year built')
plt.ylabel('Sale Price')
plt.title('Sale Price vs Year Built')
plt.legend()
plt.show()
```



The fitted line shows a positive overall relationship between 'YearBuilt' and 'SalePrice': newer houses seem to sell more. While the number of houses in this data set increases in more recent years, the prices only increase a little for slighlty newer houses. There seems to be a steeper increase in sale prices for houses built after the 1990s, which is when the most expensive houses seem to have been built. A lot of houses are built after 2000 and the mean of the prices seems to be clearly higher than in previous years as there are a lot more data points in the 250 000 to 500 000 range than before.

1.4.5 Question C-5

[72]:
```
# data types
print(df_train['MSZoning'].dtype)
print(df_train['MSSubClass'].dtype)
print(df_train['YearBuilt'].dtype)
print(df_train['YearRemodAdd'].dtype)
print(df_train['GarageType'].dtype)
print(df_train['Fence'].dtype)
print(df_train['SaleCondition'].dtype)
```

```
object
int64
int64
int64
object
object
object
```

[73]:
```
# separate numerical and categorical columns
num_c = ['MSSubClass', 'YearBuilt', 'YearRemodAdd']
cat_c = ['GarageType', 'Fence', 'SaleCondition']

# df with means of numerical features
num_means = df_train.groupby('MSZoning')[num_c].mean()

# df with modes of categorical features
cat_modes = df_train.groupby('MSZoning')[cat_c].apply(lambda x: x.mode().
 ↪iloc[0])
```

[74]:
```
# join dataframes
df_by_zone = num_means.join(cat_modes)
```

[75]:
```
print(df_by_zone)
```

```
          MSSubClass    YearBuilt  YearRemodAdd GarageType  Fence  \
MSZoning
C (all)    56.000000  1924.100000   1958.200000     Detchd  MnPrv
FV         91.562500  2004.515625   2004.828125     Attchd   GdWo
RH         80.312500  1945.687500   1971.937500     Detchd  MnPrv
RL         49.440070  1975.446194   1985.608049     Attchd  MnPrv
RM         84.050926  1942.847222   1976.796296     Detchd  MnPrv

         SaleCondition
MSZoning
C (all)        Abnorml
FV              Normal
RH              Normal
RL              Normal
RM              Normal
```

1.4.6 Question C-6

[76]:
```
# display missing values
missing_vals_p
```

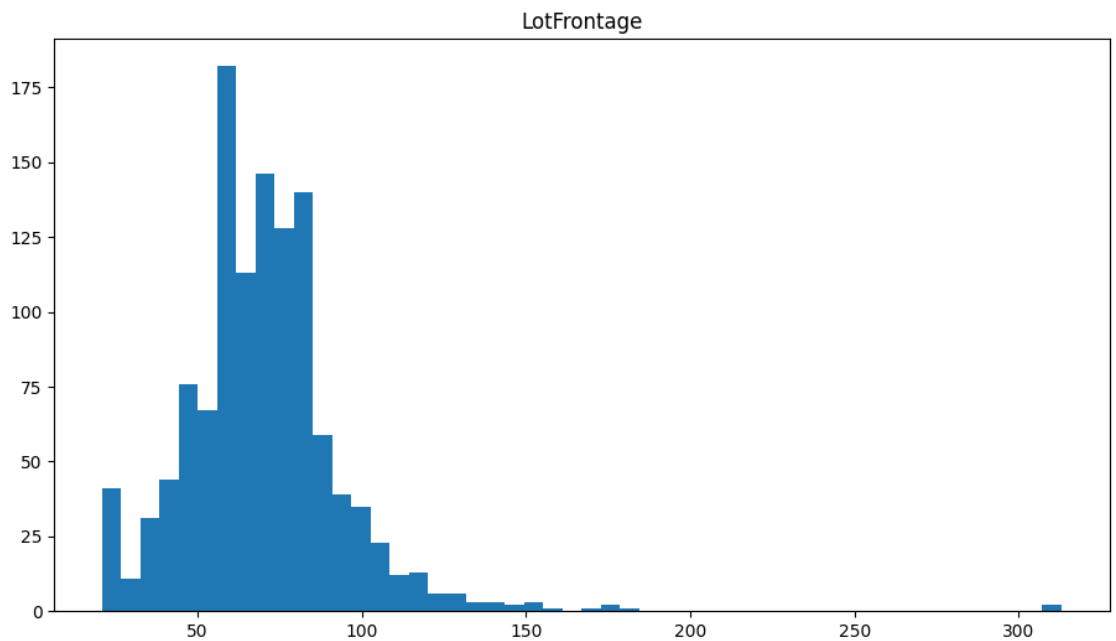
[76]:
```
LotFrontage     17.874396
Alley           93.788820
MasVnrType      59.627329
MasVnrArea       0.552105
BsmtQual         2.553485
BsmtCond         2.553485
BsmtExposure     2.622498
BsmtFinType1     2.553485
BsmtFinType2     2.622498
Electrical       0.069013
FireplaceQu     47.135956
```

```
GarageType       5.383023
GarageYrBlt      5.383023
GarageFinish     5.383023
GarageQual       5.383023
GarageCond       5.383023
PoolQC          99.516908
Fence           80.745342
MiscFeature     96.342305
dtype: float64
```

[77]:
```
# copy df
df_train_filled = df_train.copy()
```

LotFrontage: Only 17% of the data is missing which means the linear feet of street connected to the property was probably not measured.

[78]:
```
ex_lotFrontage = df_train_filled[df_train_filled['LotFrontage'].notna()]
plt.figure(figsize=(11,6))
plt.hist(ex_lotFrontage['LotFrontage'], bins=50)
plt.title('LotFrontage')
plt.show()
```



The data is right-skewed so I will replace the missing values by the median so that the distribution can be a little more symmetric and decrease the effect extreme values have on it.

[79]:
```
lotFr_median = (ex_lotFrontage['LotFrontage']).median()
print(lotFr_median)
```

```
70.0
```

[80]:
```
# replace values
df_train_filled.loc[df_train_filled['LotFrontage'].isna(),'LotFrontage'] =␣
↪lotFr_median
```

[81]:
```
# check
df_train_filled['LotFrontage'].isna().sum()
```

[81]:
```
np.int64(0)
```

Alley: 93% of the data is missing but the feature 'NA' is given to properties with no alley access.

[82]:
```
df_train_filled['Alley'].value_counts()
```

[82]:
```
Alley
Grvl    50
Pave    40
Name: count, dtype: int64
```

[83]:
```
# replace with 'None'
df_train_filled.loc[df_train_filled['Alley'].isna(), 'Alley'] = 'None'
```

[84]:
```
# check
print(df_train_filled['Alley'].value_counts())
print("\n",df_train_filled['Alley'].isna().sum())
```

```
Alley
None    1359
Grvl      50
Pave      40
Name: count, dtype: int64

 0
```

MasVnrType: 59% of masonry veneer type is NA, which is probably what was put for properties with no masonry veneer.

[85]:
```
df_train_filled['MasVnrType'].value_counts()
```

[85]:
```
MasVnrType
BrkFace    444
Stone      126
BrkCmn      15
Name: count, dtype: int64
```

[86]:
```
# replace with 'None'
df_train_filled.loc[df_train_filled['MasVnrType'].isna(), 'MasVnrType'] = 'None'
```

[87]:
```
# check
print(df_train_filled['MasVnrType'].value_counts())
print("\n",df_train_filled['MasVnrType'].isna().sum())
```

```
MasVnrType
None       864
BrkFace    444
Stone      126
BrkCmn      15
Name: count, dtype: int64

 0
```

MasVnrArea: Since MasVnrType often indicates no masonry veneer, missing values in MAsVnrArea can be appropriately filled with 0. This preserves the interpretation that the house likely has no masonry veneer.

[89]:
```
# replace values
df_train_filled.loc[df_train_filled['MasVnrArea'].isna(),'MasVnrArea'] = 0
```

[90]:
```
# check
df_train_filled['MasVnrArea'].isna().sum()
```

[90]:
```
np.int64(0)
```

BsmtQual: Has only about 2% of values missing which is likely NA to represent properties with no basement, I will replace them with 'None'.

[91]:
```
df_train_filled['BsmtQual'].value_counts()
```

[91]:
```
BsmtQual
TA    646
Gd    610
Ex    121
Fa     35
Name: count, dtype: int64
```

[92]:
```
# replace with 'None'
df_train_filled.loc[df_train_filled['BsmtQual'].isna(), 'BsmtQual'] = 'None'
```

[93]:
```
# check
print(df_train_filled['BsmtQual'].value_counts())
print("\n",df_train_filled['BsmtQual'].isna().sum())
```

```
BsmtQual
TA      646
Gd      610
Ex      121
None     37
Fa       35
```

```
Name: count, dtype: int64
```

```
0
```

BsmtCond: Has the same percentage as for BsmtQual, likely again for properties with no basements.

[94]:
```
# replace with 'None'
df_train_filled.loc[df_train_filled['BsmtCond'].isna(), 'BsmtCond'] = 'None'
```

[95]:
```
# check
print("\n",df_train_filled['BsmtCond'].isna().sum())
```

```
0
```

BsmtExposure: Has a similar percentage, I will replace NA with 'None' again.

[96]:
```
# replace with 'None'
df_train_filled.loc[df_train_filled['BsmtExposure'].isna(), 'BsmtExposure'] =␣
↪'None'
```

[97]:
```
# check
print("\n",df_train_filled['BsmtExposure'].isna().sum())
```

```
0
```

BsmtFinType1: The rating of basement finished areas has the same percentage, so NA is replaced with 'None'.

[98]:
```
# replace with 'None'
df_train_filled.loc[df_train_filled['BsmtFinType1'].isna(), 'BsmtFinType1'] =␣
↪'None'
```

[99]:
```
# check
print("\n",df_train_filled['BsmtFinType1'].isna().sum())
```

```
0
```

BsmFinType2: Is similar as well, replacing with 'None'.

[100]:
```
# replace with 'None'
df_train_filled.loc[df_train_filled['BsmtFinType2'].isna(), 'BsmtFinType2'] =␣
↪'None'
```

[101]:
```
# check
print("\n",df_train_filled['BsmtFinType2'].isna().sum())
```

```
0
```

Electrical: Electrical system values are barely missing (less than 1%) so it was likely just not recorded, I will replace the missing values with the mode to keep the data similar.

[102]:
```
df_train_filled['Electrical'].value_counts()
```

[102]:
```
Electrical
SBrkr    1324
FuseA      93
FuseF      27
FuseP       3
Mix         1
Name: count, dtype: int64
```

[103]:
```
electrical_mode = df_train['Electrical'].mode()[0]
print(electrical_mode)
```

```
SBrkr
```

[104]:
```
df_train_filled.loc[df_train_filled['Electrical'].isna(), 'Electrical'] =␣
↪electrical_mode
```

[105]:
```
# check
print(df_train_filled['Electrical'].value_counts())
print("\n",df_train_filled['Electrical'].isna().sum())
```

```
Electrical
SBrkr    1325
FuseA      93
FuseF      27
FuseP       3
Mix         1
Name: count, dtype: int64

 0
```

FireplaceQu: Has 47% of values missing. Since NA is used for no fireplace, I will replace it with 'None'.

[106]:
```
# replace with 'None'
df_train_filled.loc[df_train_filled['FireplaceQu'].isna(), 'FireplaceQu'] =␣
↪'None'
```

[107]:
```
# check
print("\n",df_train_filled['FireplaceQu'].isna().sum())
```

```
 0
```

GarageType: Has 5% of data missing, it uses NA for no garage, so replace with 'None'.

[108]:
```
# replace with 'None'
df_train_filled.loc[df_train_filled['GarageType'].isna(), 'GarageType'] = 'None'
```

[109]:
```
# check
print("\n",df_train_filled['GarageType'].isna().sum())
```

```
0
```

For properties with no garages, replace NA in GarageYrBlt with 0 so data is all numeric, GarageFinish, GarageQual,and GarageCond all have the same percentages, NA represents no garage so replace with 'none'.

[110]:
```
# replace with 'None' or 0
df_train_filled.loc[df_train_filled['GarageYrBlt'].isna(), 'GarageYrBlt'] = 0

df_train_filled.loc[df_train_filled['GarageFinish'].isna(), 'GarageFinish'] =␣
↪'None'

df_train_filled.loc[df_train_filled['GarageQual'].isna(), 'GarageQual'] = 'None'

df_train_filled.loc[df_train_filled['GarageCond'].isna(), 'GarageCond'] = 'None'
```

PoolQC: 99% of PoolQC is missing, which would mean 99% of properties don't have a pool, so replace NA with 'none'

[111]:
```
df_train_filled['PoolQC'].value_counts()
```

[111]:
```
PoolQC
Gd    3
Ex    2
Fa    2
Name: count, dtype: int64
```

[112]:
```
# replace with 'None'
df_train_filled.loc[df_train_filled['PoolQC'].isna(), 'PoolQC'] = 'None'
```

[113]:
```
# check
print("\n",df_train_filled['PoolQC'].isna().sum())
```

```
0
```

Fence: 80% of Fence data is missing, 80% of properties must have no fence.

[114]:
```
# replace with 'None'
df_train_filled.loc[df_train_filled['Fence'].isna(), 'Fence'] = 'None'
```

[115]:
```
# check
print("\n",df_train_filled['Fence'].isna().sum())
```

```
0
```

MiscFeature: 96% of the data for miscellaneous feature is missing since if none was observed, it was put down as NA, so replace with none.

[116]:
```
# replace with 'None'
df_train_filled.loc[df_train_filled['MiscFeature'].isna(), 'MiscFeature'] =␣
↪'None'
```

[117]:
```
# check
print("\n",df_train_filled['MiscFeature'].isna().sum())
```

```
0
```

[118]:
```
# columns for filled df
col_filled = ['PoolQC', 'MiscFeature', 'LotFrontage', 'GarageYrBlt',␣
↪'MasVnrArea']
#indices for filled df
indices_filled = [234, 287]

print(df_train_filled.loc[indices_filled, col_filled])
```

```
    PoolQC MiscFeature  LotFrontage  GarageYrBlt  MasVnrArea
234   None        None         70.0       2002.0         0.0
287   None        None         70.0          0.0         0.0
```

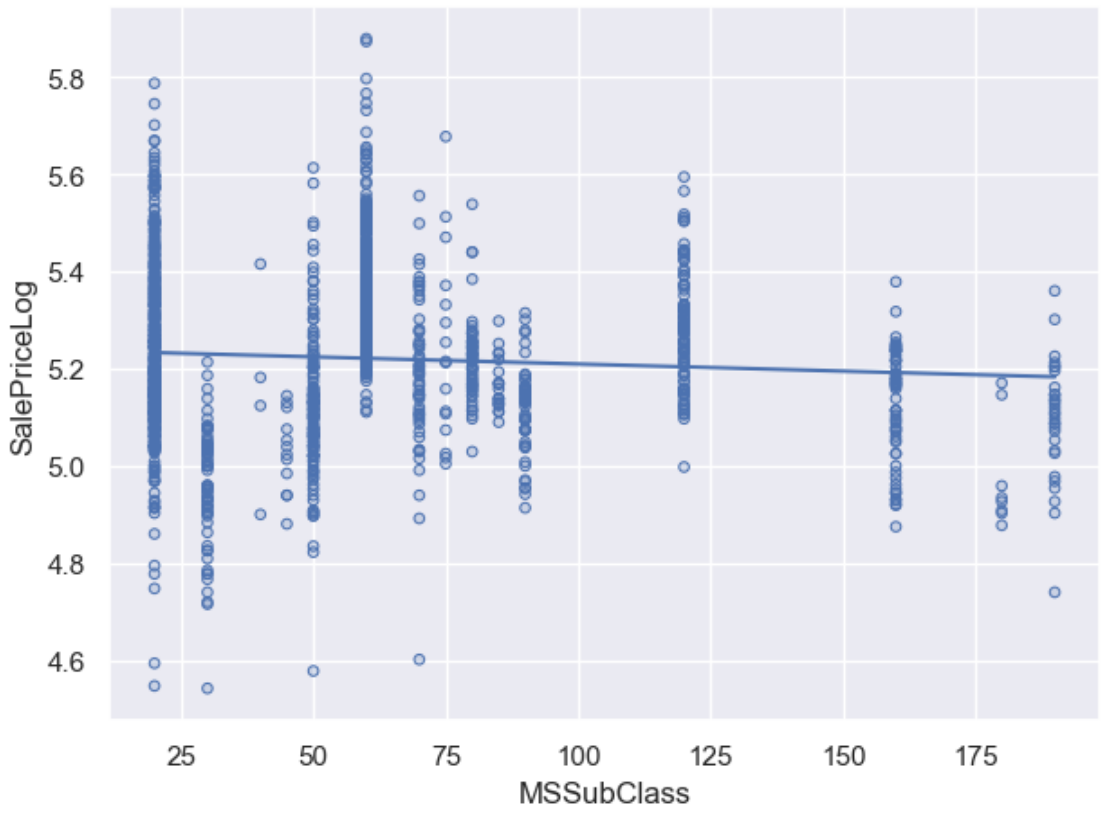
1.4.7 Question C-7

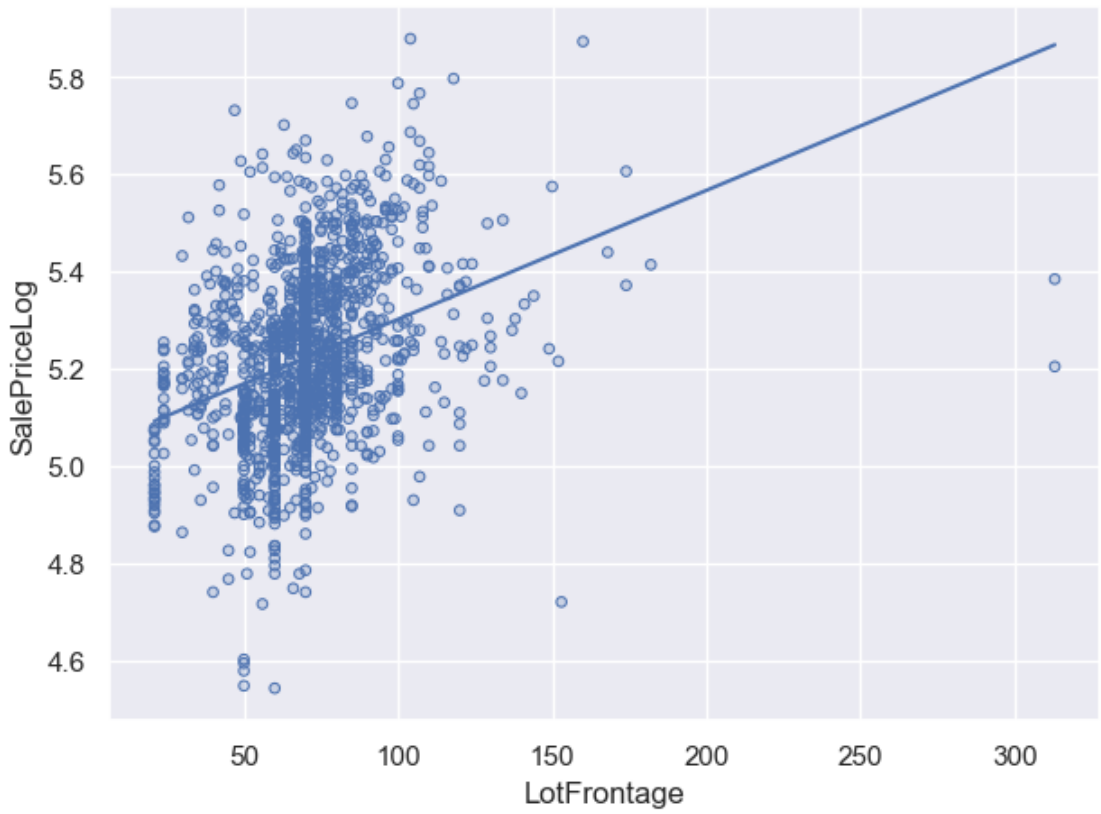
[119]:
```
# select relevant columns
num_relevant = [n for n in numerical if n not in ['SalePrice', 'Id',␣
↪'SalePriceLog']]

# filled dataframe using just relevant columns
df_numerical = df_train_filled[num_relevant + ['SalePriceLog']]
```

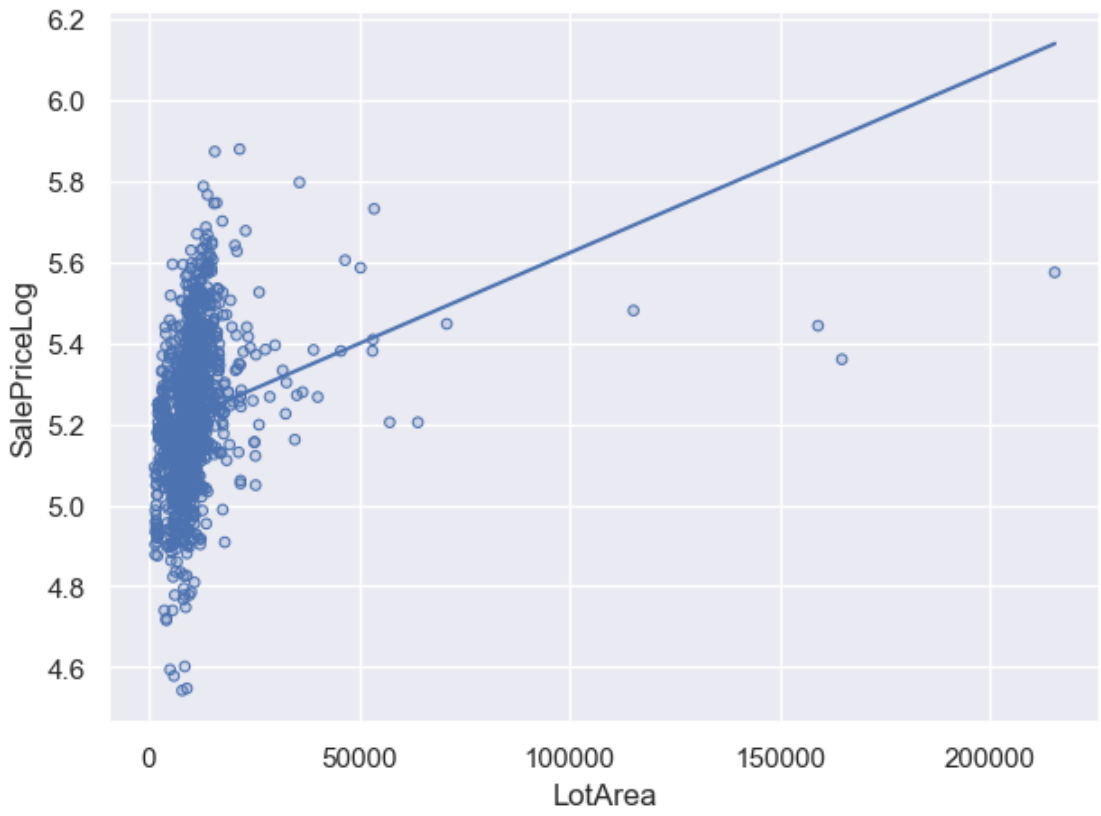
I am removing values starting at 0 because, for some properties, if they are missing a certain feature, a zero is put there instead which skews the relationship shown in the plots.

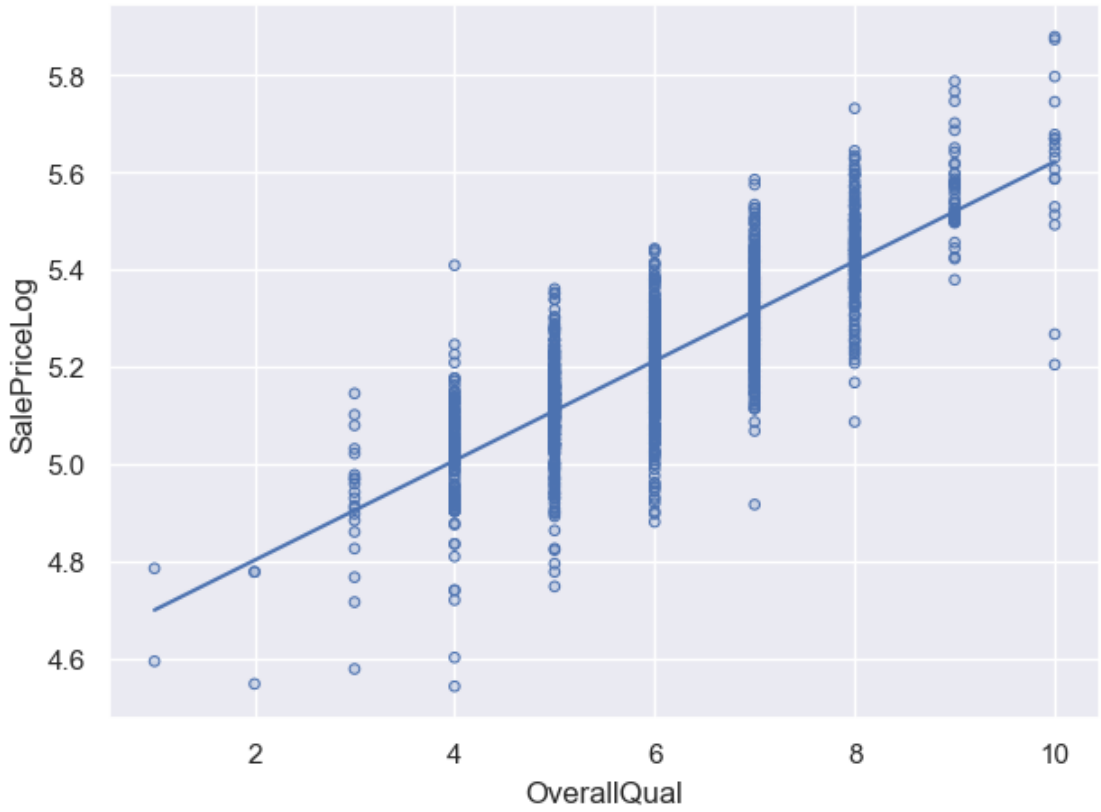
[120]:
```
plt.figure(figsize = (3,3))

for n in num_relevant:
    # exclude values at zero
    df_ex_0 = df_numerical[df_numerical[n] != 0]
    so.Plot(df_ex_0, x=n, y='SalePriceLog').add(so.Dots()).add(so.Line(), so.
↪PolyFit(order=1)).show()
```

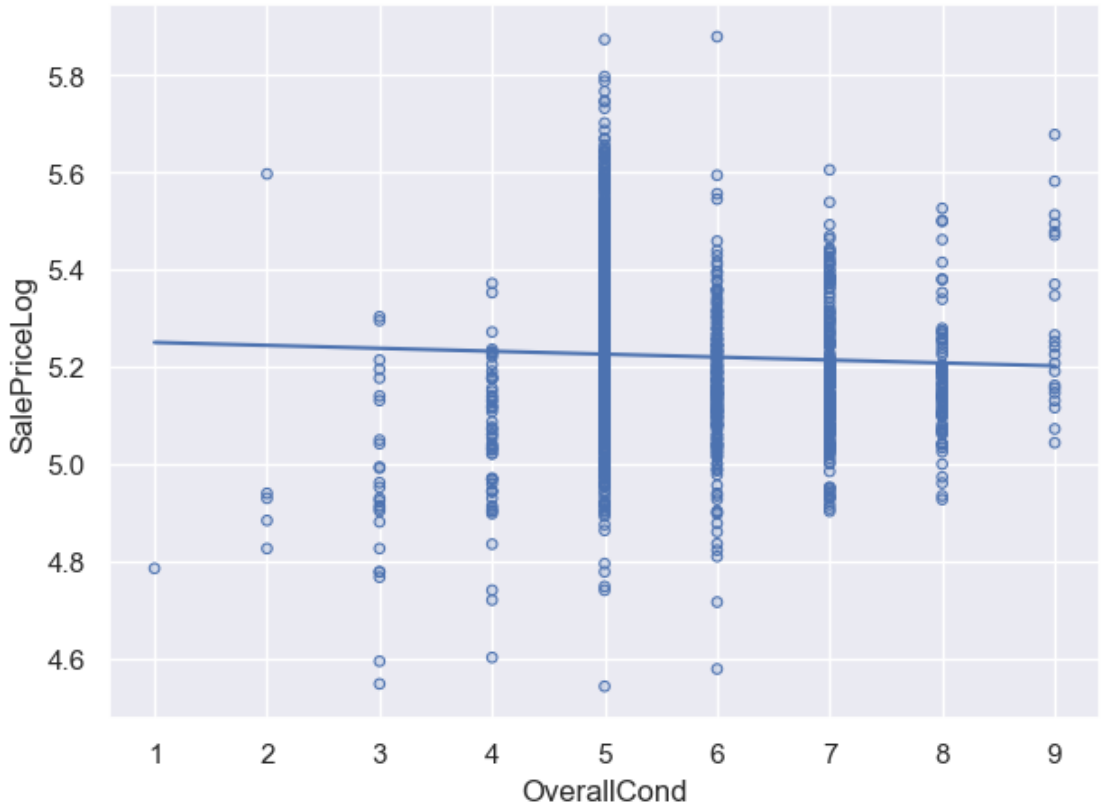
```
<Figure size 300x300 with 0 Axes>
```

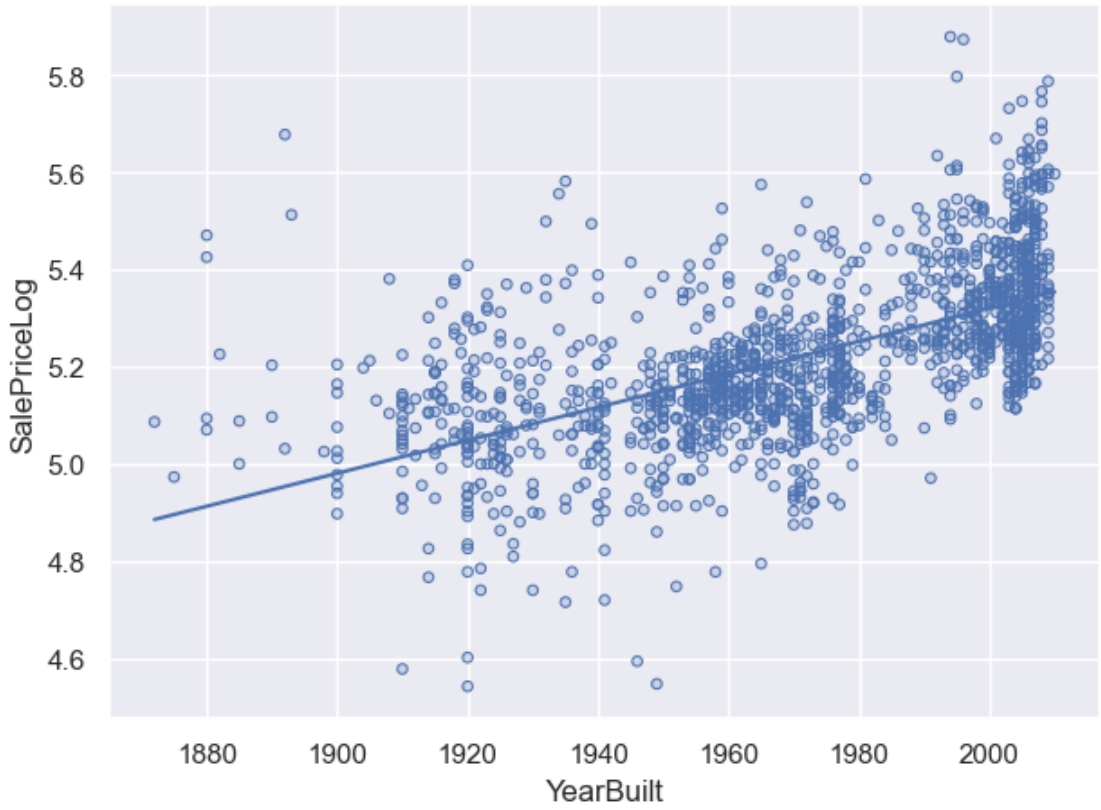

5.8
5.6
5.4
5.2
5.0
4.8
4.6
SalePriceLog
25
50
75
100
125
150
175
MSSubClass

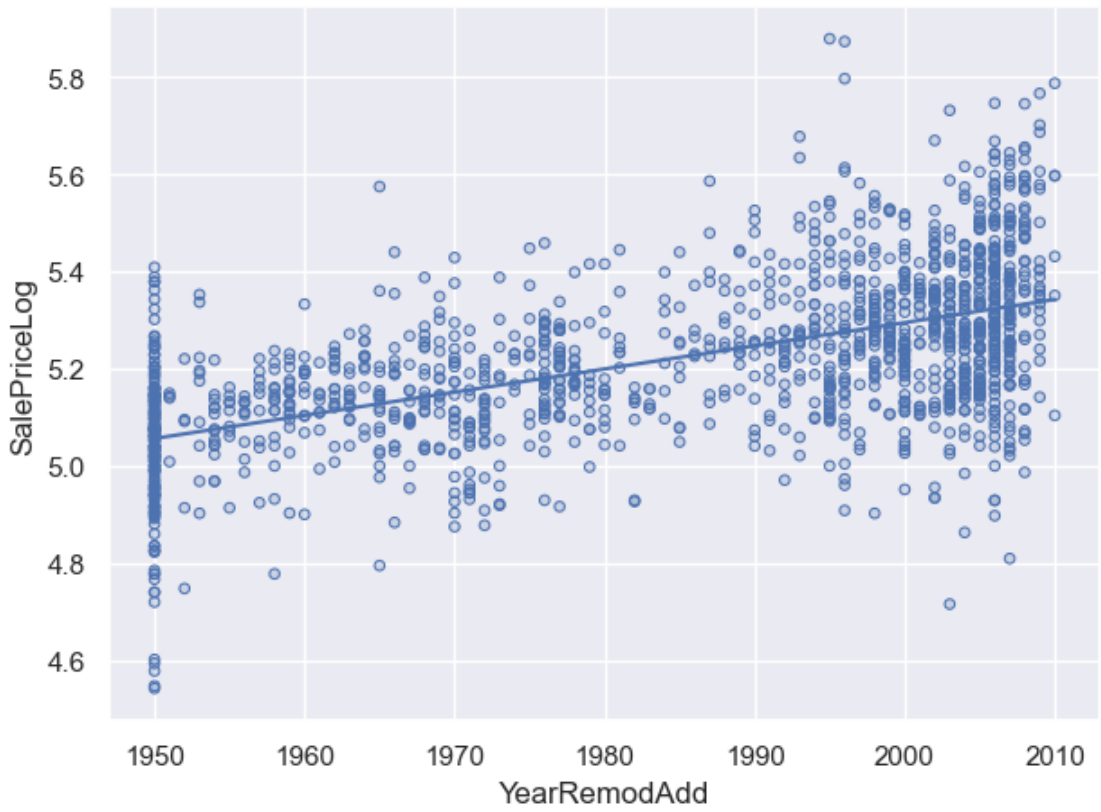

5.8
5.6
5.4
5.2
5.0
4.8
4.6
SalePriceLog
50
100
150
200
250
300
LotFrontage

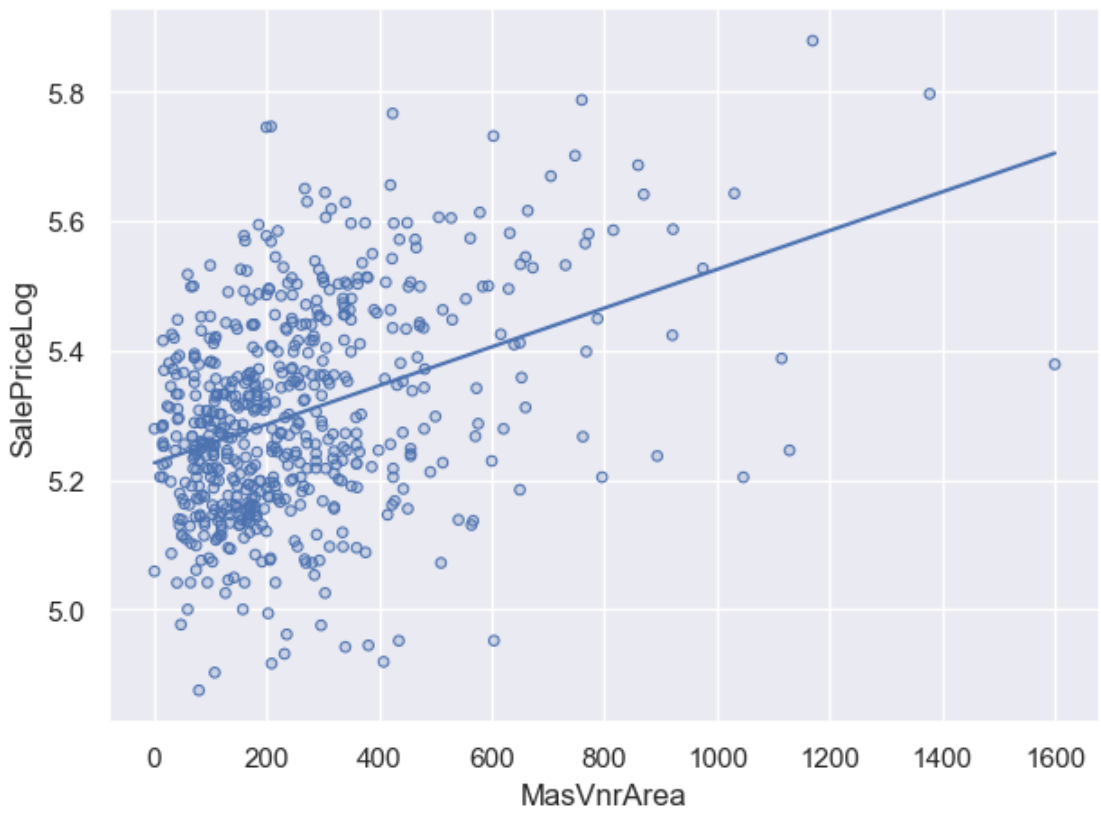

6.2
6.0
5.8
5.6
5.4
5.2
5.0
4.8
4.6
SalePriceLog
0
50000
100000
150000
200000
LotArea

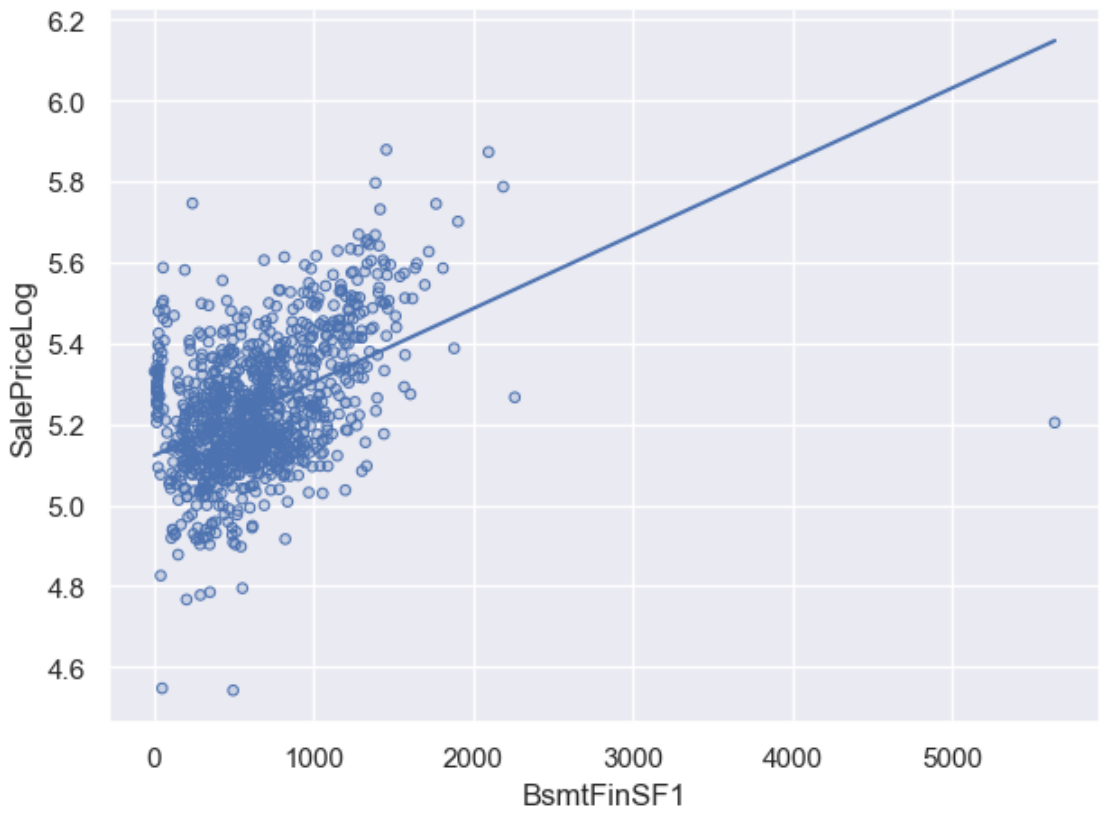

5.8
5.6
5.4
5.2
5.0
4.8
4.6
SalePriceLog
2
4
6
8
10
OverallQual

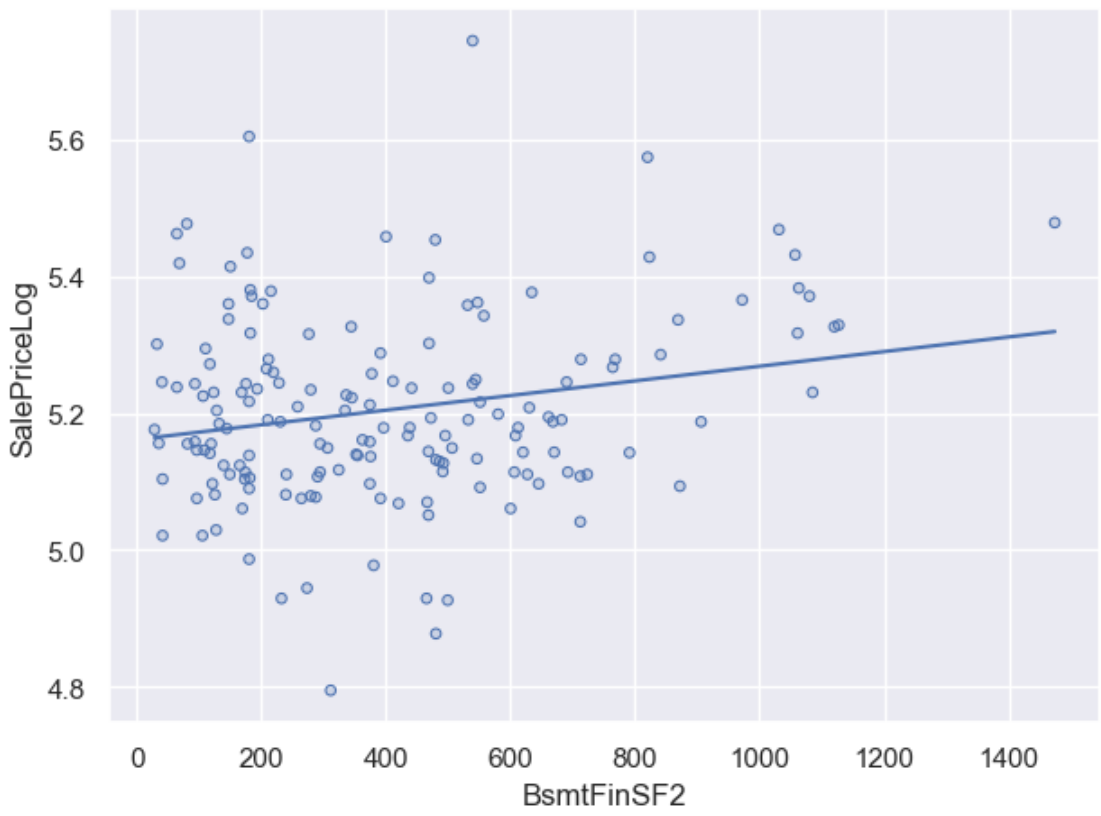

5.8
5.6
5.4
5.2
5.0
4.8
4.6
SalePriceLog
1
2
3
4
5
6
7
8
9
OverallCond

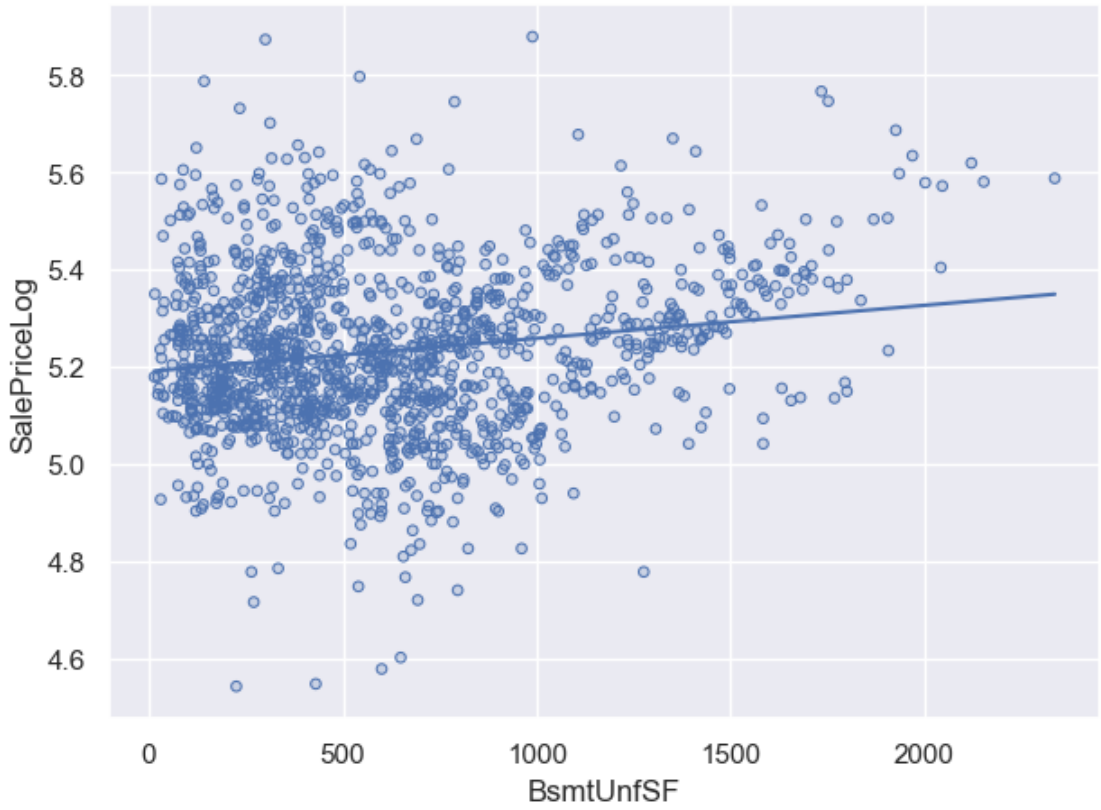

5.8
5.6
5.4
5.2
5.0
4.8
4.6
SalePriceLog
1880
1900
1920
1940
1960
1980
2000
YearBuilt

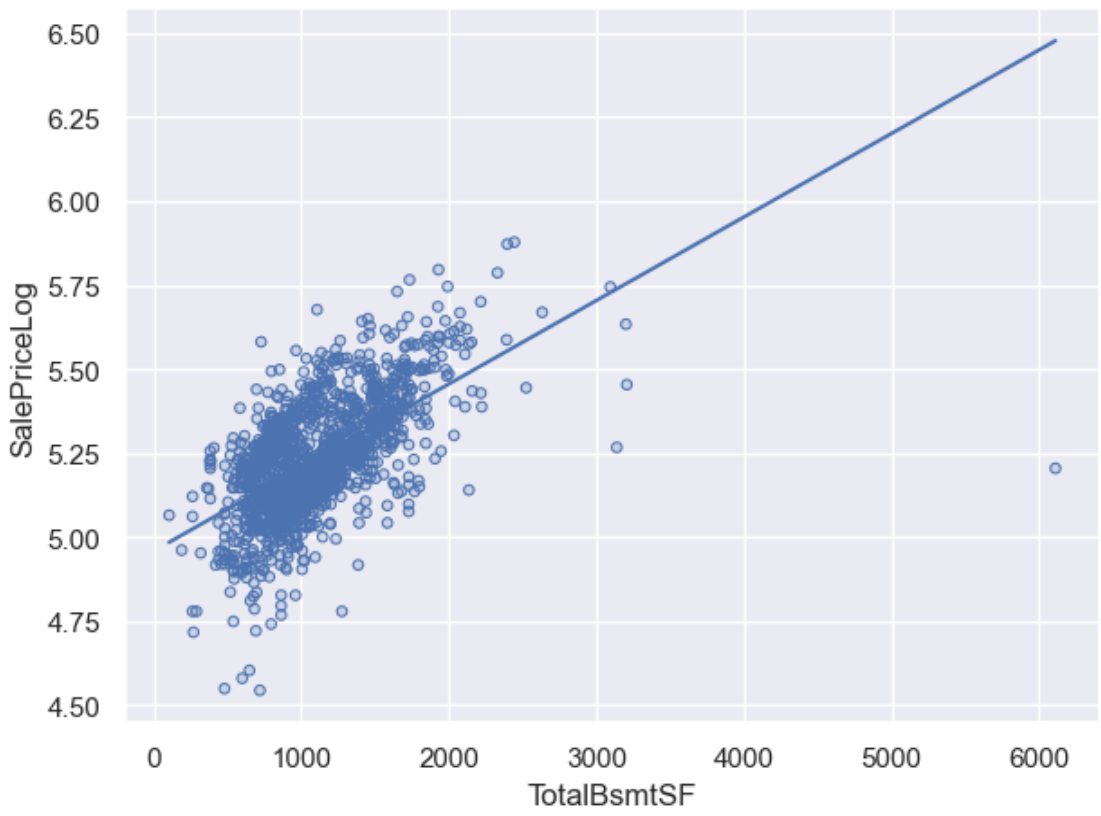

5.8
5.6
5.4
5.2
5.0
4.8
4.6
SalePriceLog
1950
1960
1970
1980
1990
2000
2010
YearRemodAdd

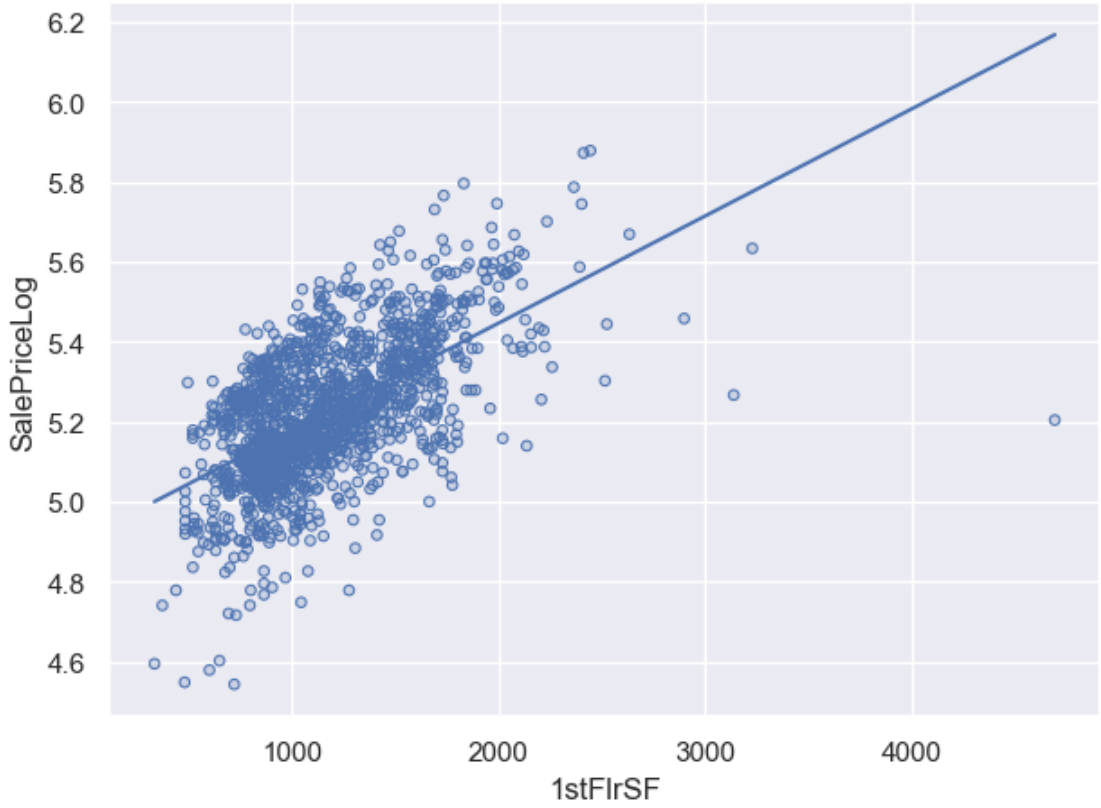

5.8
5.6
5.4
5.2
5.0
SalePriceLog
0
200
400
600
800
1000
1200
1400
1600
MasVnrArea


6.2
6.0
5.8
5.6
5.4
5.2
5.0
4.8
4.6
SalePriceLog
0
1000
2000
3000
4000
5000
BsmtFinSF1

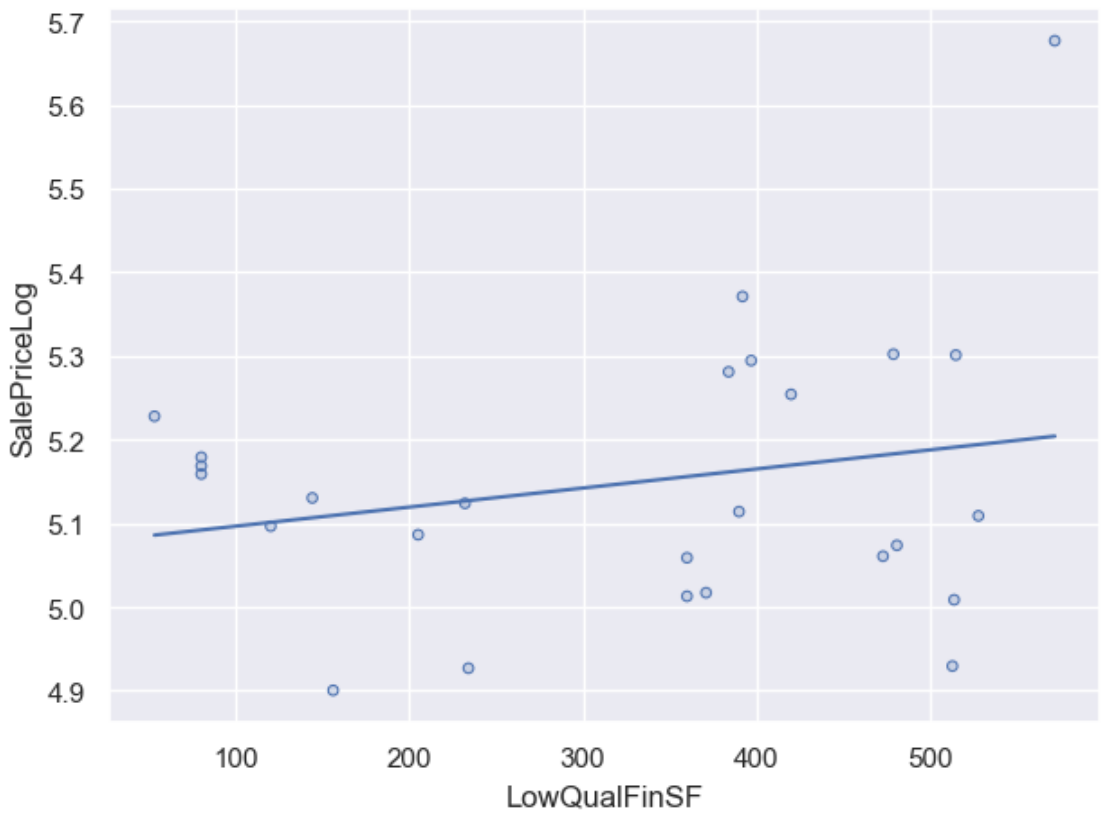

5.6
5.4
5.2
5.0
4.8
SalePriceLog
0
200
400
600
800
1000
1200
1400
BsmtFinSF2

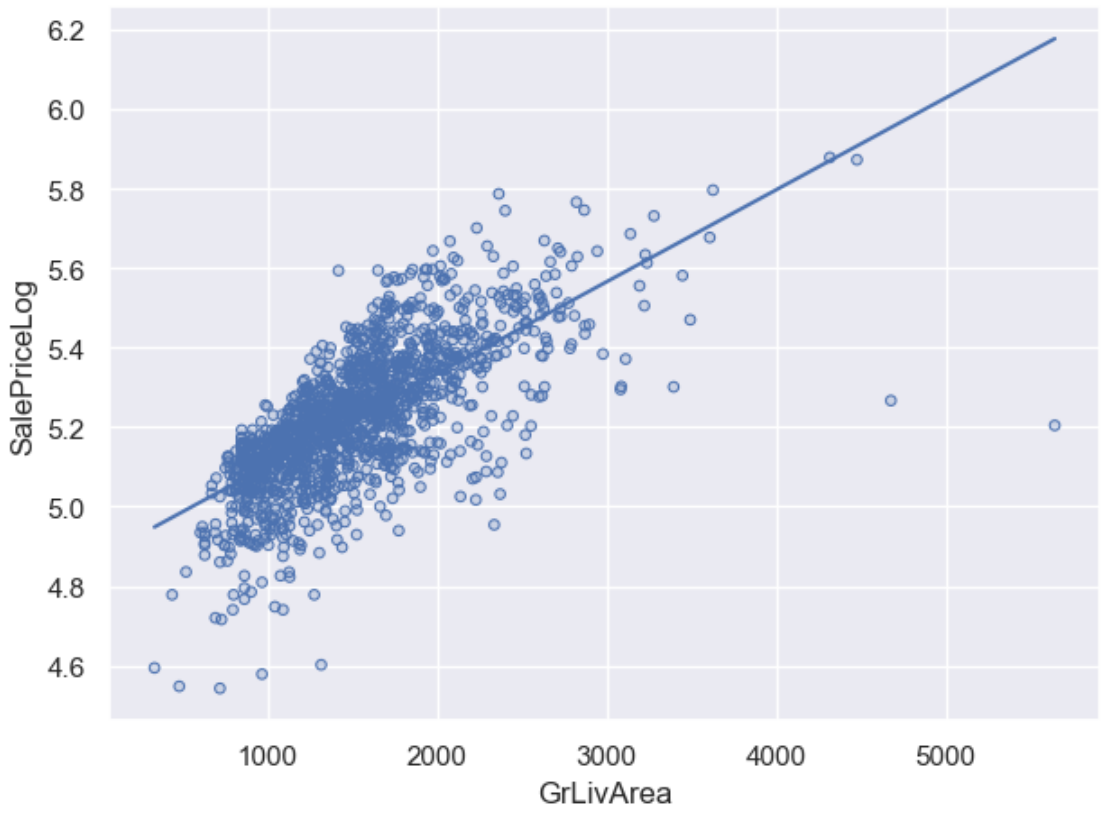

5.8
5.6
5.4
5.2
5.0
4.8
4.6
SalePriceLog
0
500
1000
1500
2000
BsmtUnfSF

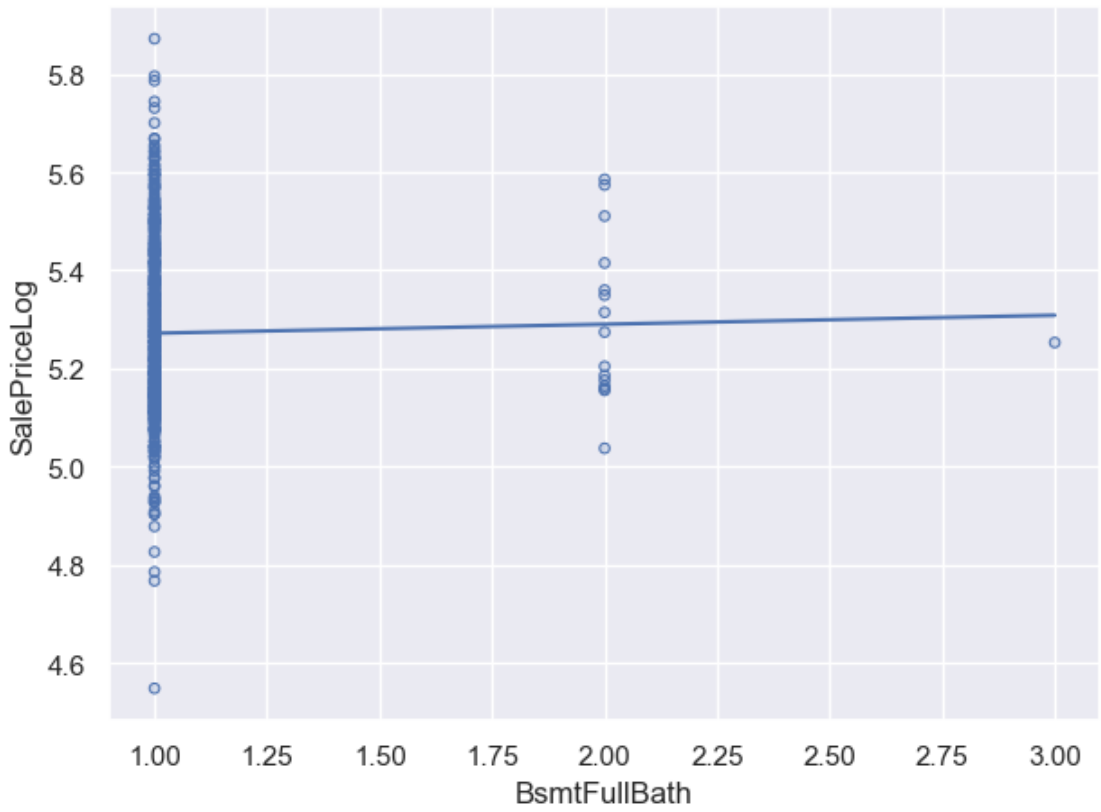

6.50
6.25
6.00
5.75
5.50
5.25
5.00
4.75
4.50
SalePriceLog
0
1000
2000
3000
4000
5000
6000
TotalBsmtSF

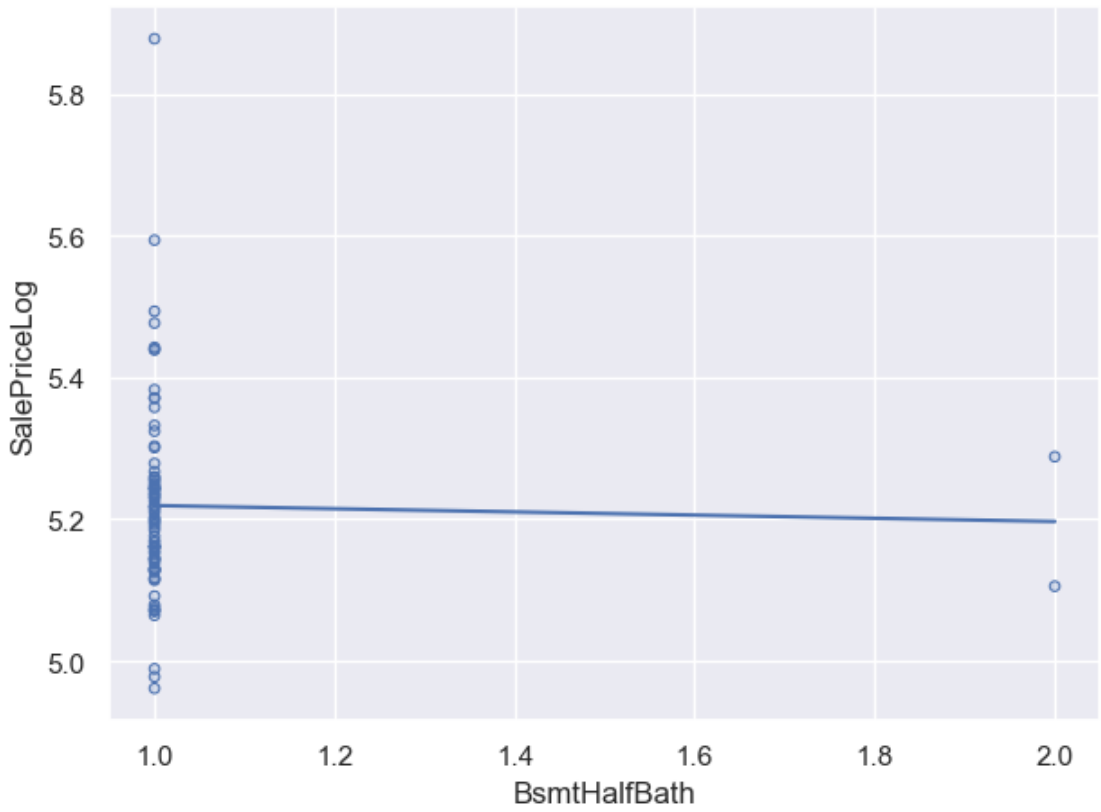

6.2
6.0
5.8
5.6
5.4
5.2
5.0
4.8
4.6
SalePriceLog
1000
2000
3000
4000
1stFlrSF

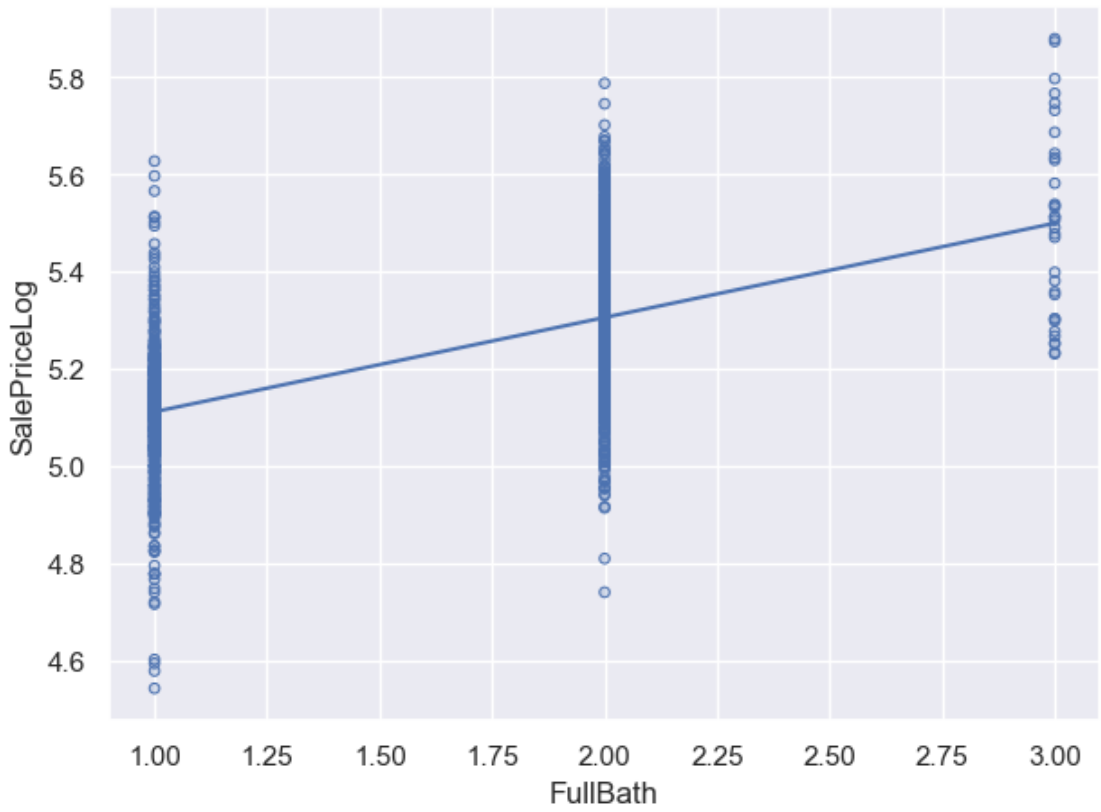
5.8
5.6
5.4
5.2
5.0
4.8
4.6
SalePriceLog
250
500
750
1000
1250
1500
1750
2000
2ndFlrSF

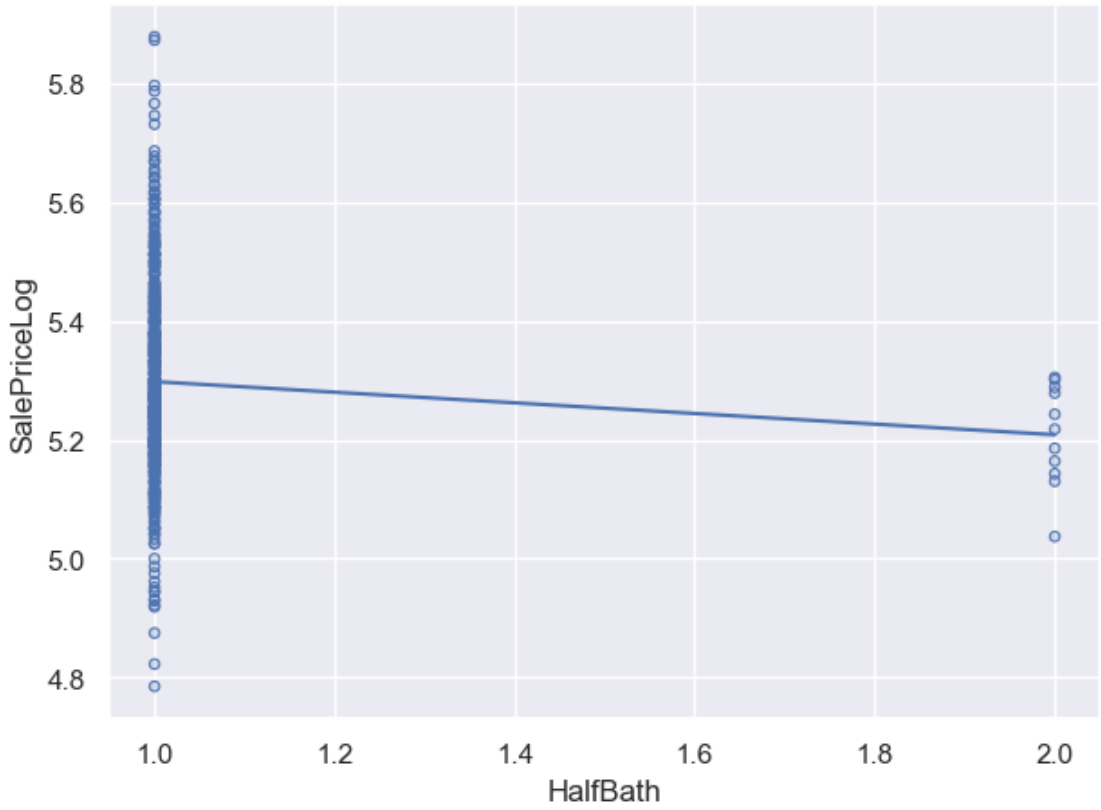

5.7
5.6
5.5
5.4
5.3
5.2
5.1
5.0
4.9
SalePriceLog
100
200
300
400
500
LowQualFinSF

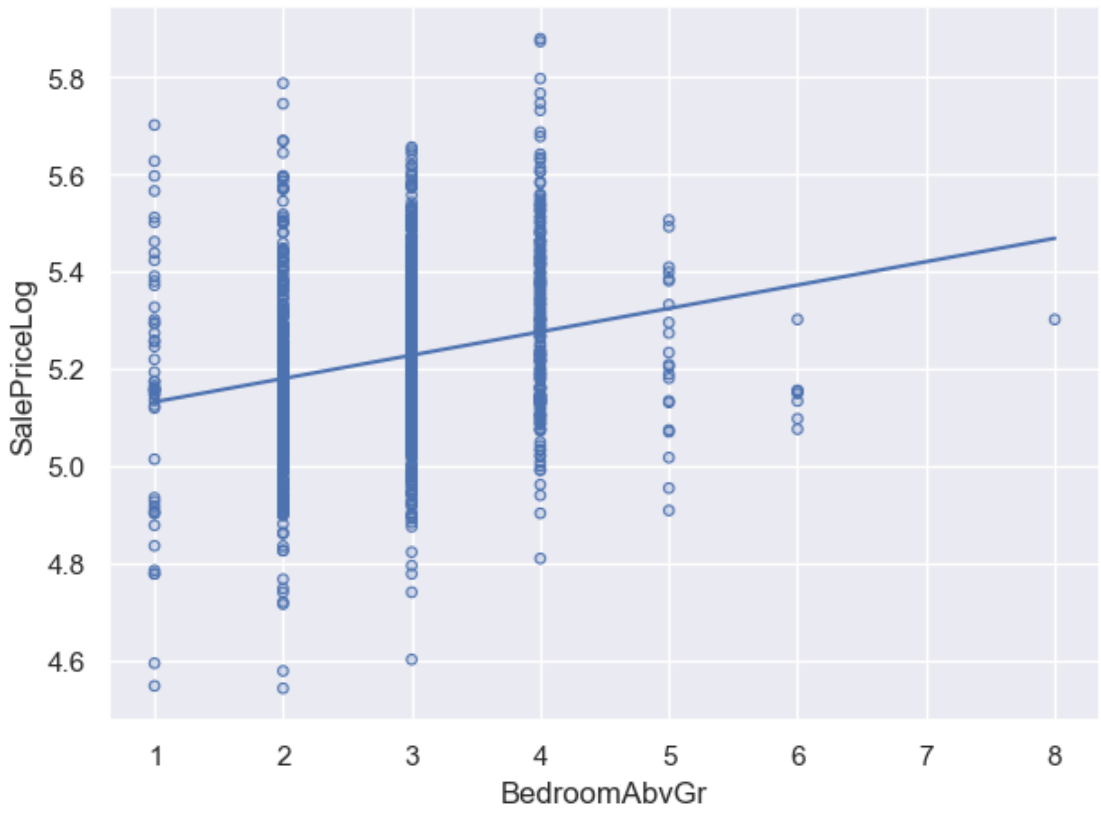

6.2
6.0
5.8
5.6
5.4
5.2
5.0
4.8
4.6
SalePriceLog
1000
2000
3000
4000
5000
GrLivArea

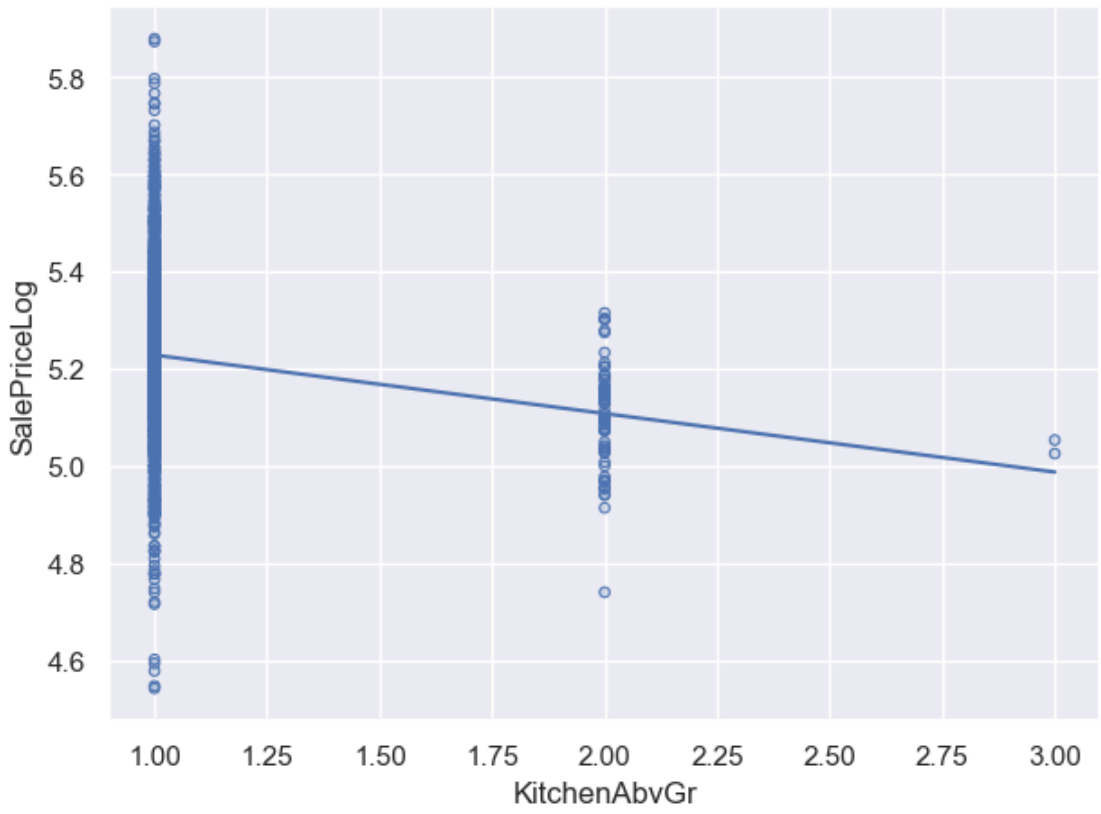

5.8
5.6
5.4
5.2
5.0
4.8
4.6
SalePriceLog
1.00
1.25
1.50
1.75
2.00
2.25
2.50
2.75
3.00
BsmtFullBath

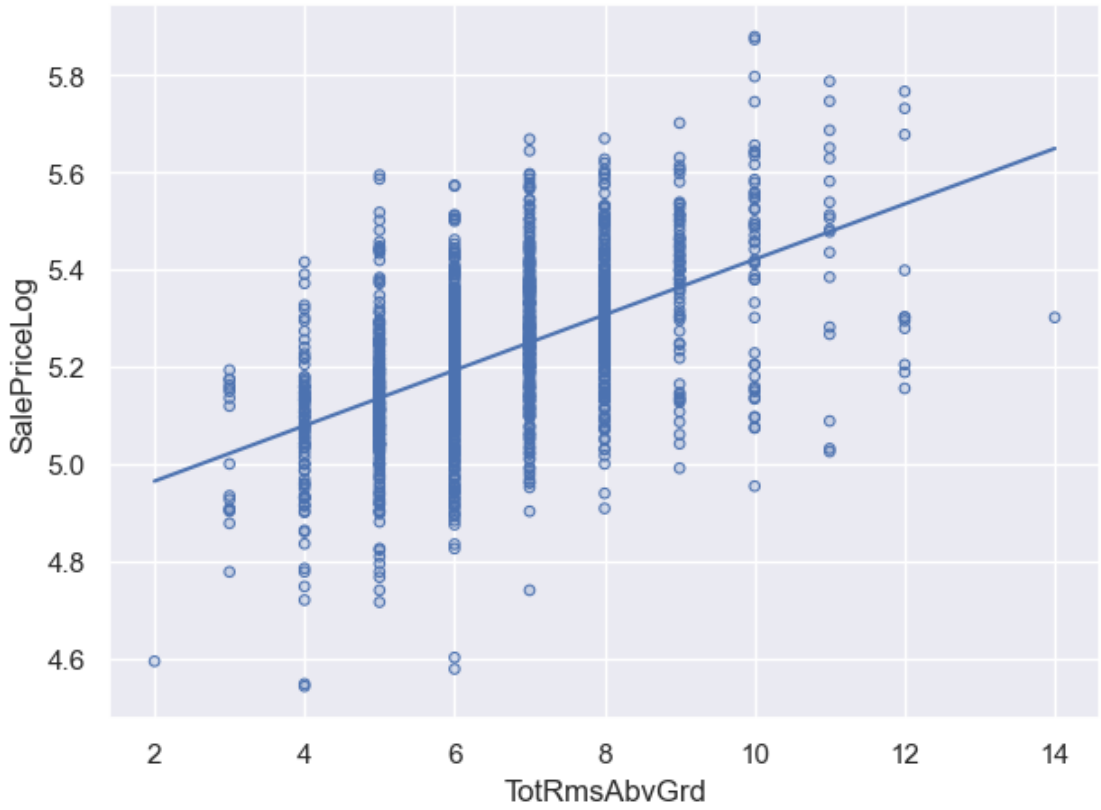

5.8
5.6
5.4
5.2
5.0
SalePriceLog
1.0
1.2
1.4
1.6
1.8
2.0
BsmtHalfBath

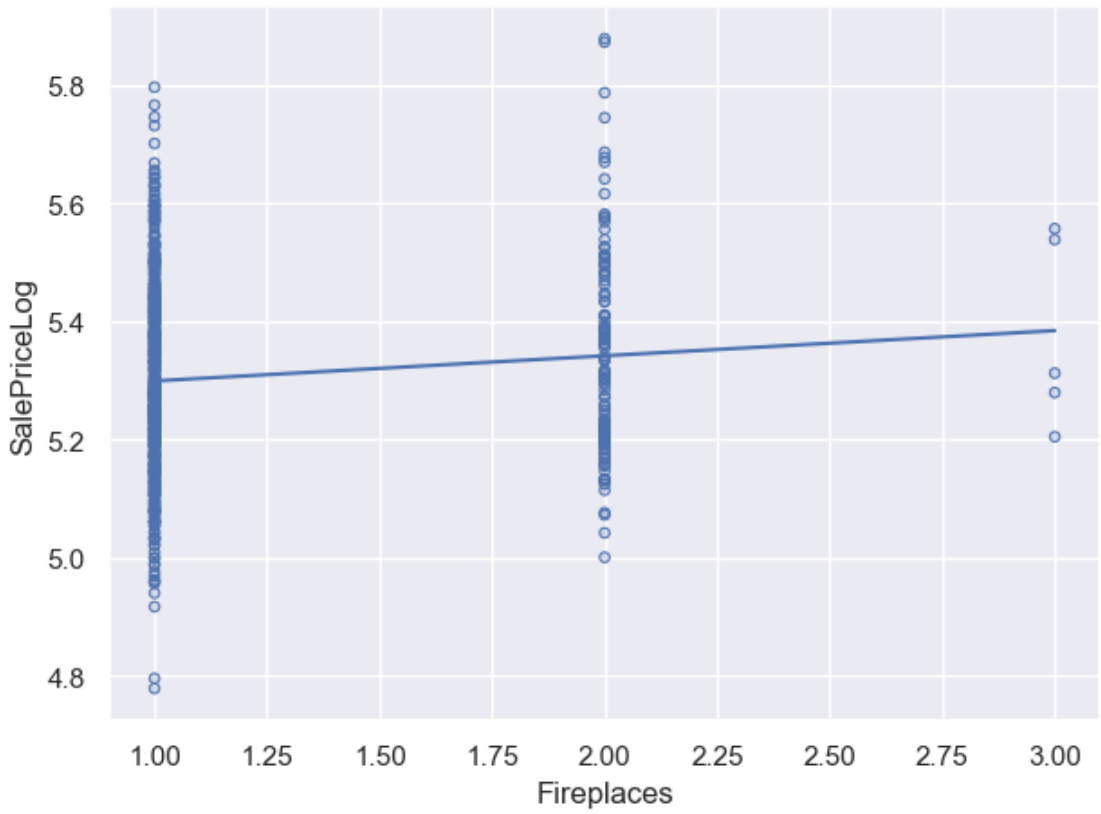

5.8
5.6
5.4
5.2
5.0
4.8
4.6
SalePriceLog
1.00
1.25
1.50
1.75
2.00
2.25
2.50
2.75
3.00
FullBath

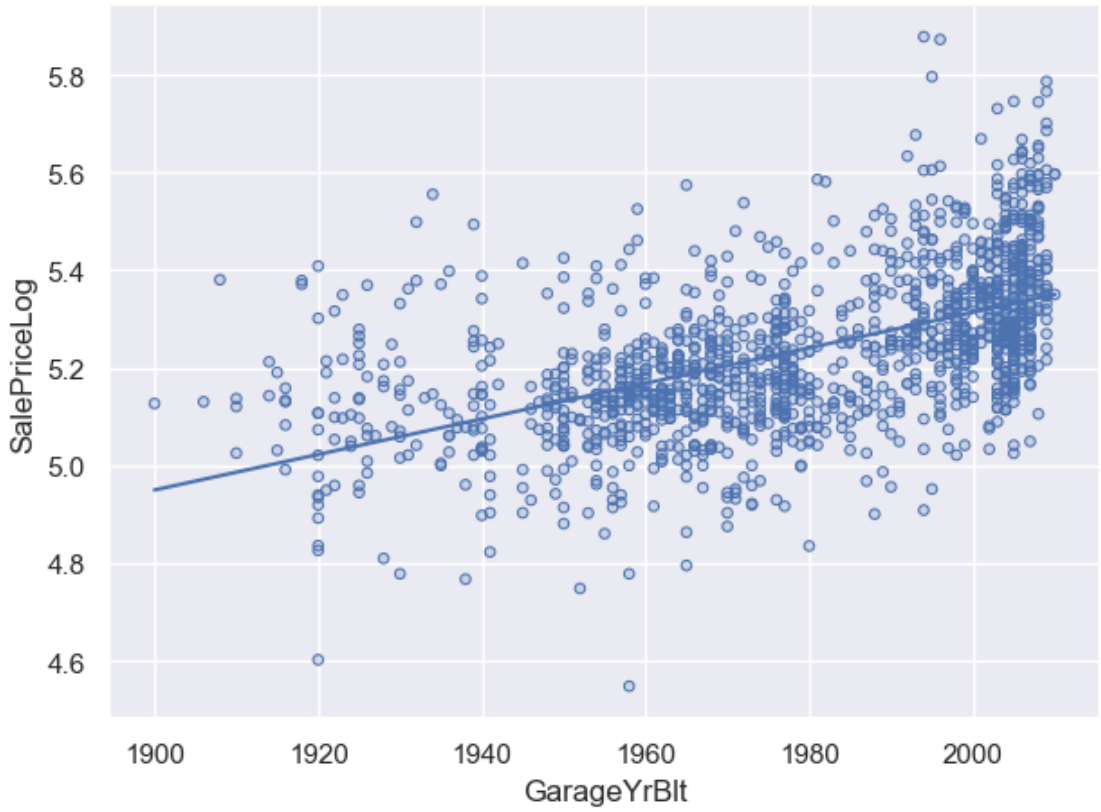

5.8
5.6
5.4
5.2
5.0
4.8
SalePriceLog
1.0
1.2
1.4
1.6
1.8
2.0
HalfBath

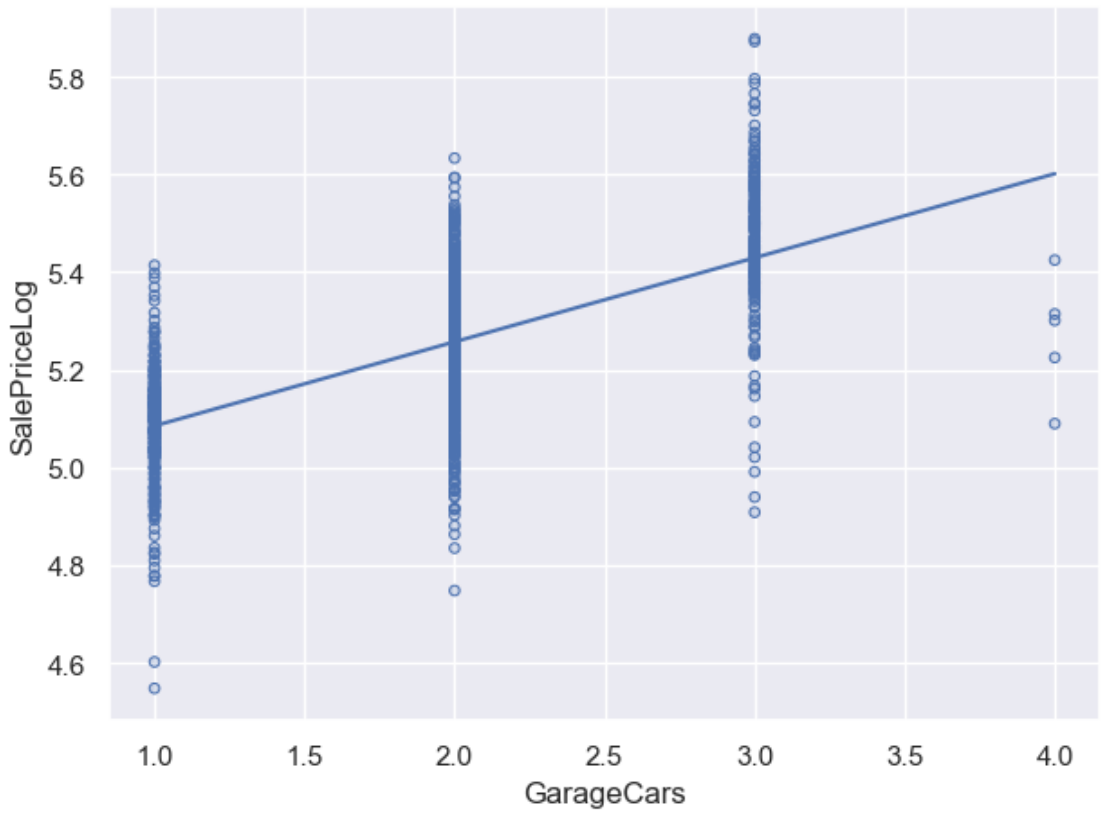

5.8
5.6
5.4
5.2
5.0
4.8
4.6
SalePriceLog
1
2
3
4
5
6
7
8
BedroomAbvGr

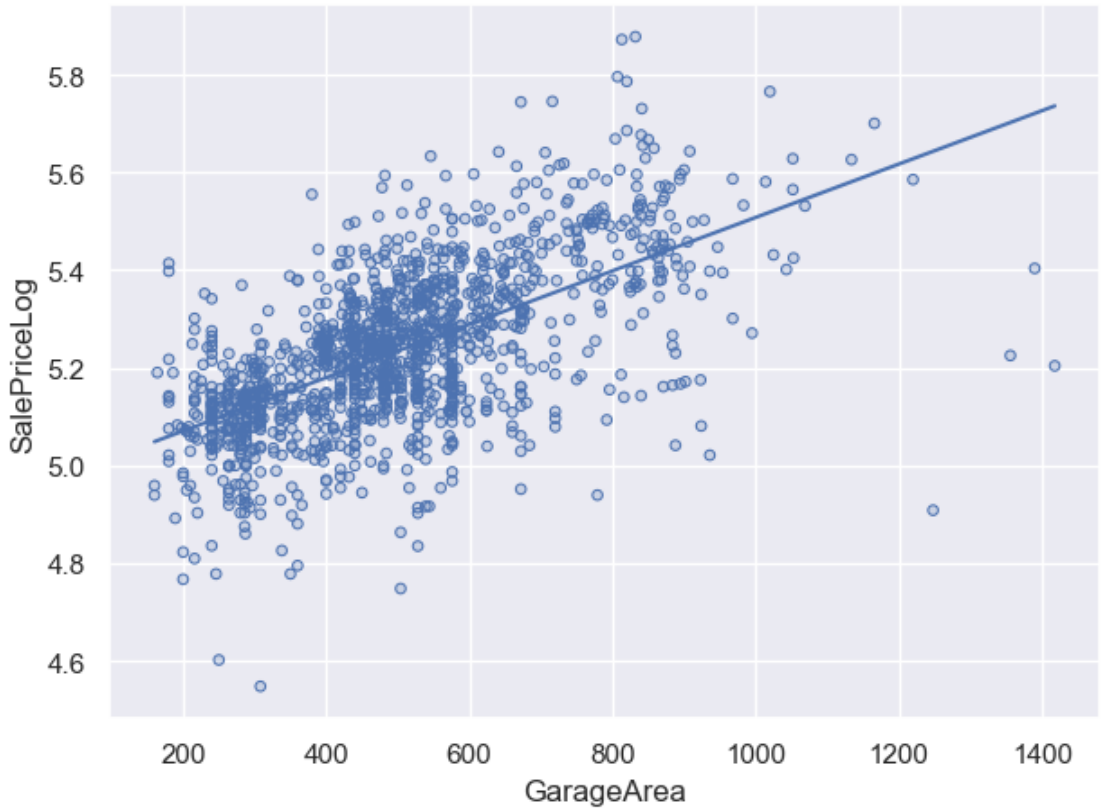

5.8
5.6
5.4
5.2
5.0
4.8
4.6
SalePriceLog
1.00
1.25
1.50
1.75
2.00
2.25
2.50
2.75
3.00
KitchenAbvGr

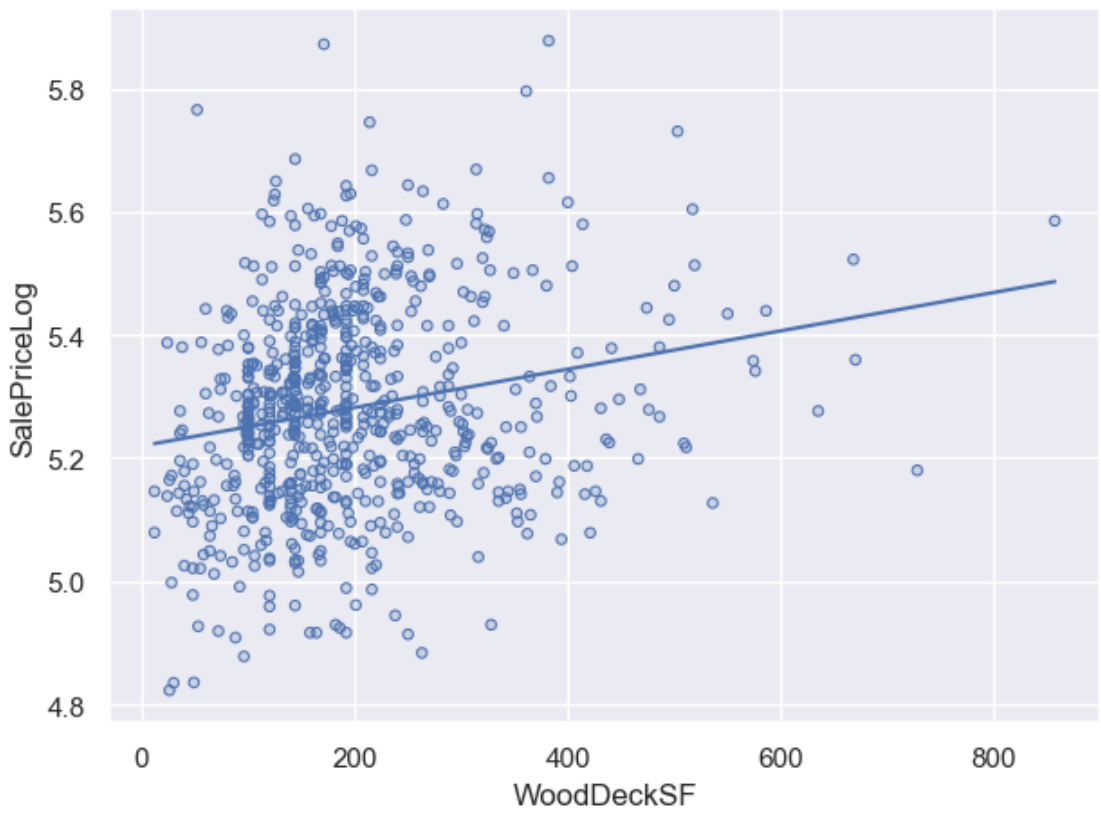

5.8
5.6
5.4
5.2
5.0
4.8
4.6
SalePriceLog
2
4
6
8
10
12
14
TotRmsAbvGrd


5.8
5.6
5.4
5.2
5.0
4.8
SalePriceLog
1.00
1.25
1.50
1.75
2.00
2.25
2.50
2.75
3.00
Fireplaces

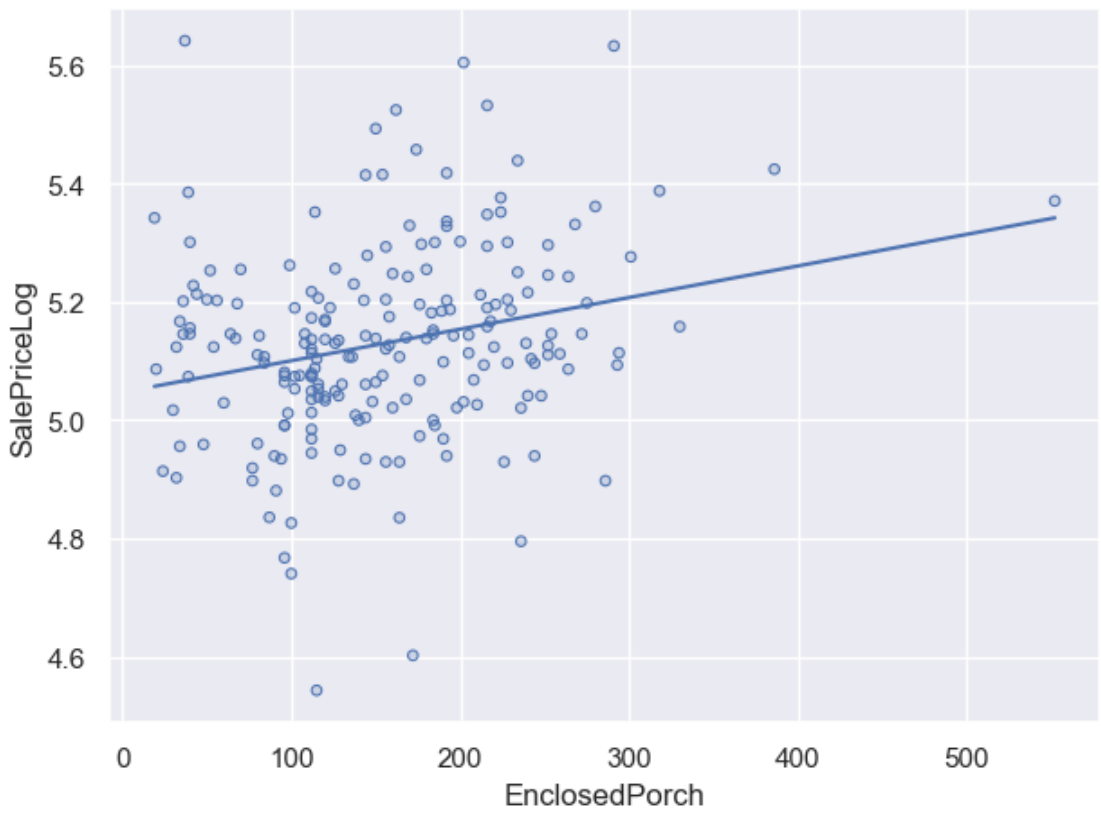

5.8
5.6
5.4
5.2
5.0
4.8
4.6
SalePriceLog
1900
1920
1940
1960
1980
2000
GarageYrBlt

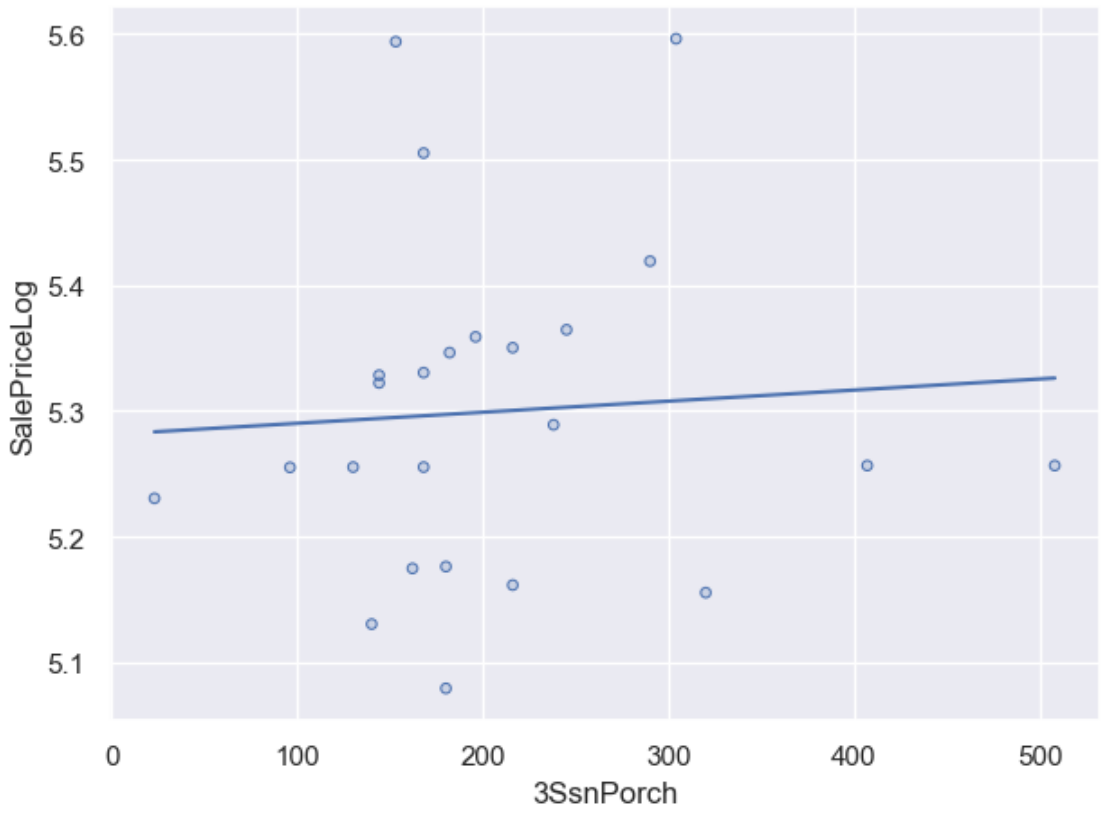

5.8
5.6
5.4
5.2
5.0
4.8
4.6
SalePriceLog
1.0
1.5
2.0
2.5
3.0
3.5
4.0
GarageCars

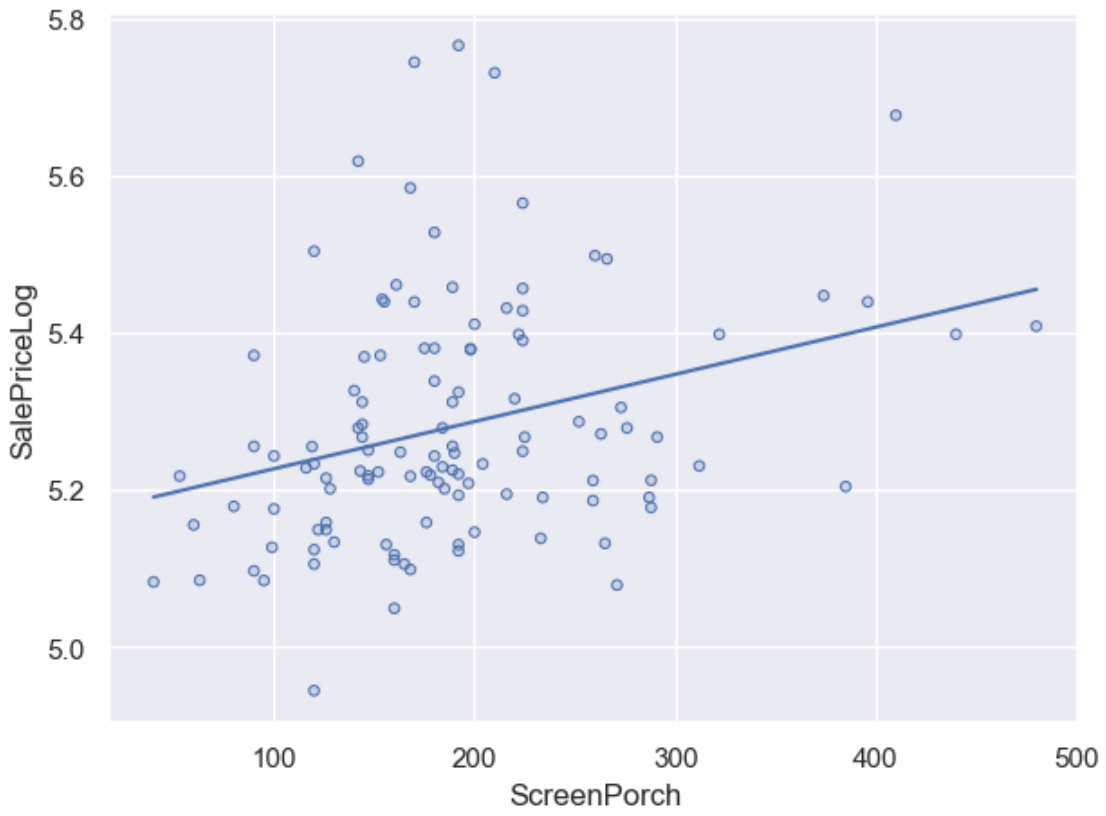

5.8
5.6
5.4
5.2
5.0
4.8
4.6
SalePriceLog
200
400
600
800
1000
1200
1400
GarageArea

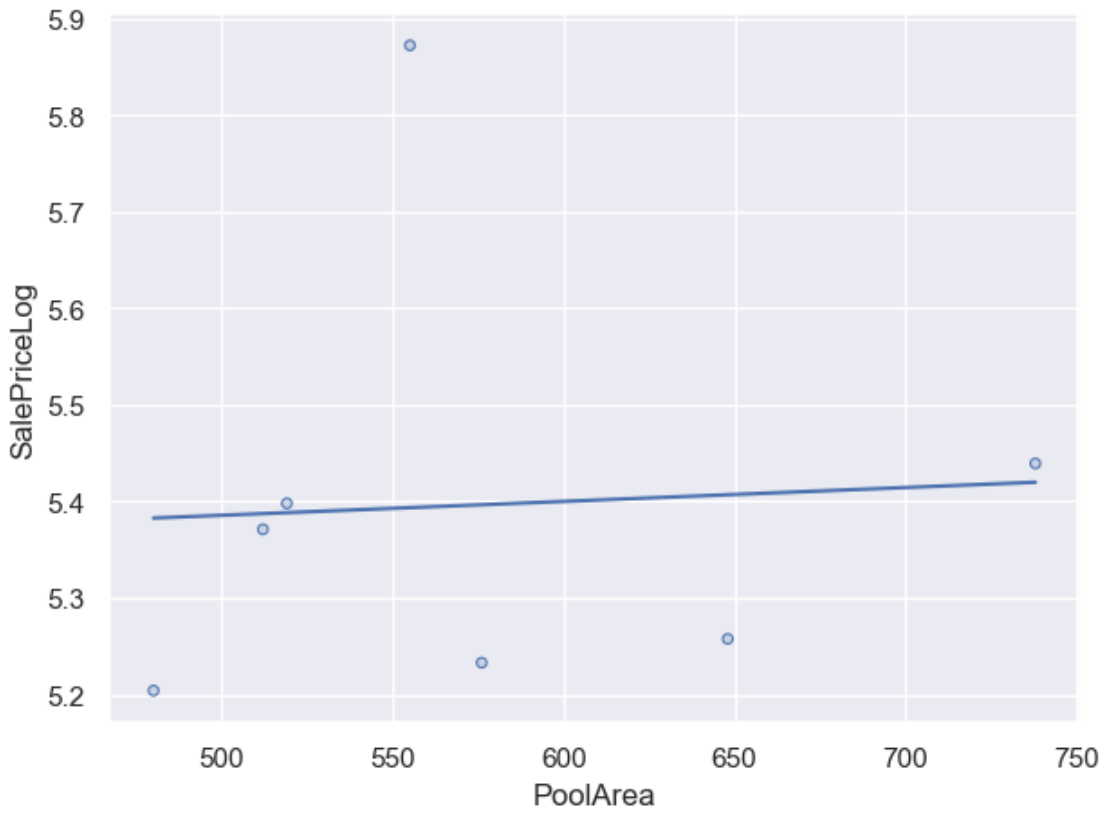

5.8
5.6
5.4
5.2
5.0
4.8
SalePriceLog
0
200
400
600
800
WoodDeckSF

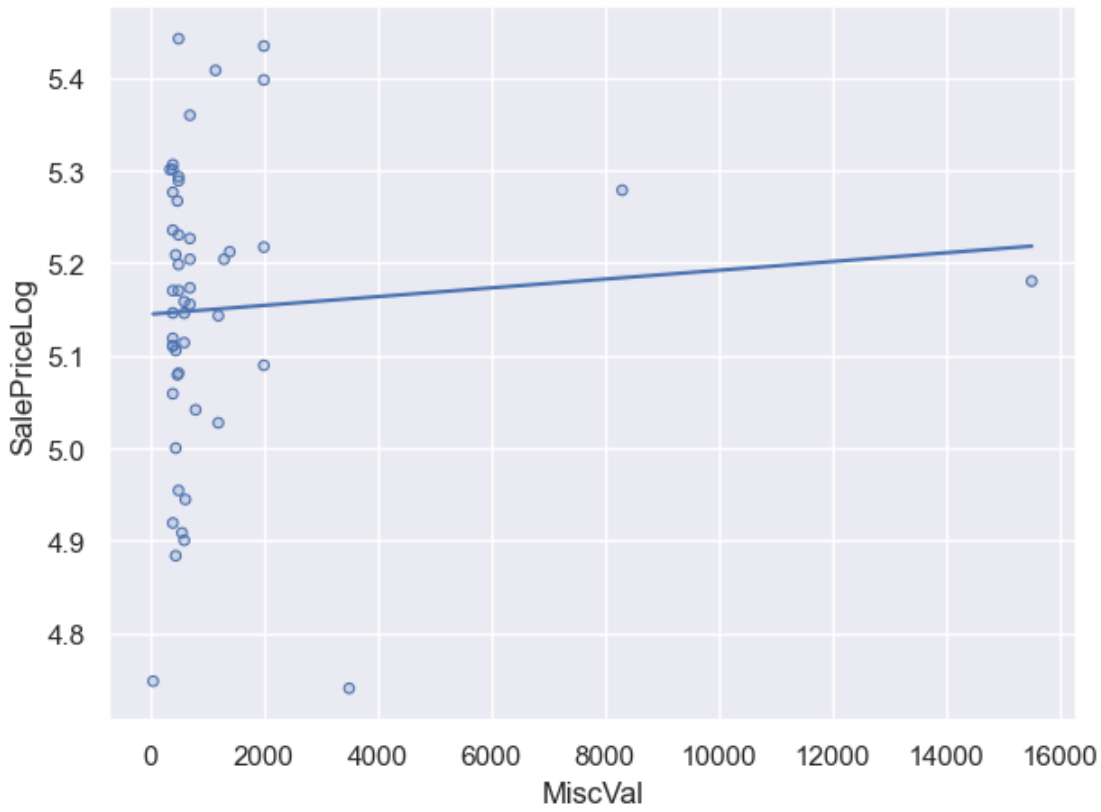
5.8
5.6
5.4
5.2
5.0
4.8
4.6
SalePriceLog
0
100
200
300
400
500
OpenPorchSF

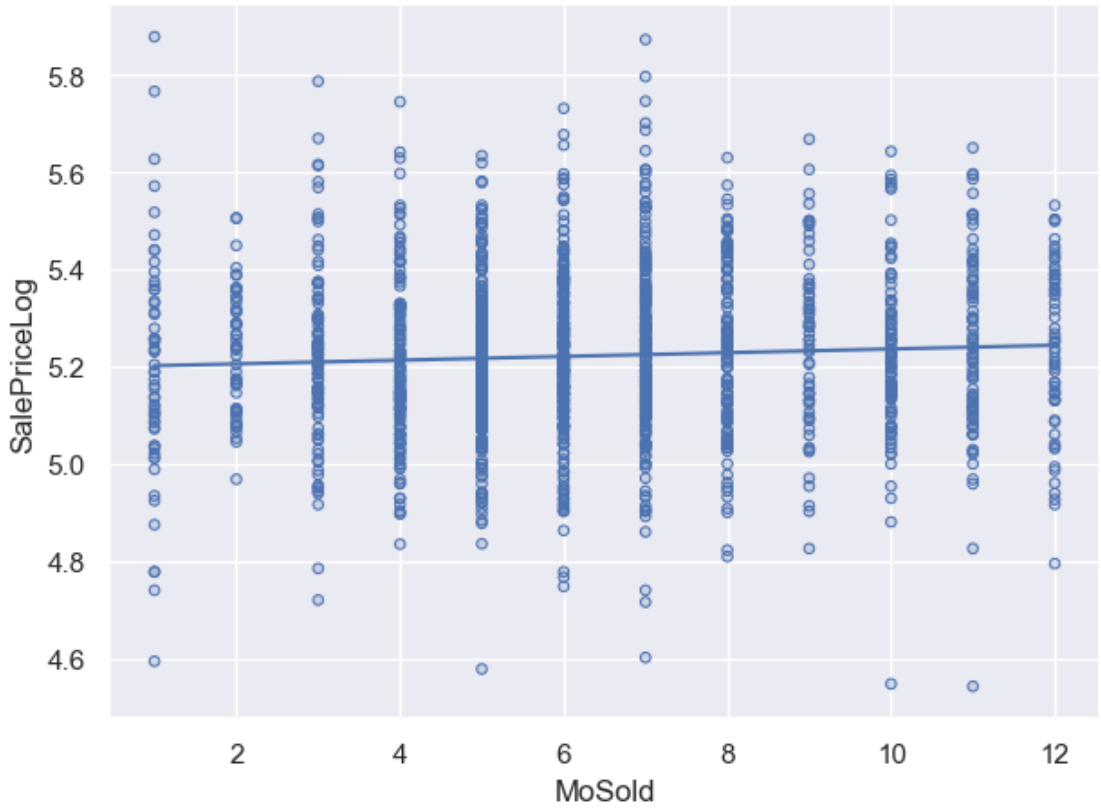

5.6
5.4
5.2
5.0
4.8
4.6
SalePriceLog
0
100
200
300
400
500
EnclosedPorch


5.6
5.5
5.4
5.3
5.2
5.1
SalePriceLog
0
100
200
300
400
500
3SsnPorch


5.8
5.6
5.4
5.2
5.0
SalePriceLog
100
200
300
400
500
ScreenPorch


5.9
5.8
5.7
5.6
5.5
5.4
5.3
5.2
SalePriceLog
500
550
600
650
700
750
PoolArea


5.4
5.3
5.2
5.1
5.0
4.9
4.8
SalePriceLog
0
2000
4000
6000
8000
10000
12000
14000
16000
MiscVal


5.8
5.6
5.4
5.2
5.0
4.8
4.6
SalePriceLog
2
4
6
8
10
12
MoSold

From a visual inspection, those that seem the most linearly correlated seem to be LotFrontage, LotArea, OverallQual, YearTotalBsmtSF, 1stFlrSF, 2ndFlrSF, GrLivArea, TotRmsAbvGrd, GarageCars, and GarageArea.

1.4.8 Question C-8

[121]:
```
corr_price_log = [df_train_filled['SalePriceLog'].corr(df_train_filled[n]) for␣
↪n in num_relevant]

# add to df with column names
data_corr = {'Feature': num_relevant, 'Corr_SalePriceLog': np.
↪abs(corr_price_log)}
corr_price_log_df = pd.DataFrame(data_corr)

# sort to find stronger correlations
corr_price_log_df = corr_price_log_df.sort_values(by='Corr_SalePriceLog',␣
↪ascending=False)
```

[122]:
```
corr_price_log_df[0:10]
```

[122]:

	Feature	Corr_SalePriceLog
3	OverallQual	0.817168
15	GrLivArea	0.700788
25	GarageCars	0.680871
26	GarageArea	0.650865
11	TotalBsmtSF	0.612439
12	1stFlrSF	0.597110
18	FullBath	0.593914
5	YearBuilt	0.590704
6	YearRemodAdd	0.567227
22	TotRmsAbvGrd	0.533599

[123]:

```
# compare with SalePrice

corr_price = [df_train_filled['SalePrice'].corr(df_train_filled[n]) for n in␣
 ↪num_relevant]
# add to df with column names
data_corr2 = {'Feature': num_relevant, 'Corr_SalePrice': np.abs(corr_price)}
corr_price_df = pd.DataFrame(data_corr2)
# sort to find stronger correlations
corr_price_df = corr_price_df.sort_values(by='Corr_SalePrice', ascending=False)
```

[124]:

```
corr_price_df[0:10]
```

[124]:

	Feature	Corr_SalePrice
3	OverallQual	0.790513
15	GrLivArea	0.708645
25	GarageCars	0.640901
26	GarageArea	0.623577
11	TotalBsmtSF	0.613678
12	1stFlrSF	0.606136
18	FullBath	0.559904
22	TotRmsAbvGrd	0.533100
5	YearBuilt	0.526092
6	YearRemodAdd	0.508104

The only effect in the top 10 ranking of correlated variables with SalePrice is that the log transformation changes the rank positions of YearBuilt, YearRemodAdd, and TotRmsAbvGrd. Since YearBlt and YearRemodAdd went up in the ranking, they probably had a non-linear relationship with SalePrice which became linear by taking the log transformation.

1.4.9 Question C-9

[125]:

```
# mean sale prices for categorical features

# empty list for correlations
corr_price_cat=[]
```

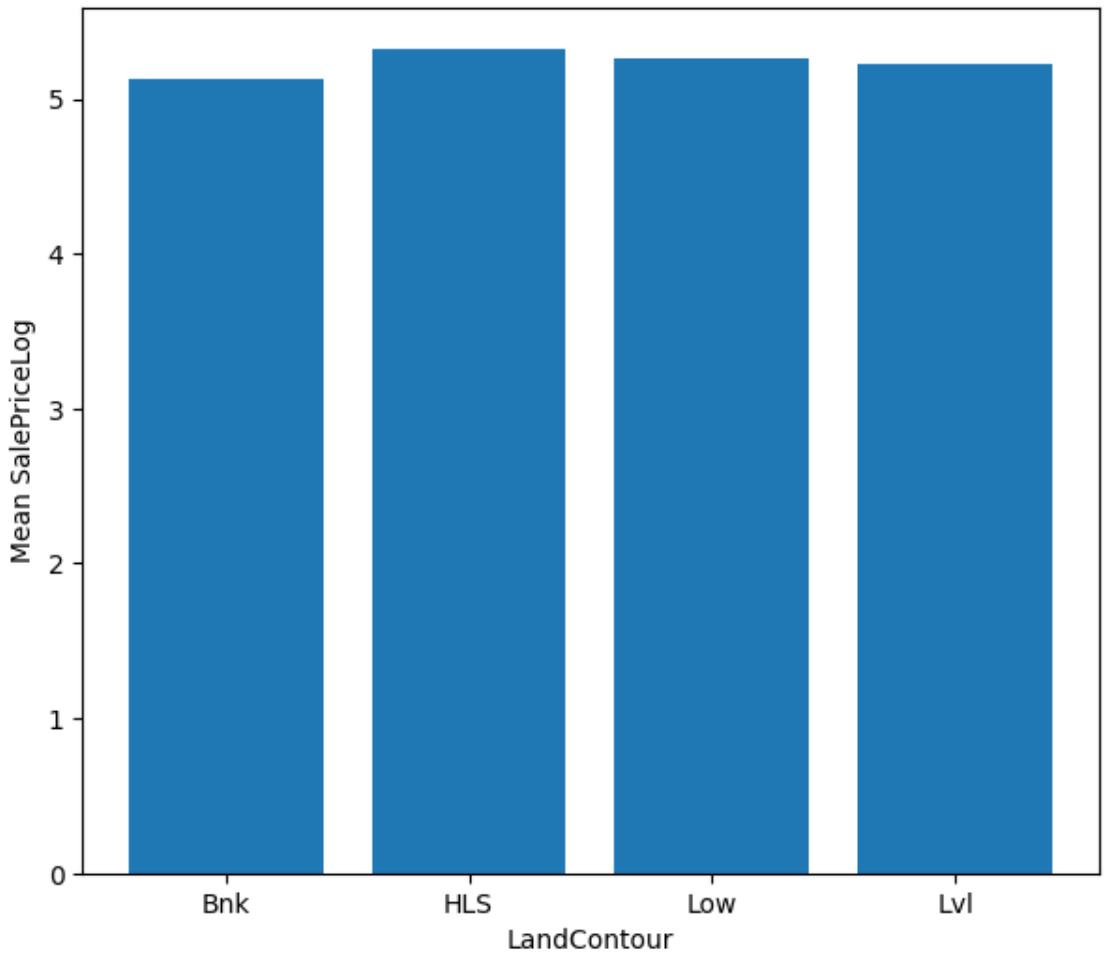
```
for c in categorical:
    cat_means = df_train_filled.groupby(c)['SalePriceLog'].mean()

    # reaplce values with the sale price log means
    cat_means_map = df_train_filled[c].map(cat_means)

    # correlation
    corr_price_cat.append(cat_means_map.corr(df_train_filled['SalePriceLog']))

# data for dataframe
data_corr_cat = {'Feature': categorical, 'Corr_SalePriceLog_Mean': np.
 ↪abs(corr_price_cat)}

# dataframe of means of SalePriceLog for each category of categorical variables
corr_price_mean_cat_df = pd.DataFrame(data_corr_cat)

# sort to find stronger correlations
corr_price_mean_cat_df = corr_price_mean_cat_df.
 ↪sort_values(by='Corr_SalePriceLog_Mean', ascending=False)
```

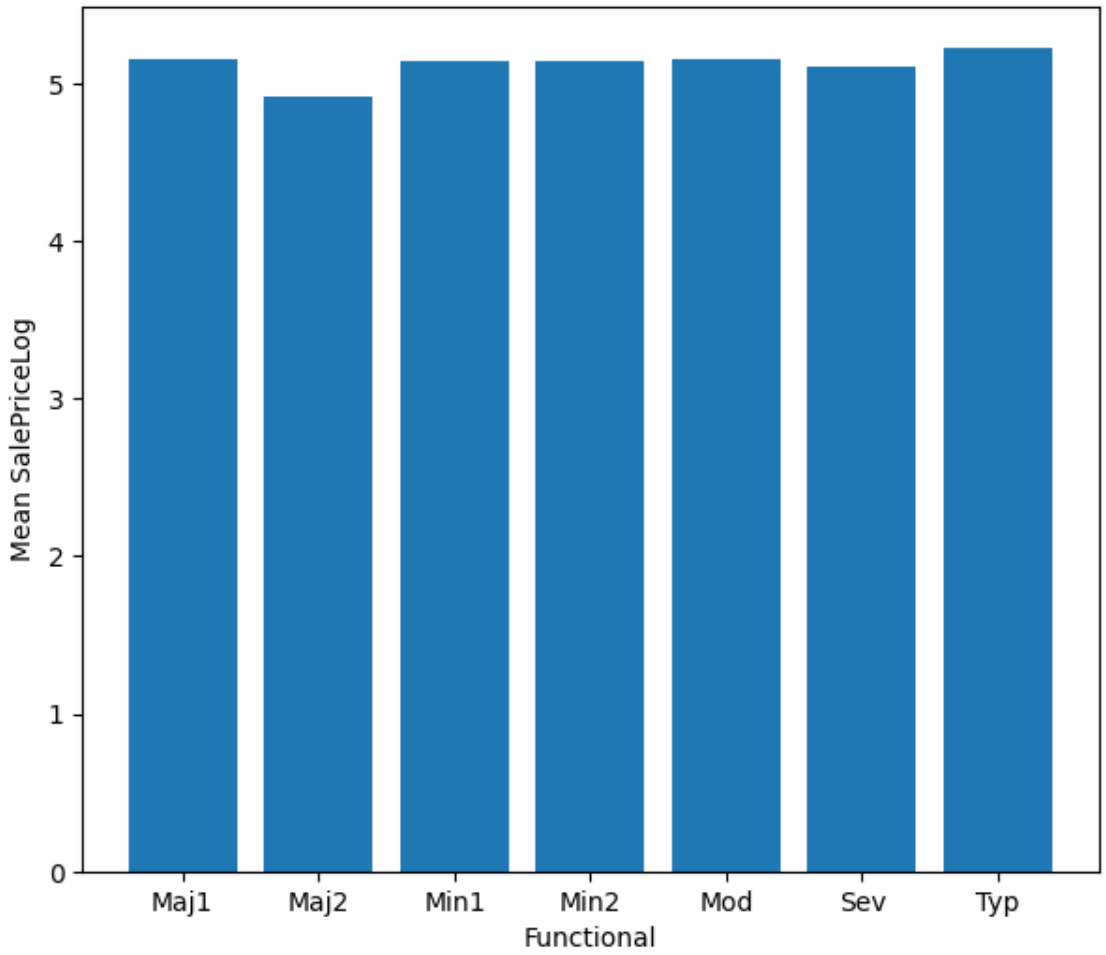
[126]:
```
print("10 least correlated features based on the means of SalePriceLog for each␣
 ↪category:")
print(corr_price_mean_cat_df.tail(10))
```

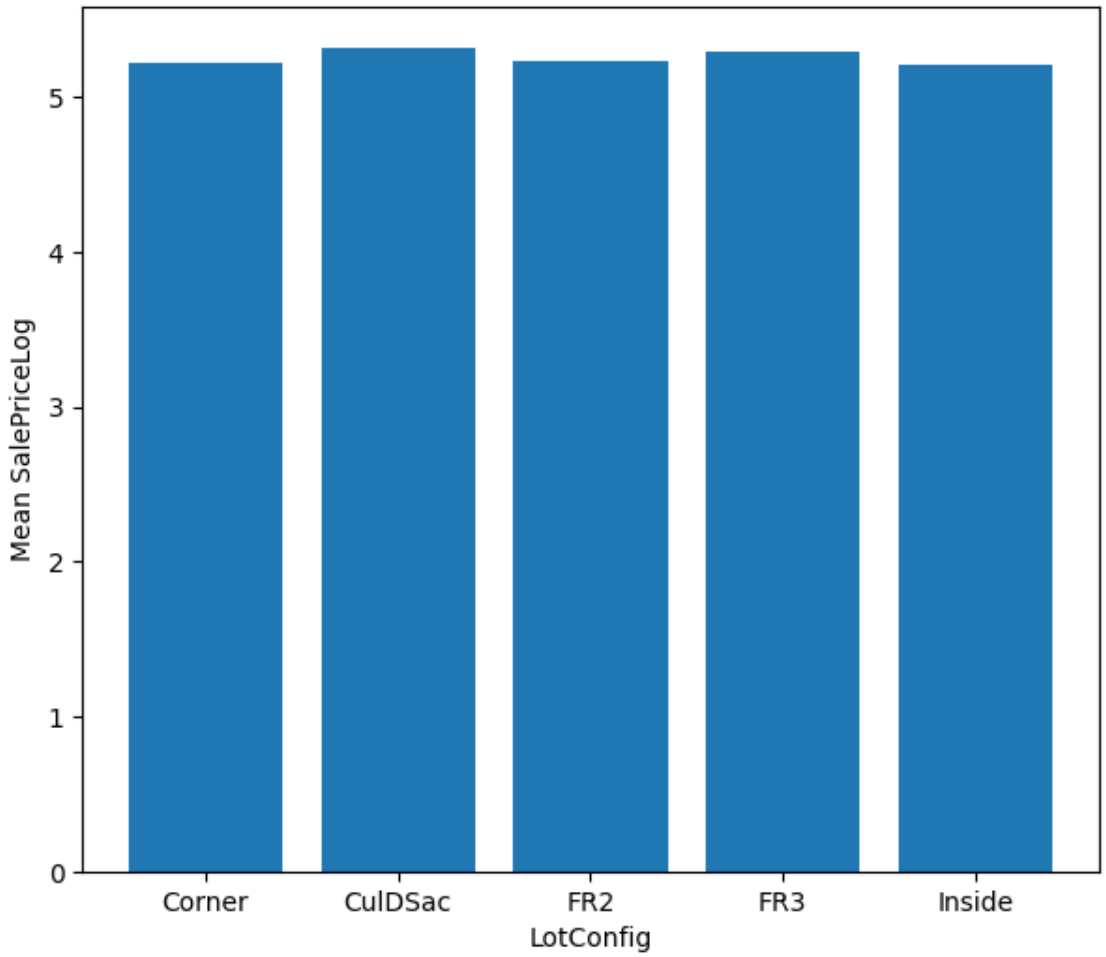
```
10 least correlated features based on the means of SalePriceLog for each
category:
        Feature  Corr_SalePriceLog_Mean
4   LandContour                0.160600
31   Functional                0.156538
6     LotConfig                0.153222
14     RoofMatl                0.134149
10   Condition2                0.110116
40  MiscFeature                0.103670
38       PoolQC                0.090491
1        Street                0.057612
7     LandSlope                0.038463
5     Utilities                0.012696
```

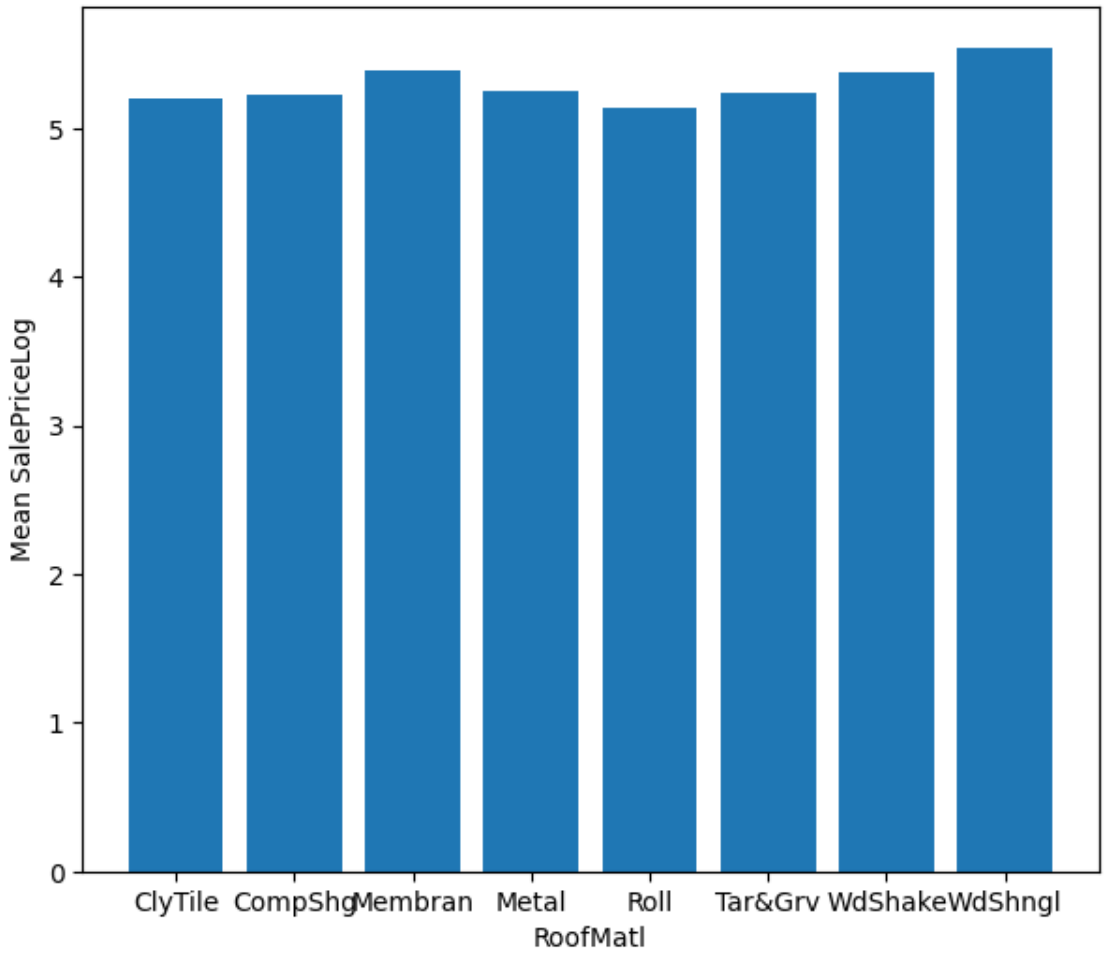
[127]:
```
# visualize with bar plot of least correlated features
least_corr_cat = corr_price_mean_cat_df.tail(10)['Feature']

for i in least_corr_cat:

    # means
    cat_means = df_train_filled.groupby(i)['SalePriceLog'].mean()

    plt.figure(figsize=(7,6))
```

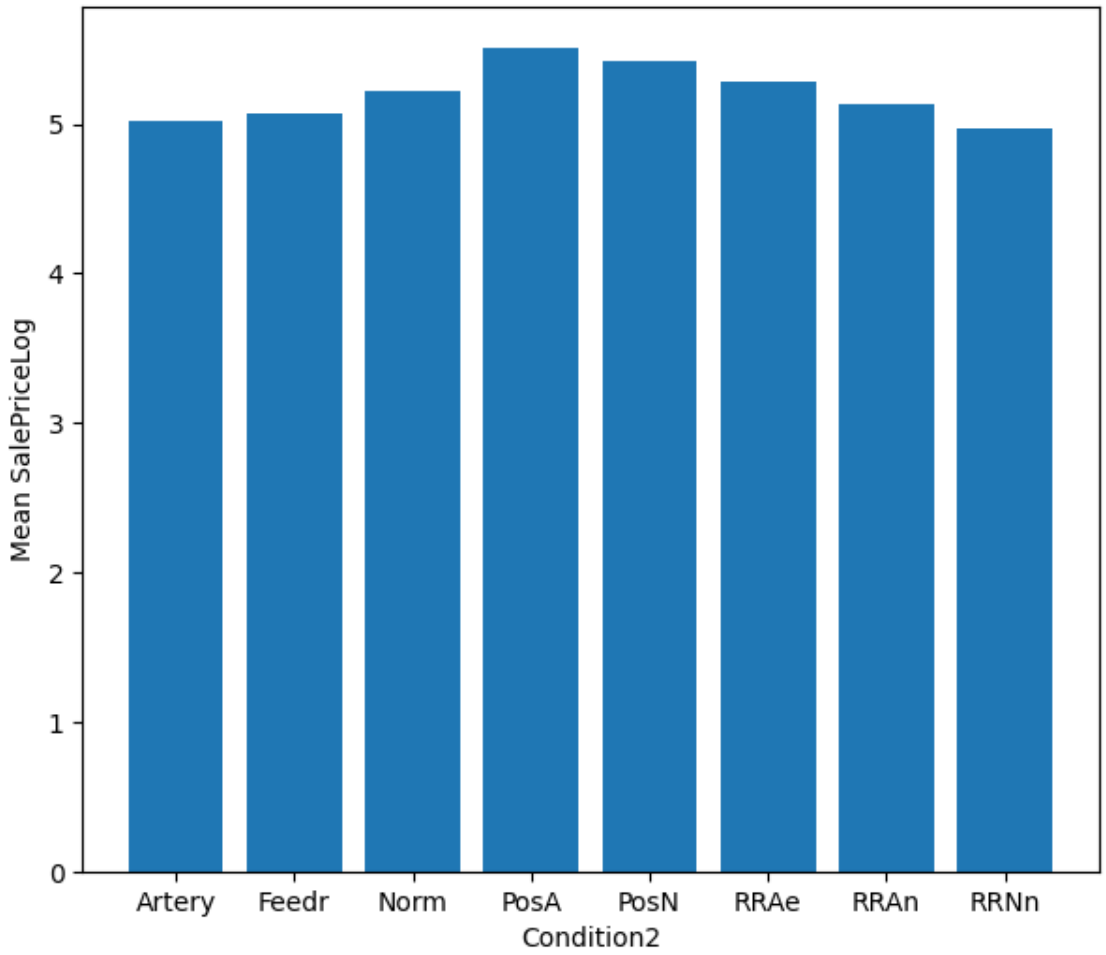
```
# plot
plt.bar(cat_means.index, cat_means.values)
plt.ylabel('Mean SalePriceLog')
plt.xlabel(i)
```

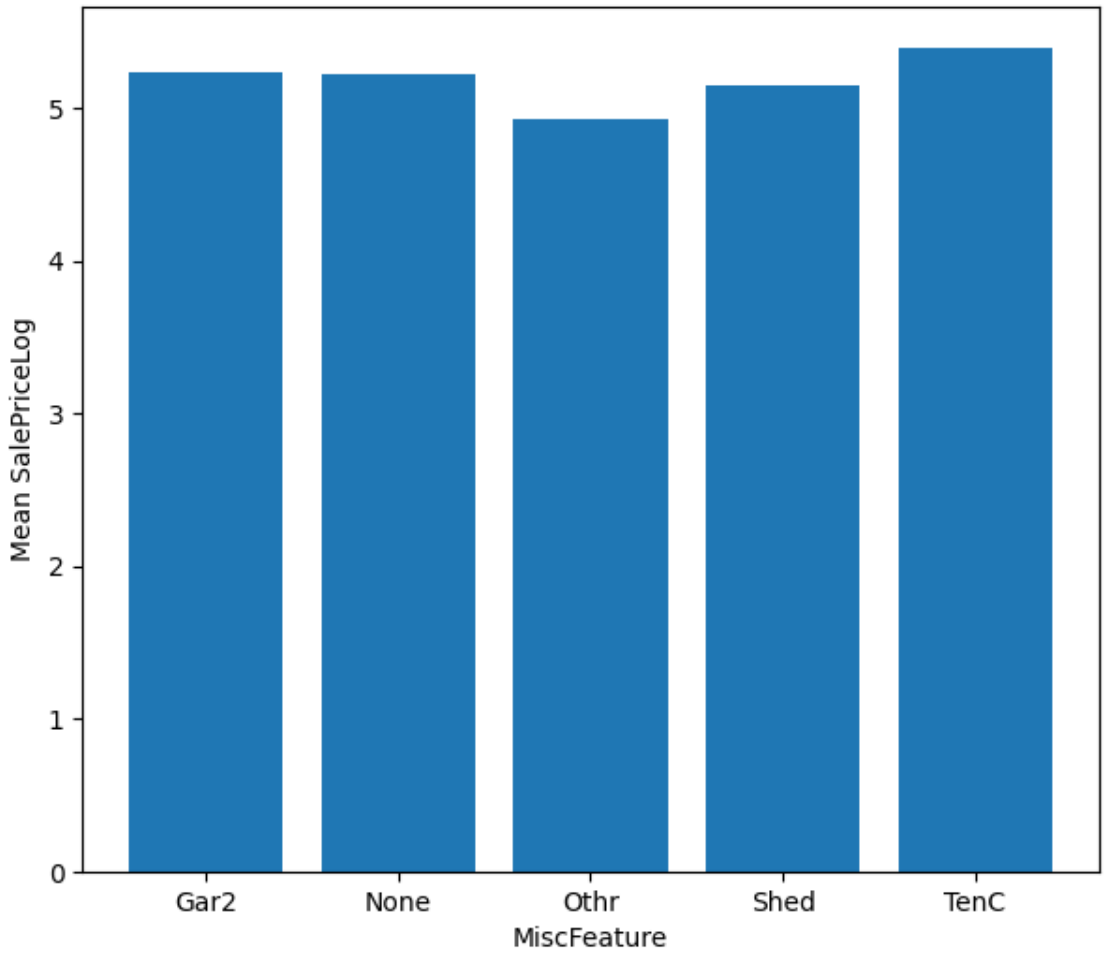


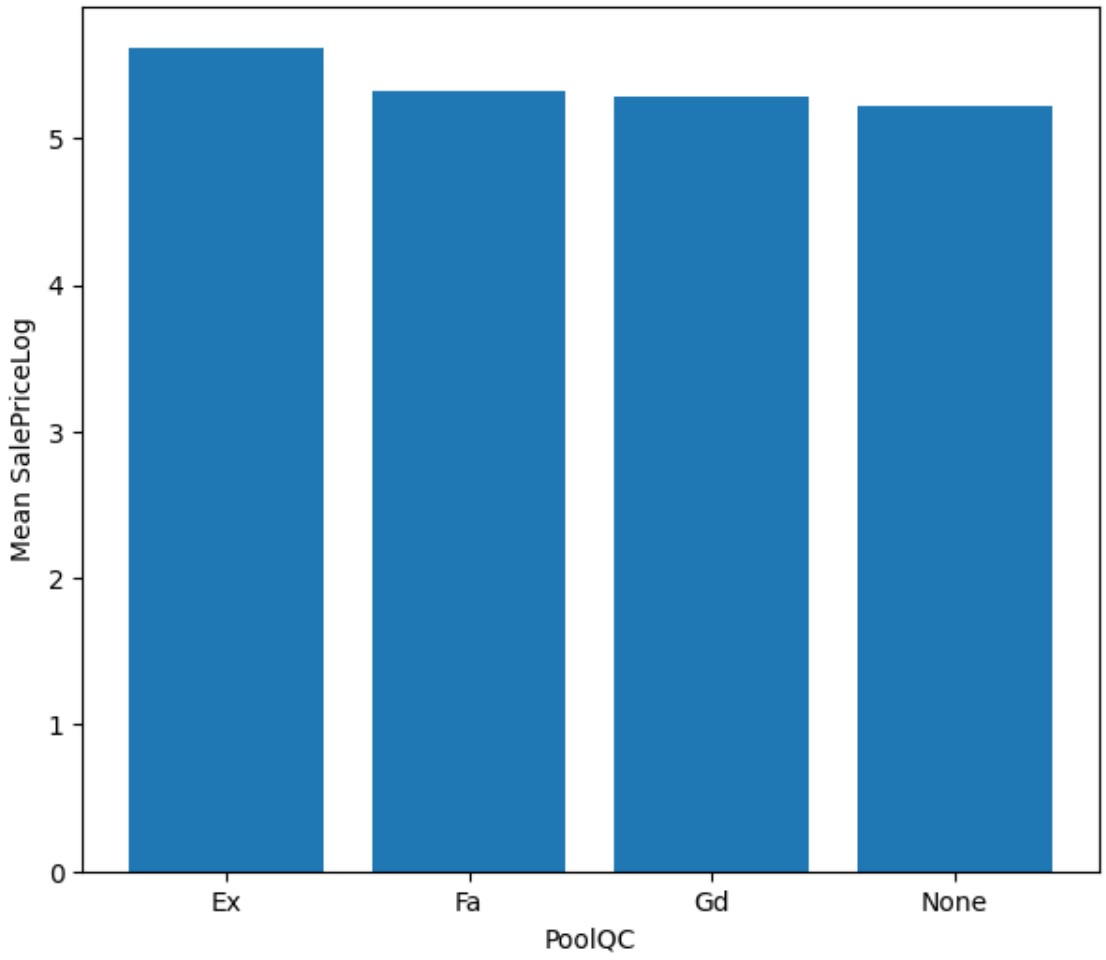

Mean SalePriceLog
0
1
2
3
4
5
Maj1
Maj2
Min1
Min2
Mod
Sev
Typ
Functional

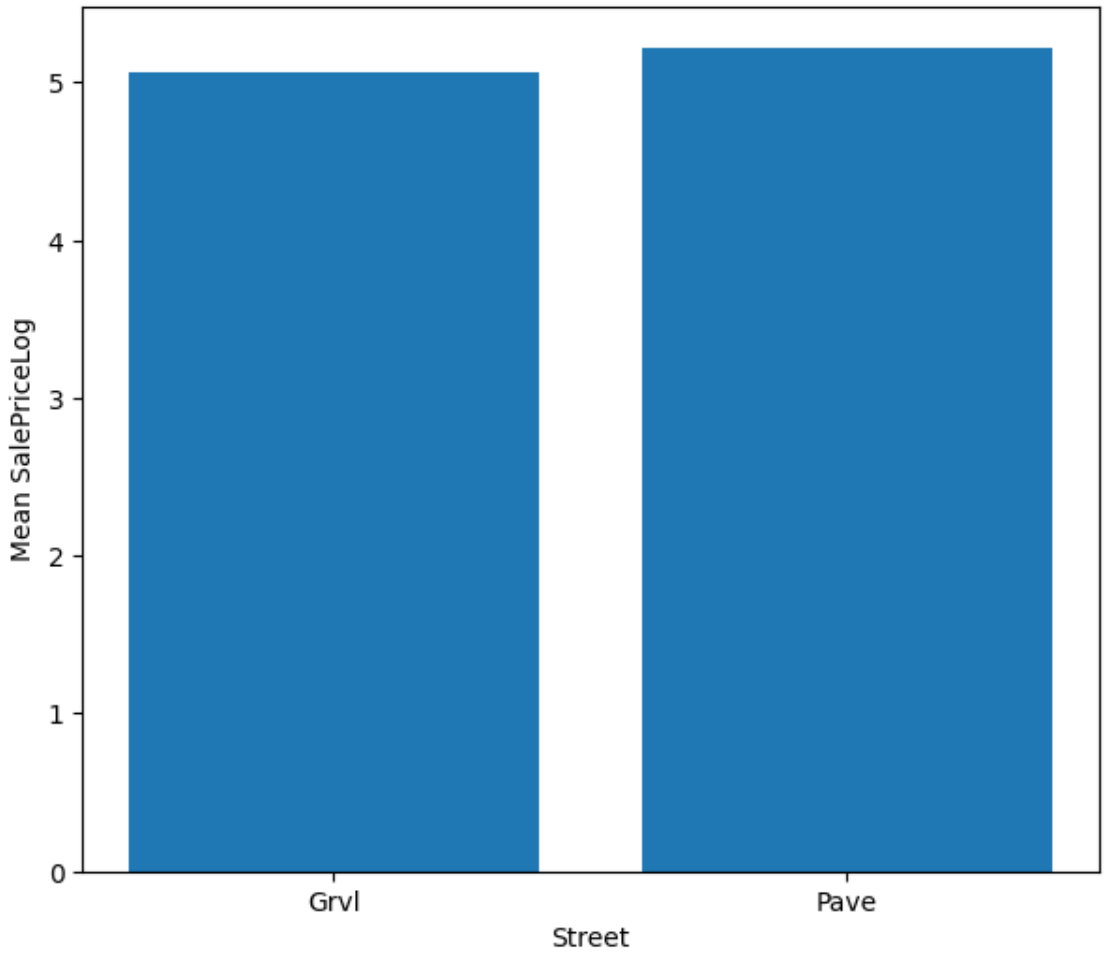

Mean SalePriceLog
0
1
2
3
4
5
Corner
CulDSac
FR2
FR3
Inside
LotConfig

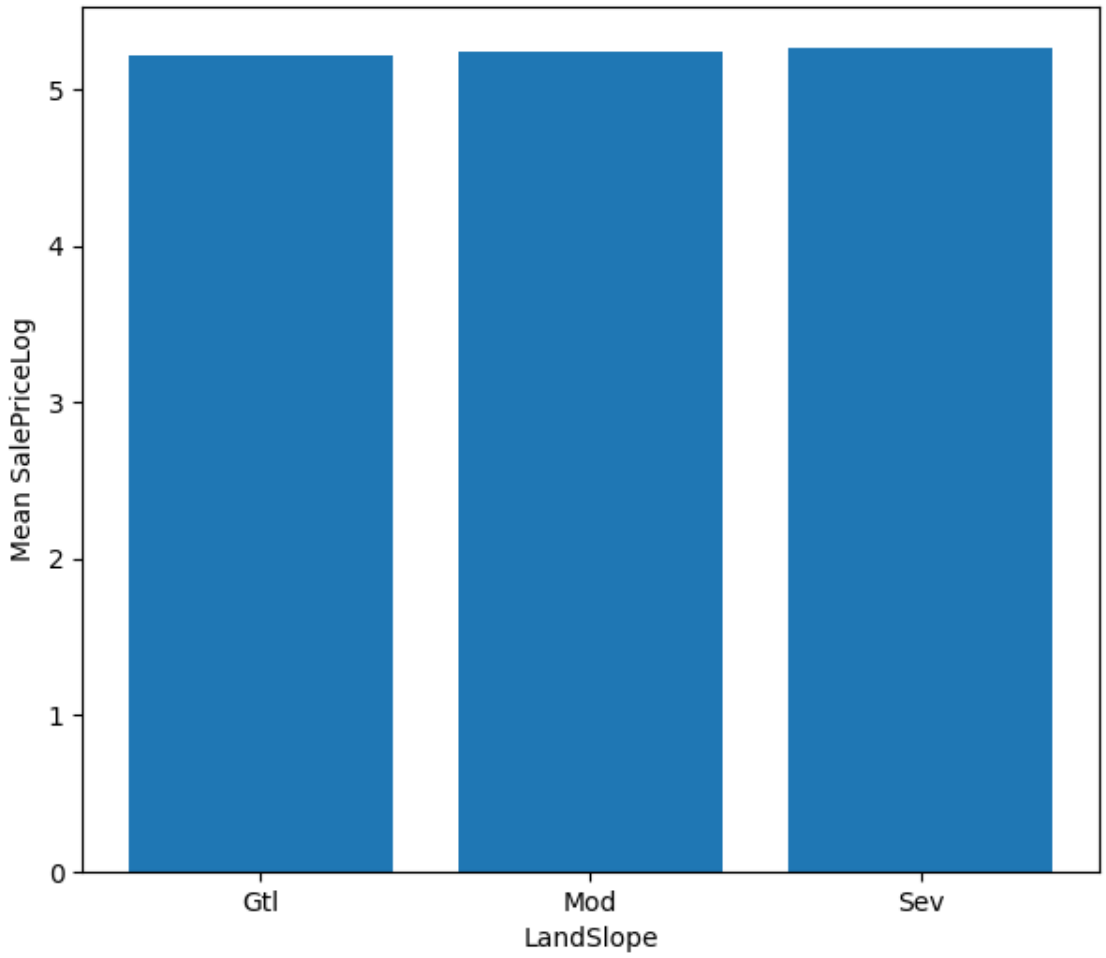

Mean SalePriceLog
0
1
2
3
4
5
ClyTile
CompShg
Membran
Metal
Roll
Tar&Grv
WdShake
WdShngl
RoofMatl

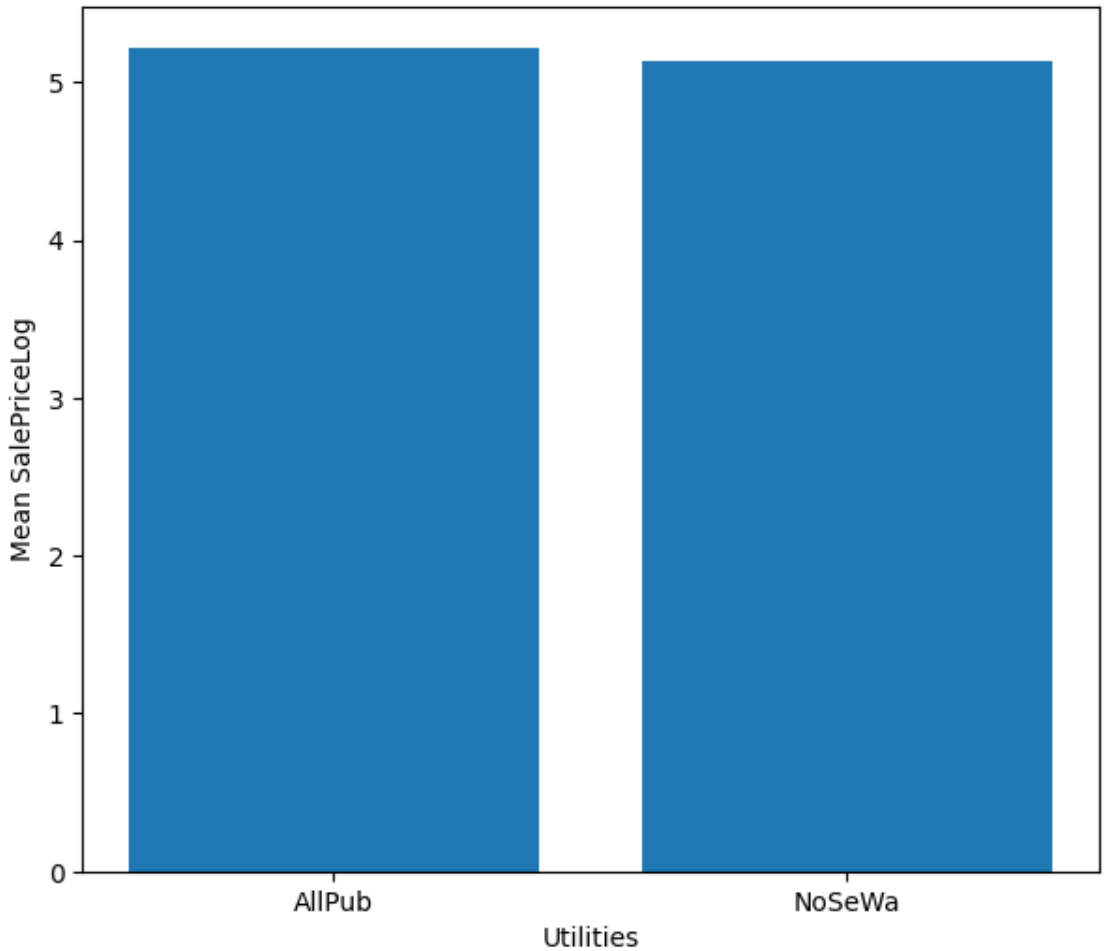

Mean SalePriceLog
0
1
2
3
4
5
Artery
Feedr
Norm
PosA
PosN
RRAe
RRAn
RRNn
Condition2


Mean SalePriceLog
0
1
2
3
4
5
Gar2
None
Othr
Shed
TenC
MiscFeature


Mean SalePriceLog
0
1
2
3
4
5
Ex
Fa
Gd
None
PoolQC


Mean SalePriceLog
0
1
2
3
4
5
Grvl
Pave
Street


Mean SalePriceLog
0
1
2
3
4
5
Gtl
Mod
Sev
LandSlope



These categorical features appear to be weakly associated with 'SalePriceLog' because their distributions overlap heavily across categories, and the medians are quite similar. Looking only at category means is not enough to assess the relationship as the spread and overlap of the full distributions show that these variables do not provide very different house prices.

1.4.10 Question C-10

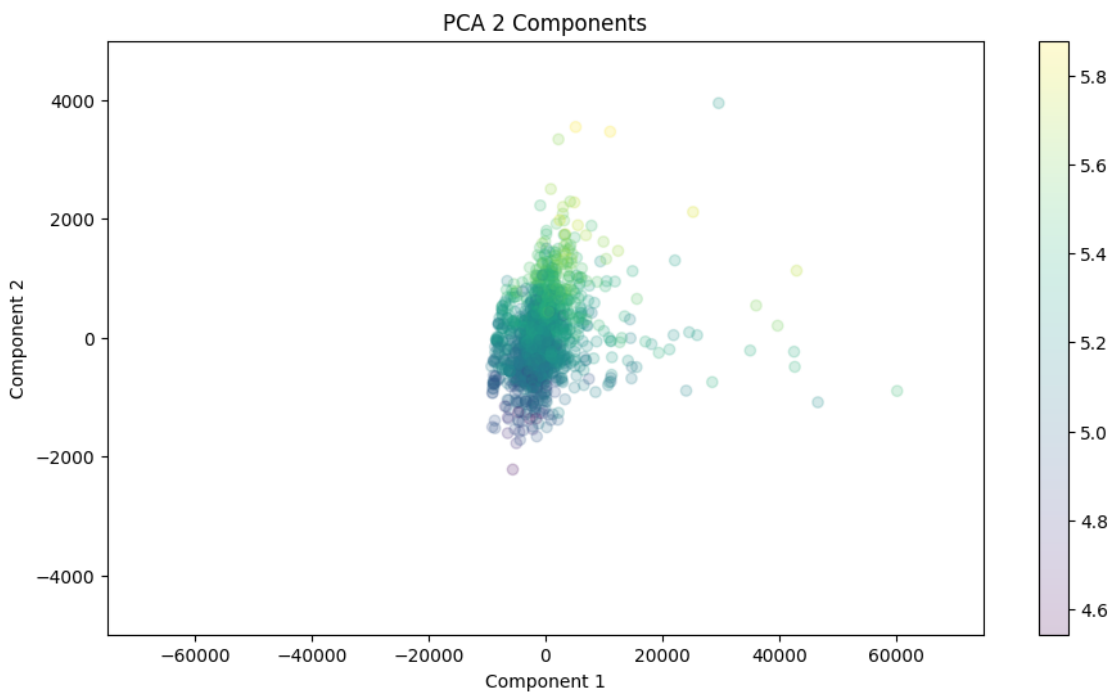
[128]:
```
X = df_train_filled[num_relevant].values

# pca on num_relevant with 2 components
pca2 = PCA(n_components=2)
X_pca2 = pca2.fit_transform(X)

# pca on num_relevant with 3 components
pca3 = PCA(n_components=3)
X_pca3 = pca3.fit_transform(X)
```

Setting axis limits to the plots to reduce outliers, which were causing the distribution of data points

to be unsymmetrical:

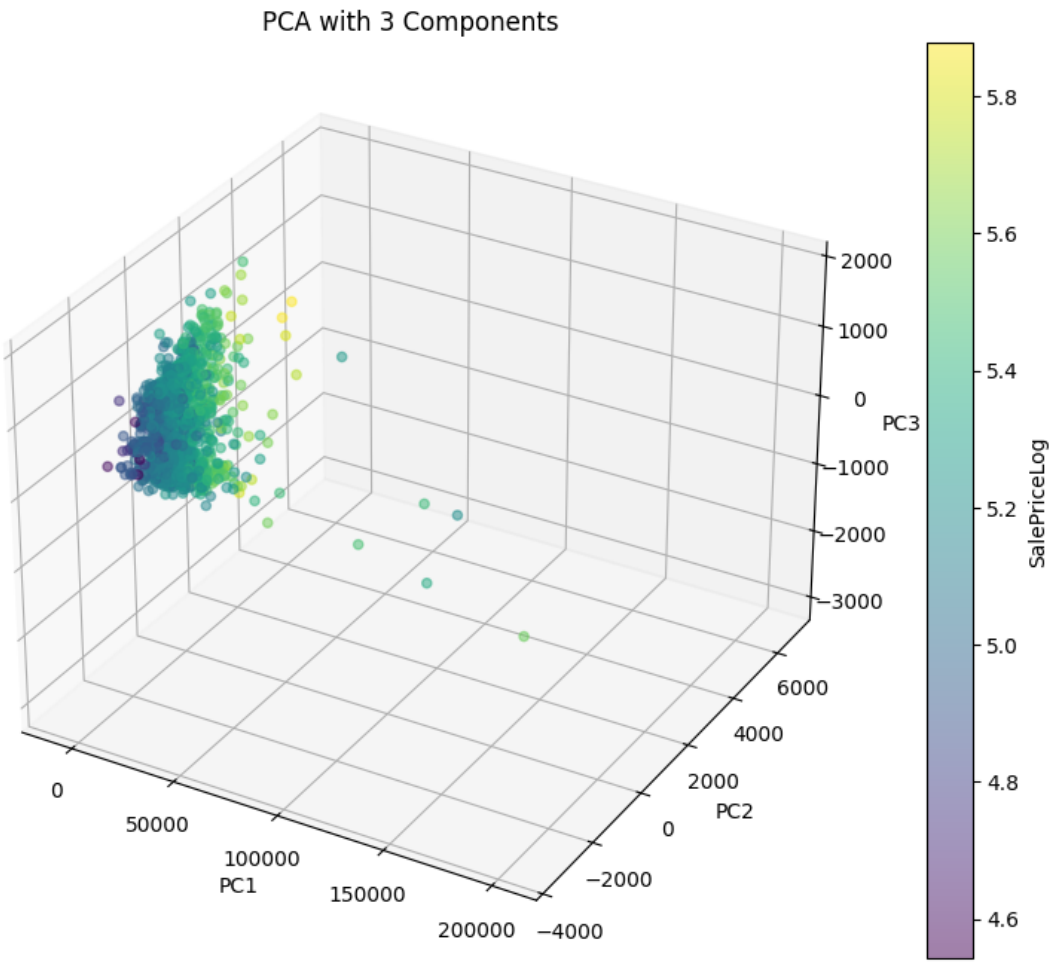
[131]:
```
# plot pca3 with limits
plt.figure(figsize=(11,6))
plt.scatter(X_pca2[:,0], X_pca2[:,1], c=df_train_filled['SalePriceLog'],␣
↪alpha=0.2)
plt.xlim(-75000, 75000)
plt.ylim(-5000, 5000)
plt.xlabel("Component 1")
plt.ylabel("Component 2")
plt.title("PCA 2 Components")
plt.colorbar()
```

[131]: <matplotlib.colorbar.Colorbar at 0x20024a65550>



[133]:
```
# plot pca3
fig = plt.figure(figsize=(10,8))
ax = fig.add_subplot(111, projection='3d')
sc = ax.scatter(X_pca3[:,0], X_pca3[:,1], X_pca3[:,2],␣
↪c=df_train_filled['SalePriceLog'], cmap='viridis', alpha=0.5)
ax.set_xlabel('PC1')
ax.set_ylabel('PC2')
ax.set_zlabel('PC3')
ax.set_title('PCA with 3 Components')
fig.colorbar(sc, ax=ax, label='SalePriceLog')
```

```
plt.show()
```



The first three principal components do not completely separate houses by price, but coloring by 'SalePriceLog' shows a slight gradient along the space. This suggests that price is influenced by combinations of numerical features, but not in a perfectly low-dimensional way.

1.4.11 Question C-11

[134]:
```
# testing data
house_test = pd.read_csv("C:\\Users\\tessa\\Downloads\\sem 5 concordia\\mast␣
↪387\\house_test.csv")
```

[135]:
```
len(house_test.columns)
```

[135]: 80

Missing values of testing set

[136]:
```
# check testing data for missing values
missing_test = house_test.isna().sum()
missing_vals_test = missing_test[missing_test > 0]
missing_vals_test
```

[136]:
```
Alley           10
MasVnrType       8
FireplaceQu      7
GarageType       3
GarageYrBlt      3
GarageFinish     3
GarageQual       3
GarageCond       3
PoolQC          11
Fence            9
MiscFeature     10
dtype: int64
```

[137]:
```
# take care of missing values same way as for training set
df_test_filled = house_test.copy()
```

[138]:
```
# alley: the feature 'NA' is given to properties with no alley access
# replace with 'None'
df_test_filled.loc[df_test_filled['Alley'].isna(), 'Alley'] = 'None'

# check
print(df_test_filled['Alley'].value_counts())
print("\n",df_test_filled['Alley'].isna().sum())
```

```
Alley
None    10
Pave     1
Name: count, dtype: int64

 0
```

[139]:
```
# MasVnrType: 'NA' given to properties with no masonry veneer
# replace with 'None'
df_test_filled.loc[df_test_filled['MasVnrType'].isna(), 'MasVnrType'] = 'None'

# check
print(df_test_filled['MasVnrType'].value_counts())
print("\n",df_test_filled['MasVnrType'].isna().sum())
```

```
MasVnrType
None       8
Stone      2
BrkFace    1
```

```
Name: count, dtype: int64
```

```
0
```

[140]:
```python
# FireplaceQu: 'NA' given to properties with no fireplace
# replace with 'None'
df_test_filled.loc[df_test_filled['FireplaceQu'].isna(), 'FireplaceQu'] = 'None'

# check
print("\n",df_test_filled['FireplaceQu'].isna().sum())
```

```
0
```

[141]:
```python
# GarageType, GarageYrBlt, GarageFinish, GarageQual, GarageCond: 'NA' given to␣
↪properties with no garage
# replace with 'None' or 0 for numeric value
df_test_filled.loc[df_test_filled['GarageType'].isna(), 'GarageType'] = 'None'

df_test_filled.loc[df_test_filled['GarageYrBlt'].isna(), 'GarageYrBlt'] = 0

df_test_filled.loc[df_test_filled['GarageFinish'].isna(), 'GarageFinish'] =␣
↪'None'

df_test_filled.loc[df_test_filled['GarageQual'].isna(), 'GarageQual'] = 'None'

df_test_filled.loc[df_test_filled['GarageCond'].isna(), 'GarageCond'] = 'None'

# check
print("\n",df_test_filled['GarageType'].isna().sum())
print("\n",df_test_filled['GarageYrBlt'].isna().sum())
print("\n",df_test_filled['GarageFinish'].isna().sum())
print("\n",df_test_filled['GarageQual'].isna().sum())
print("\n",df_test_filled['GarageCond'].isna().sum())
```

```
0

0

0

0

0
```

[142]:
```python
df_test_filled['PoolQC']
```

[142]:
```
0    NaN
1    NaN
2    NaN
3    NaN
4    NaN
5    NaN
6    NaN
7    NaN
8    NaN
9    NaN
10   NaN
Name: PoolQC, dtype: float64
```

[143]:
```
# PoolQC: 'NA' for properties with no pool, it is of type float here so I will␣
↪make it match the training set
# replace with 'None'
df_test_filled['PoolQC'] = df_test_filled['PoolQC'].astype(object)
df_test_filled.loc[df_test_filled['PoolQC'].isna(), 'PoolQC'] = 'None'

# check
print("\n",df_test_filled['PoolQC'].isna().sum())
```

```
 0
```

[144]:
```
# Fence: 'NA' given to properties with no fence
# replace with 'None'
df_test_filled.loc[df_test_filled['Fence'].isna(), 'Fence'] = 'None'

# check
print("\n",df_test_filled['Fence'].isna().sum())
```

```
 0
```

[145]:
```
# MiscFeature: 'NA' given to properties with no miscellaneous features
# replace with 'None'
df_test_filled.loc[df_test_filled['MiscFeature'].isna(), 'MiscFeature'] = 'None'

# check
print("\n",df_test_filled['MiscFeature'].isna().sum())
```

```
 0
```

Prepare training and testing sets

[146]:
```
# comparison between data with lots of columns and very little columns so cut␣
↪down to highly correlated ones
```

```python
# only keep 15 rows with highest correlation with SalePriceLog
top_features = corr_price_log_df.head(15)['Feature'].tolist()
print(top_features)
```

```
['OverallQual', 'GrLivArea', 'GarageCars', 'GarageArea', 'TotalBsmtSF',
'1stFlrSF', 'FullBath', 'YearBuilt', 'YearRemodAdd', 'TotRmsAbvGrd',
'Fireplaces', 'MasVnrArea', 'BsmtFinSF1', 'GarageYrBlt', 'WoodDeckSF']
```

[147]:
```python
# training
X_train = df_train_filled[top_features]

# test
X_test = df_test_filled[top_features]
```

[148]:
```python
print(X_train.dtypes)
print(X_test.dtypes)
```

```
OverallQual       int64
GrLivArea         int64
GarageCars        int64
GarageArea        int64
TotalBsmtSF       int64
1stFlrSF          int64
FullBath          int64
YearBuilt         int64
YearRemodAdd      int64
TotRmsAbvGrd      int64
Fireplaces        int64
MasVnrArea      float64
BsmtFinSF1        int64
GarageYrBlt     float64
WoodDeckSF        int64
dtype: object
OverallQual       int64
GrLivArea         int64
GarageCars        int64
GarageArea        int64
TotalBsmtSF       int64
1stFlrSF          int64
FullBath          int64
YearBuilt         int64
YearRemodAdd      int64
TotRmsAbvGrd      int64
Fireplaces        int64
MasVnrArea        int64
BsmtFinSF1        int64
GarageYrBlt     float64
WoodDeckSF        int64
```

```
dtype: object
```

[149]:
```python
# check shapes
print(X_train.shape)
print(X_test.shape)
```

```
(1449, 15)
(11, 15)
```

[150]:
```python
# training y with log
y_train = np.log1p(df_train_filled['SalePrice'])
```

1. Linear Regression with OLS

[151]:
```python
# linear regression
ols_reg = LinearRegression()
ols_reg.fit(X_train, y_train)
ols_pred = ols_reg.predict(X_test)
```

[152]:
```python
# print transformed back log predictions
np.expm1(ols_pred)
```

[152]:
```
array([ 91213.52574606, 119663.32579797, 253870.73434434, 143125.21862055,
       121431.57196573, 184523.25609385, 181877.30639575, 232671.39556835,
       219839.95128948, 126111.54728248, 142793.82665374])
```

These predictions seem to make sense since in part 2, the mean of sale prices was around $181 000

[161]:
```python
# error analysis
# predict using training data since there is no target to compare it to
ols_train_pred = ols_reg.predict(X_train)

print("\nR-Squared:", round(r2_score(y_train, ols_train_pred), 4))
print("MSE:", round(mean_squared_error(y_train, ols_train_pred), 4))
```

```
R-Squared: 0.8393
MSE: 0.0257
```

[160]:
```python
# OLS coefficient table
ols_coef_df = pd.DataFrame({
    'Feature' : X_train.columns,
    'Coefficient': ols_reg.coef_}).sort_values(by='Coefficient',␣
 ↪ascending=False)

print(ols_coef_df)
print('Intercept:', ols_reg.intercept_)
```

```
         Feature  Coefficient
0    OverallQual     0.090296
10    Fireplaces     0.064658
```

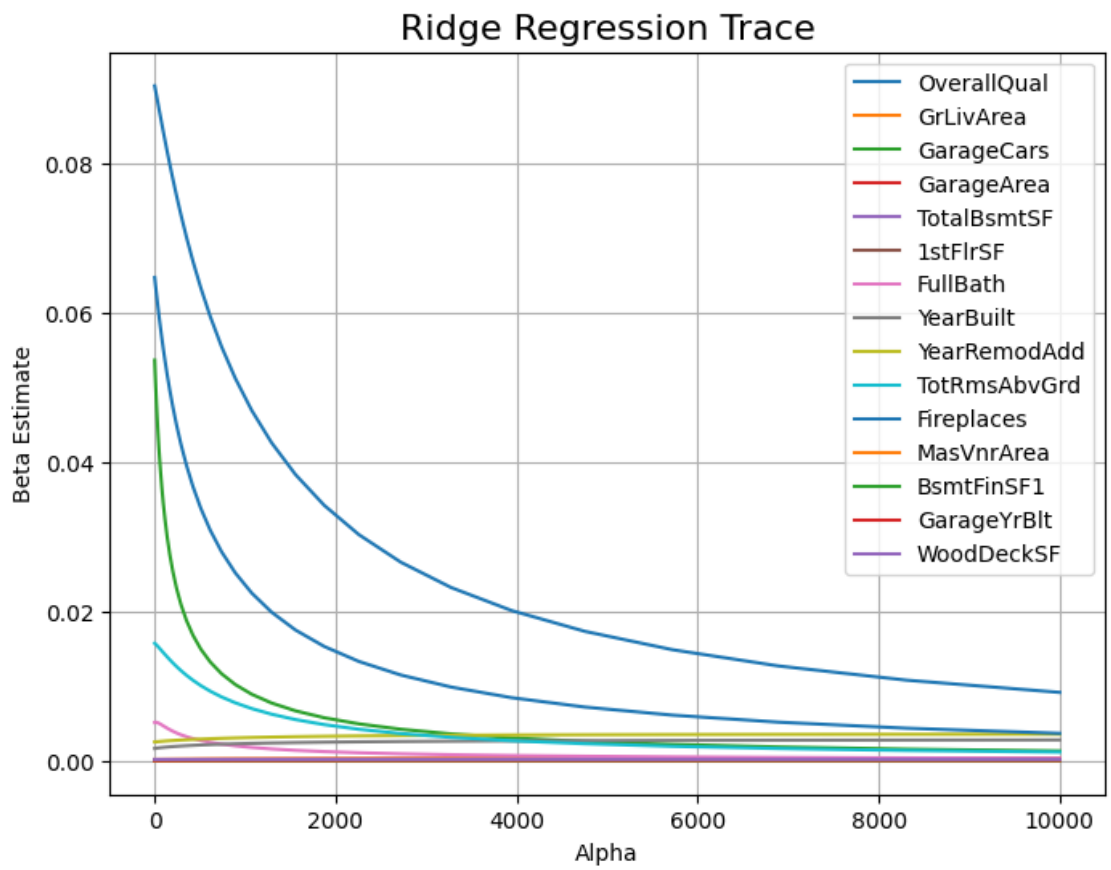
```
2     GarageCars     0.053586
9   TotRmsAbvGrd     0.015701
6       FullBath     0.005110
8   YearRemodAdd     0.002504
7      YearBuilt     0.001629
1      GrLivArea     0.000158
14    WoodDeckSF     0.000155
12    BsmtFinSF1     0.000074
4    TotalBsmtSF     0.000064
3     GarageArea     0.000050
13   GarageYrBlt     0.000033
5       1stFlrSF     0.000024
11    MasVnrArea    -0.000006
Intercept: 2.5777772079775527
```

The OLS model predicts the log-transformed sale price using the selected housing features in 'top_features'. Positive coefficients mean that increasing the feature is associated with a higher predicted sale price, while other variables are fixed. Negative coefficients mean the opposite with a larger value being associated with lower predicted sale prices.

2. Ridge Regression

[154]:
```
# Ridge regression trace
ridge_reg = Ridge(alpha=0)
ridge_reg.fit(X_train, y_train)
ridge_train_pred = []
ridge_test_pred = []
coef_list = []

# using log scale for alpha
alphas = np.logspace(-4, 4, 100)

for alpha in alphas:
    ridge_reg = Ridge(alpha=alpha)
    ridge_reg.fit(X_train, y_train)
    var_name = 'estimate' + str(alpha)

    ridge_train_pred.append(ridge_reg.predict(X_train))
    ridge_test_pred.append(ridge_reg.predict(X_test))

    coef_list.append(ridge_reg.coef_)

# dataframe of coefficients
ridge_df = pd.DataFrame(coef_list, columns=X_train.columns)
ridge_df['alpha'] = alphas

# plot
fig, ax = plt.subplots(figsize=(8,6))
```

```
for c in X_train.columns:
    ax.plot(ridge_df['alpha'], ridge_df[c])

ax.set_xlabel('Alpha')
ax.set_ylabel('Beta Estimate')
ax.set_title('Ridge Regression Trace', fontsize=16)
ax.legend(labels=X_train.columns)
ax.grid(True)
```



The regression trace makes it hard to see which alpha value is optimal visually since the stronger correlated features start converging from 1000 to almost 4000, but other weaker ones are immediately converged. Using GridSearchCV:

[155]:
```
# gridsearch cross-validation to find best alpha value
param = {'alpha': np.logspace(-2, 2, 100)}

ridge_reg_grid = GridSearchCV(Ridge(), param)
ridge_reg_grid.fit(X_train, y_train)
ridge_grid_pred = ridge_reg_grid.predict(X_test)
```

```
print(ridge_reg_grid.best_estimator_)
```

```
Ridge(alpha=np.float64(32.745491628777316))
```

[156]:
```
# error analysis
# predict using training data
ridge_train_pred = ridge_reg_grid.best_estimator_.predict(X_train)

print("\nR-Squared:", round(r2_score(y_train, ridge_train_pred), 4))
print("MSE:", round(mean_squared_error(y_train, ridge_train_pred), 4))
```

```
R-Squared: 0.8392
MSE: 0.0257
```

[162]:
```
ridge_best_coef_df = pd.DataFrame({'Feature' : X_train.columns,
                                   'Coefficient' : ridge_reg_grid.
 ↪best_estimator_.coef_}).sort_values(by='Coefficient', ascending=False)
print('Best Ridge alpha:', ridge_reg_grid.best_estimator_.alpha)
print('Ridge intercept:', ridge_reg_grid.best_estimator_.intercept_)
```

```
Best Ridge alpha: 32.745491628777316
Ridge intercept: 2.401774189467499
```

[157]:
```
# transform back log predictions
np.expm1(ridge_grid_pred)
```

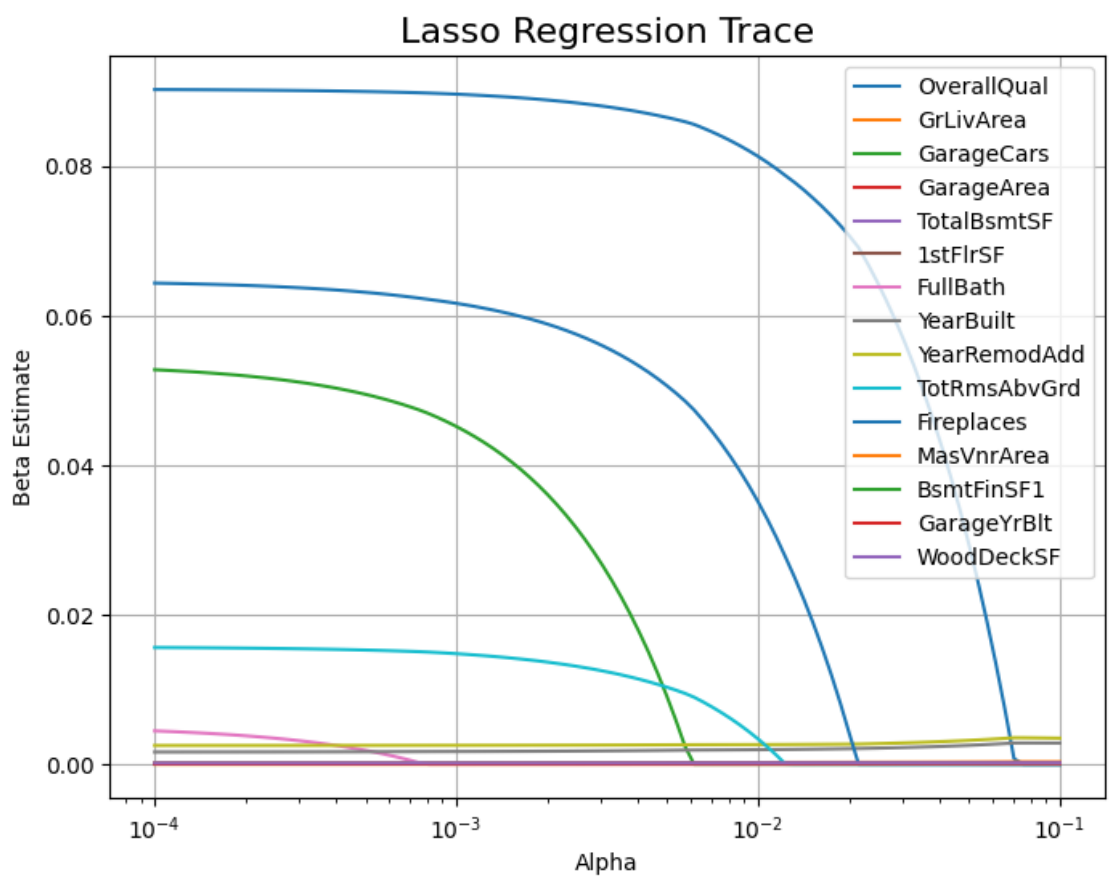
[157]:
```
array([ 91061.87840547, 120181.50065898, 252924.0140872 , 143675.86926079,
       122017.95277098, 184173.77685444, 181746.2972764 , 232402.16241377,
       219450.49855036, 126669.75440348, 143611.36226969])
```

These predictions also seem to be acceptable, now trying with lasso.

3. Lasso Regression

[163]:
```
lasso_reg = Lasso(alpha=1)
lasso_reg.fit(X_train, y_train)
lasso_train_pred = []
lasso_test_pred = []
coef_list = []

alphas = np.logspace(-4, -1, 100)

for alpha in alphas:
    lasso_reg = Lasso(alpha=alpha)
    lasso_reg.fit(X_train, y_train)
    var_name = 'estimate' + str(alpha)

    lasso_train_pred.append(lasso_reg.predict(X_train))
    lasso_test_pred.append(lasso_reg.predict(X_test))
```

```
    coef_list.append(lasso_reg.coef_)

# dataframe of coefficients
lasso_df = pd.DataFrame(coef_list, columns=X_train.columns)
lasso_df['alpha'] = alphas

# plot
fig, ax = plt.subplots(figsize=(8,6))

for c in X_train.columns:
    ax.plot(lasso_df['alpha'], lasso_df[c])
ax.set_xscale('log')
ax.set_xlabel('Alpha')
ax.set_ylabel('Beta Estimate')
ax.set_title('Lasso Regression Trace', fontsize=16)
ax.legend(labels=X_train.columns, loc='upper right')
ax.grid(True)
```



The stronger coefficients go all the way to 0.01 but weaker ones converge right away, checking for

optimal alpha with GridSearchCV:

[164]:
```
# gridsearch cross-validation to find best alpha value
param = {'alpha': np.logspace(-8, -1, 100)}

lasso_reg_grid = GridSearchCV(Lasso(), param)
lasso_reg_grid.fit(X_train, y_train)
lasso_grid_pred = lasso_reg_grid.predict(X_test)

print(lasso_reg_grid.best_estimator_)
```

```
Lasso(alpha=np.float64(1e-08))
```

[168]:
```
lasso_best_coef_df = pd.DataFrame({'Feature' : X_train.columns,
                                   'Coefficient' : lasso_reg_grid.
 ↪best_estimator_.coef_}).sort_values(by='Coefficient', ascending=False)
print(lasso_best_coef_df)
print('Best Lasso alpha:', lasso_reg_grid.best_estimator_.alpha)
print('Lasso intercept:', lasso_reg_grid.best_estimator_.intercept_)
```

```
         Feature  Coefficient
0    OverallQual     0.090296
10    Fireplaces     0.064658
2     GarageCars     0.053586
9   TotRmsAbvGrd     0.015701
6       FullBath     0.005110
8   YearRemodAdd     0.002504
7      YearBuilt     0.001629
1      GrLivArea     0.000158
14    WoodDeckSF     0.000155
12    BsmtFinSF1     0.000074
4    TotalBsmtSF     0.000064
3     GarageArea     0.000050
13   GarageYrBlt     0.000033
5       1stFlrSF     0.000024
11    MasVnrArea    -0.000006
Best Lasso alpha: 1e-08
Lasso intercept: 2.577775151246419
```

The optimal lasso is extremely small (alpha = 1e-08) which means CV preferred a model very close to OLS. This suggest that a strong coefficient shrinkage will not help much for this feature set.

[169]:
```
# transform back log predictions
np.expm1(lasso_grid_pred)
```

[169]:
```
array([ 91213.52635213, 119663.32851869, 253870.7267675 , 143125.22742795,
       121431.58200201, 184523.2492826 , 181877.30275466, 232671.39084608,
       219839.94753582, 126111.55344515, 142793.83471814])
```

[170]:
```
# error analysis
# predict using training data
lasso_train_pred = lasso_reg_grid.best_estimator_.predict(X_train)

print("\nR-Squared:", round(r2_score(y_train, lasso_train_pred), 4))
print("MSE:", round(mean_squared_error(y_train, lasso_train_pred), 4))
```

```
R-Squared: 0.8393
MSE: 0.0257
```

Choice: Ridge Regression Each technique has the same high R-squared valued (0.8393) and low MSE (0.0257), Ridge Regression would be the better choice to handle overfitting when it comes to some of the extreme values seen in this dataset. Property prices involves features that can spread out in large ranges, so to prevent the training set from overly trying to adapt to these points, Ridge Regression will shrink these estimates and give a better prediction of unseen property prices.

1.5 Question D

[197]:
```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import RBF, ConstantKernel as C,␣
↪WhiteKernel
from sklearn.metrics import r2_score, mean_squared_error
from sklearn.ensemble import RandomForestRegressor
```

[198]:
```
# download data
air_quality = pd.read_csv('C:\\Users\\tessa\\Downloads\\sem 5 concordia\\mast␣
↪387\\AirQuality.csv', sep=';')
```

Pre-process data

[199]:
```
# create clean dataset
air_q_clean = air_quality.copy()
```

[200]:
```
# shape of dataset
print(air_q_clean.shape)
```

```
(9471, 17)
```

[201]:
```
air_q_clean.head()
```

[201]:

	Date	Time	CO(GT)	PT08.S1(CO)	NMHC(GT)	C6H6(GT)	PT08.S2(NMHC)	\
0	10/03/2004	18.00.00	2,6	1360.0	150.0	11,9	1046.0	
1	10/03/2004	19.00.00	2	1292.0	112.0	9,4	955.0	

```
2  10/03/2004  20.00.00    2,2       1402.0      88.0      9,0          939.0
3  10/03/2004  21.00.00    2,2       1376.0      80.0      9,2          948.0
4  10/03/2004  22.00.00    1,6       1272.0      51.0      6,5          836.0

   NOx(GT)  PT08.S3(NOx)  NO2(GT)  PT08.S4(NO2)  PT08.S5(O3)     T    RH  \
0    166.0        1056.0    113.0        1692.0       1268.0  13,6  48,9
1    103.0        1174.0     92.0        1559.0        972.0  13,3  47,7
2    131.0        1140.0    114.0        1555.0       1074.0  11,9  54,0
3    172.0        1092.0    122.0        1584.0       1203.0  11,0  60,0
4    131.0        1205.0    116.0        1490.0       1110.0  11,2  59,6

       AH  Unnamed: 15  Unnamed: 16
0  0,7578          NaN          NaN
1  0,7255          NaN          NaN
2  0,7502          NaN          NaN
3  0,7867          NaN          NaN
4  0,7888          NaN          NaN
```

[202]:
```python
# check which features have missing values
missing = air_q_clean.isna().sum()
print(missing)
```

```
Date              114
Time              114
CO(GT)            114
PT08.S1(CO)       114
NMHC(GT)          114
C6H6(GT)          114
PT08.S2(NMHC)     114
NOx(GT)           114
PT08.S3(NOx)      114
NO2(GT)           114
PT08.S4(NO2)      114
PT08.S5(O3)       114
T                 114
RH                114
AH                114
Unnamed: 15      9471
Unnamed: 16      9471
dtype: int64
```

[203]:
```python
# the two last columns have no values so I will remove them
air_q_clean=air_q_clean.drop(columns=['Unnamed: 15', 'Unnamed: 16'])
```

[204]:
```python
# drop 114 missing values
air_q_clean = air_q_clean.dropna()

# check
```

```
missing = air_q_clean.isna().sum()
print(missing)
print(air_q_clean.shape)
```

```
Date             0
Time             0
CO(GT)           0
PT08.S1(CO)      0
NMHC(GT)         0
C6H6(GT)         0
PT08.S2(NMHC)    0
NOx(GT)          0
PT08.S3(NOx)     0
NO2(GT)          0
PT08.S4(NO2)     0
PT08.S5(O3)      0
T                0
RH               0
AH               0
dtype: int64
(9357, 15)
```

[205]:
```
# check data types
air_q_clean.dtypes
```

[205]:
```
Date              object
Time              object
CO(GT)            object
PT08.S1(CO)      float64
NMHC(GT)         float64
C6H6(GT)          object
PT08.S2(NMHC)    float64
NOx(GT)          float64
PT08.S3(NOx)     float64
NO2(GT)          float64
PT08.S4(NO2)     float64
PT08.S5(O3)      float64
T                 object
RH                object
AH                object
dtype: object
```

[206]:
```
# date and time can be objects but the rest should be floats
to_float = ['CO(GT)', 'C6H6(GT)', 'T', 'RH', 'AH']

# replace the comma in each value with a period and change to float
for c in to_float:
```

```
    air_q_clean[c] = (air_q_clean[c].str.replace(',','.',regex=False).
↪astype(float))
```

[207]:
```
# check
air_q_clean.dtypes
```

[207]:
```
Date              object
Time              object
CO(GT)           float64
PT08.S1(CO)      float64
NMHC(GT)         float64
C6H6(GT)         float64
PT08.S2(NMHC)    float64
NOx(GT)          float64
PT08.S3(NOx)     float64
NO2(GT)          float64
PT08.S4(NO2)     float64
PT08.S5(O3)      float64
T                float64
RH               float64
AH               float64
dtype: object
```

[208]:
```
# check for concentrations that don't make sense
air_q_clean[air_q_clean['PT08.S1(CO)']<0]
```

[208]:
```
            Date      Time  CO(GT)  PT08.S1(CO)  NMHC(GT)  C6H6(GT)  \
524   01/04/2004  14.00.00     1.7       -200.0     222.0    -200.0
525   01/04/2004  15.00.00     1.9       -200.0     197.0    -200.0
526   01/04/2004  16.00.00     2.3       -200.0     319.0    -200.0
701   08/04/2004  23.00.00     2.0       -200.0     137.0    -200.0
702   09/04/2004  00.00.00     2.4       -200.0     189.0    -200.0
…           …        …      …           …        …        …
8111  11/02/2005  17.00.00     6.6       -200.0    -200.0    -200.0
8112  11/02/2005  18.00.00     6.5       -200.0    -200.0    -200.0
8113  11/02/2005  19.00.00     7.1       -200.0    -200.0    -200.0
8114  11/02/2005  20.00.00     4.9       -200.0    -200.0    -200.0
8777  11/03/2005  11.00.00     3.0       -200.0    -200.0    -200.0

      PT08.S2(NMHC)  NOx(GT)  PT08.S3(NOx)  NO2(GT)  PT08.S4(NO2)  \
524          -200.0     99.0        -200.0     72.0        -200.0
525          -200.0    108.0        -200.0     81.0        -200.0
526          -200.0    131.0        -200.0     93.0        -200.0
701          -200.0    129.0        -200.0    106.0        -200.0
702          -200.0    154.0        -200.0    109.0        -200.0
…             …       …            …        …            …
8111         -200.0   1227.0        -200.0    326.0        -200.0
```

```
8112         -200.0   1061.0        -200.0    284.0        -200.0
8113         -200.0   1075.0        -200.0    289.0        -200.0
8114         -200.0    641.0        -200.0    215.0        -200.0
8777         -200.0    812.0        -200.0    235.0        -200.0

      PT08.S5(O3)      T     RH     AH
524        -200.0 -200.0 -200.0 -200.0
525        -200.0 -200.0 -200.0 -200.0
526        -200.0 -200.0 -200.0 -200.0
701        -200.0 -200.0 -200.0 -200.0
702        -200.0 -200.0 -200.0 -200.0
…           …     …     …     …
8111       -200.0 -200.0 -200.0 -200.0
8112       -200.0 -200.0 -200.0 -200.0
8113       -200.0 -200.0 -200.0 -200.0
8114       -200.0 -200.0 -200.0 -200.0
8777       -200.0 -200.0 -200.0 -200.0

[366 rows x 15 columns]
```

[209]:
```python
# there are a lot -200 values in the data, checking percentages of values
counts_minus200 = (((air_q_clean == -200).sum())/air_q_clean.shape[0])*100
print(counts_minus200)
```

```
Date              0.000000
Time              0.000000
CO(GT)           17.986534
PT08.S1(CO)       3.911510
NMHC(GT)         90.231912
C6H6(GT)          3.911510
PT08.S2(NMHC)     3.911510
NOx(GT)          17.516298
PT08.S3(NOx)      3.911510
NO2(GT)          17.548360
PT08.S4(NO2)      3.911510
PT08.S5(O3)       3.911510
T                 3.911510
RH                3.911510
AH                3.911510
dtype: float64
```

[210]:
```python
# remove NMHC(GT) column since most of the data seems to be missing
air_q_clean = air_q_clean.drop(columns = ['NMHC(GT)'])
```

[211]:
```python
# there is about 3.91 % of values missing in the target variable, same as for a␣
↪lot of variables
# replacing -200 values with NaN
air_q_clean = air_q_clean.replace(-200, np.nan)
```

```
# drop rows where target variable is not measured
air_q_clean = air_q_clean.dropna(subset=['C6H6(GT)'])
```

[212]:
```
# check
air_q_clean.isna().sum()
```

[212]:
```
Date                0
Time                0
CO(GT)           1647
PT08.S1(CO)         0
C6H6(GT)            0
PT08.S2(NMHC)       0
NOx(GT)          1595
PT08.S3(NOx)        0
NO2(GT)          1598
PT08.S4(NO2)        0
PT08.S5(O3)         0
T                   0
RH                  0
AH                  0
dtype: int64
```

[213]:
```
# there are only a few missing values left, I will replace them by the median␣
 ↪of their column
# avoids too much shift from outliers

# CO(GT)
med_CO = air_q_clean['CO(GT)'].median()
air_q_clean['CO(GT)'] = air_q_clean['CO(GT)'].fillna(med_CO)

# NOx(GT)
med_NOx = air_q_clean['NOx(GT)'].median()
air_q_clean['NOx(GT)'] = air_q_clean['NOx(GT)'].fillna(med_NOx)

# NO2(GT)
med_NO2 = air_q_clean['NO2(GT)'].median()
air_q_clean['NO2(GT)'] = air_q_clean['NO2(GT)'].fillna(med_NO2)

# check
air_q_clean.isna().sum()
```

[213]:
```
Date            0
Time            0
CO(GT)          0
PT08.S1(CO)     0
C6H6(GT)        0
```

```
PT08.S2(NMHC)    0
NOx(GT)          0
PT08.S3(NOx)     0
NO2(GT)          0
PT08.S4(NO2)     0
PT08.S5(O3)      0
T                0
RH               0
AH               0
dtype: int64
```

[214]:
```
# no more missing values
```

Prepare Data

[215]:
```
# get set of columns without target variable
# Remove Date and Time since they do not provide meaningful information to the␣
↪model
col_relevant = [c for c in air_q_clean.columns if c not in ['Date', 'Time',␣
↪'C6H6(GT)']]

# find most correlated features to target variable
corr_air = [air_q_clean['C6H6(GT)'].corr(air_q_clean[c]) for c in col_relevant]

# add to df with column names
data_corr = {'Feature': col_relevant, 'Corr_C6H6(GT)': np.abs(corr_air)}
corr_air_df = pd.DataFrame(data_corr)

# sort
corr_air_df = corr_air_df.sort_values(by='Corr_C6H6(GT)', ascending=False)
print(corr_air_df)
```

```
          Feature  Corr_C6H6(GT)
2   PT08.S2(NMHC)       0.981950
1     PT08.S1(CO)       0.883795
7     PT08.S5(O3)       0.865689
0          CO(GT)       0.841802
6    PT08.S4(NO2)       0.765731
4    PT08.S3(NOx)       0.735744
3         NOx(GT)       0.656325
5         NO2(GT)       0.558710
8               T       0.198956
10             AH       0.167972
9              RH       0.061681
```

Feature selection was performed using the absolute correlation with the target variable 'C6H6(GT)'. Features with very low correlation (RH, AH, and T) were removed, as they provide little predictive power. This reduces model complexity and helps prevent overfitting while keeping the most informative variables.

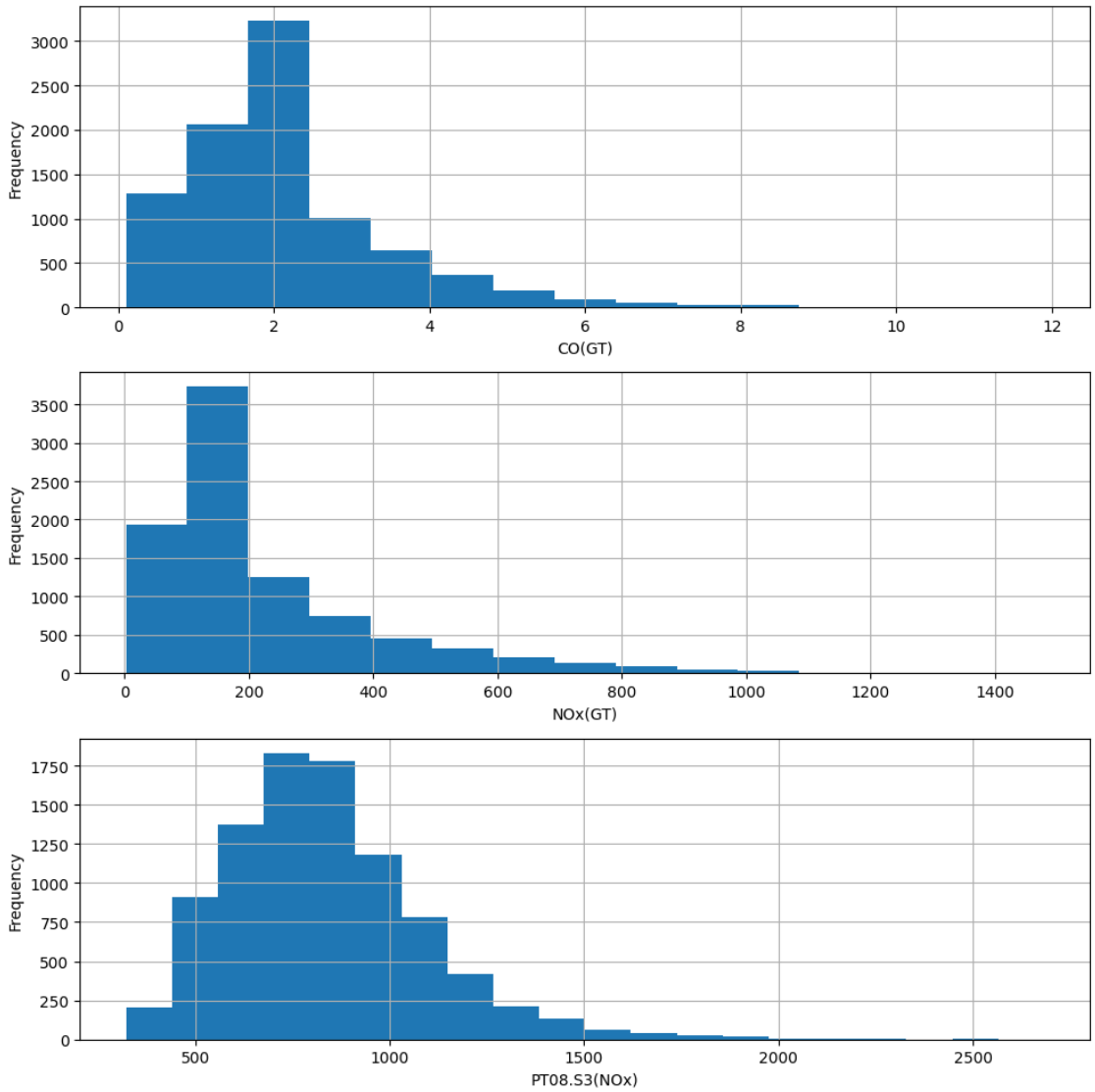
[216]:
```
# the last three columns have very low correlation so I will drop them
# split into X containing most correlated features and y containing target␣
↪variable
imp_cols = [c for c in col_relevant if c not in ['RH', 'AH', 'T']]

X = air_q_clean[imp_cols]
y = air_q_clean['C6H6(GT)']
```

[217]:
```
# check for potential outliers
# summary statistics
print(round(X.describe(),2))
print("\n",round(y.describe()))
```

```
        CO(GT)  PT08.S1(CO)  PT08.S2(NMHC)  NOx(GT)  PT08.S3(NOx)  NO2(GT)  \
count  8991.00      8991.00        8991.00  8991.00       8991.00  8991.00
mean      2.07      1099.83         939.15   230.80        835.49   111.59
std       1.30       217.08         266.83   188.72        256.82    43.21
min       0.10       647.00         383.00     2.00        322.00     2.00
25%       1.20       937.00         734.50   112.00        658.00    85.00
50%       1.80      1063.00         909.00   178.00        806.00   109.00
75%       2.60      1231.00        1116.00   280.00        969.50   132.00
max      11.90      2040.00        2214.00  1479.00       2683.00   333.00

       PT08.S4(NO2)  PT08.S5(O3)
count       8991.00      8991.00
mean        1456.26      1022.91
std          346.21       398.48
min          551.00       221.00
25%         1227.00       731.50
50%         1463.00       963.00
75%         1674.00      1273.50
max         2775.00      2523.00

 count    8991.0
mean       10.0
std         7.0
min         0.0
25%         4.0
50%         8.0
75%        14.0
max        64.0
Name: C6H6(GT), dtype: float64
```

[218]:
```
# some of the columns with more extreme outliers seem to be CO(GT), NOx(GT),␣
↪and PT08.S3(NOx).
# visualize with histograms
```

```
plt.figure(figsize=(10,10))

# CO(GT)
plt.subplot(3,1,1)
fig=X['CO(GT)'].hist(bins=15)
fig.set_xlabel('CO(GT)')
fig.set_ylabel('Frequency')

# NOx(GT)
plt.subplot(3,1,2)
fig=X['NOx(GT)'].hist(bins=15)
fig.set_xlabel('NOx(GT)')
fig.set_ylabel('Frequency')

# PT08.S3(NOx)
plt.subplot(3,1,3)
fig=X['PT08.S3(NOx)'].hist(bins=20)
fig.set_xlabel('PT08.S3(NOx)')
fig.set_ylabel('Frequency')

plt.tight_layout()
```



Before modeling, the distributions of key variables were examined. Several features (such as CO(GT), NOx(GT), and PT08.S3(NOx)) show right-skewed distributions and the presence of extreme values. This justifies the use of scaling and robust modeling approaches, as unscaled features with large variance could negatively affect model performance.

[219]:
```
# values are right-skewed and need to be scaled to reduce effect of extreme␣
↪values

# split into train/test
# split into smaller subset of full data since there are over 9000 values
# a subset of the data is used for training to reduce computational cost while␣
↪maintaining a sufficiently large sample size for learning
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.333,␣
↪test_size=0.2, random_state=10)

# scale
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

1. Gaussian Process Regression

[220]:
```
# define kernel and model

# signal variance * RBF + white noise term
kernel = (
    C(1.0, (1e-3, 1e3))*RBF(length_scale_bounds=(1e-2,10.0))
    + WhiteKernel(noise_level=0.1, noise_level_bounds=(1e-5,1.0))
)

gp = GaussianProcessRegressor(
    kernel=kernel,
    alpha=0.0, # noise is handled by WhiteKernel
    normalize_y=True,
    n_restarts_optimizer=5,
    random_state=11
)

# fit
gp.fit(X_train, y_train)

# predicttions
y_pred_gp, y_std_gp = gp.predict(X_test, return_std=True)

print('Optimized kernel hyperparameters:')
print(gp.kernel_)
```

```
C:\Users\tessa\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n
2kfra8p0\LocalCache\local-packages\Python313\site-
packages\sklearn\gaussian_process\kernels.py:450: ConvergenceWarning: The
optimal value found for dimension 0 of parameter k1__k2__length_scale is close
to the specified upper bound 10.0. Increasing the bound and calling fit again
may find a better value.
  warnings.warn(

Optimized kernel hyperparameters:
2.88**2 * RBF(length_scale=10) + WhiteKernel(noise_level=1.53e-05)
```

[221]:
```
y_pred_gp
```

[221]: array([1.57253969, 13.8539483 , 7.62206823, …, 14.53372769,
       16.45896916, 10.08476524], shape=(1799,))

[222]:
```
# evaluate model with R-squared and Mean Squared Error

# r-squared
r2_gp = r2_score(y_test, y_pred_gp)
print("R-Squared:", r2_gp)

# mse
mse_gp = mean_squared_error(y_test, y_pred_gp)
print("MSE:", mse_gp)
```

```
R-Squared: 0.9999811632536668
MSE: 0.0010276774150365714
```

The model has very low error and a suspiciously high R-squared value. Since high correlations were observed earlier, then there could be strong multicollinearity in the dataset leading to this results.

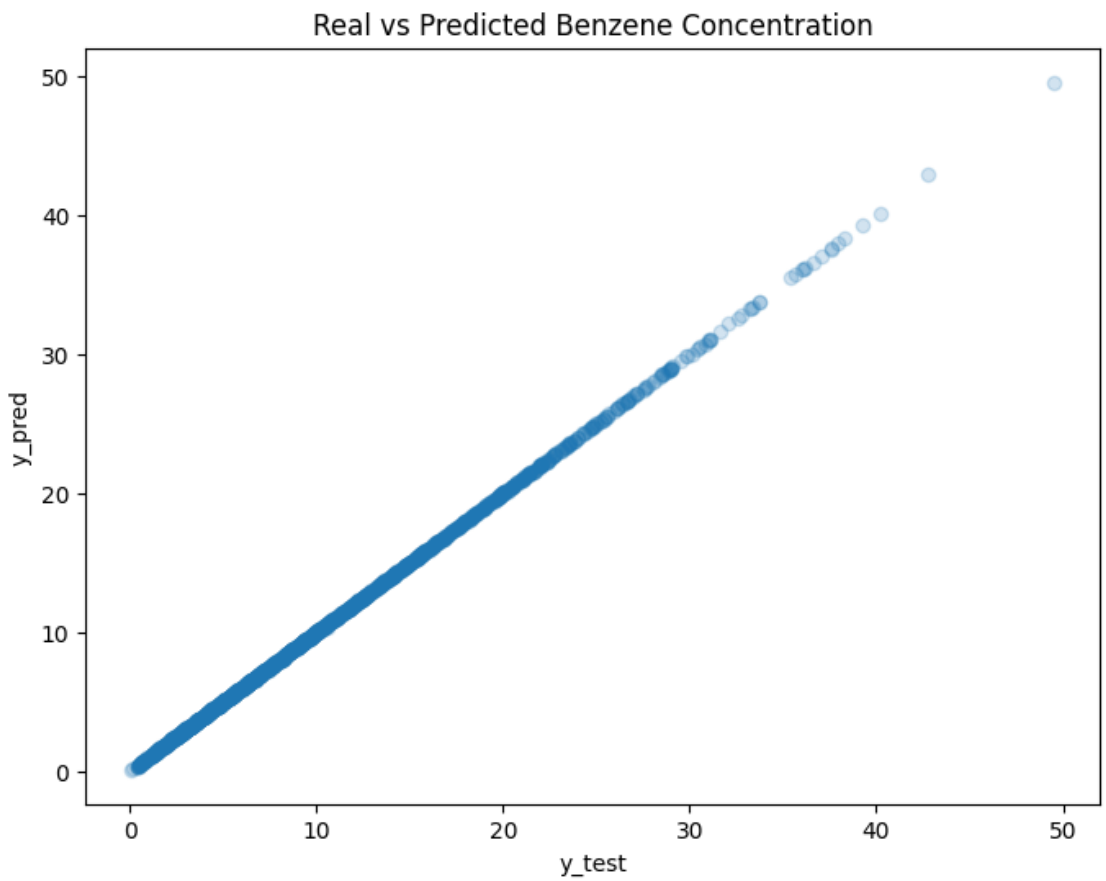
[223]:
```
# check with train data
y_train_pred = gp.predict(X_train)
r2_train = r2_score(y_train, y_train_pred)
mse_train = mean_squared_error(y_train, y_train_pred)
print("R-Squared:", r2_train)
print("MSE:", mse_train)
```

```
R-Squared: 0.99998581791237
MSE: 0.0008178966490410547
```

The training model also has very high R-squared and very low MSE, this matches the test data.

[224]:
```
# check preds vs real values
plt.figure(figsize=(8,6))
plt.scatter(y_test, y_pred_gp, alpha=0.2)
plt.xlabel('y_test')
plt.ylabel('y_pred')
plt.title('Real vs Predicted Benzene Concentration')
```

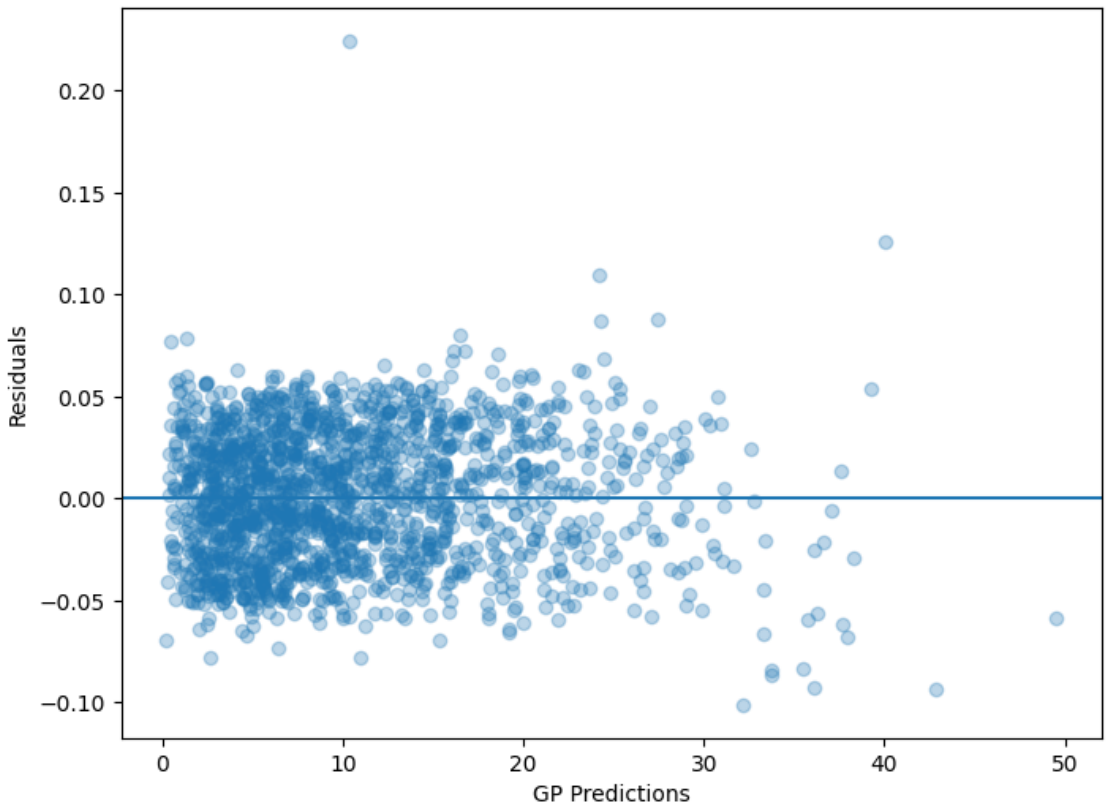
[224]: Text(0.5, 1.0, 'Real vs Predicted Benzene Concentration')



Every single predicted value seems to lie on the line y = x. This represents very small error and very good fit, although this data seems to be very predictable due to high correlations.

[225]:
```
# lastly plot residuals to check for errors
res_gp = y_test - y_pred_gp
plt.figure(figsize=(8,6))
plt.scatter(y_pred_gp, res_gp, alpha=0.3)
plt.axhline(0)
plt.xlabel('GP Predictions')
plt.ylabel('Residuals')
```

[225]: Text(0, 0.5, 'Residuals')



The residuals are centered around zero and show no clear systematic pattern, suggesting that the model does not exhibit significant bias. Because the residuals are small, this confirms that prediction errors are minimal. There is a slight in the spread of the residuals for higher predicted values.

2. Random Forest Regression

[226]:
```
# instantiate model
rf = RandomForestRegressor(n_estimators=200, random_state=11)

# fit model
rf.fit(X_train, y_train)
```

[226]:
```
RandomForestRegressor(n_estimators=200, random_state=11)
```

[227]:
```
# predict
y_pred_rf = rf.predict(X_test)
```

[228]:
```
y_pred_rf
```

[228]:
```
array([ 1.6   , 13.8305,  7.6   , …, 14.4985, 16.462 , 10.1   ],
      shape=(1799,))
```

[229]:
```
# evaluate with R-Squared and MSE
r2_rf = r2_score(y_test, y_pred_rf)
print('R-Squared:',r2_rf)

mse_rf = mean_squared_error(y_test, y_pred_rf)
print('MSE:',mse_rf)
```

```
R-Squared: 0.9998927843620261
MSE: 0.005849369510839171
```

The MSE is again extremely low and R-squared extremely high, indicating very little error in the predictions, which could yet again be due to the high correlations with the target variable.

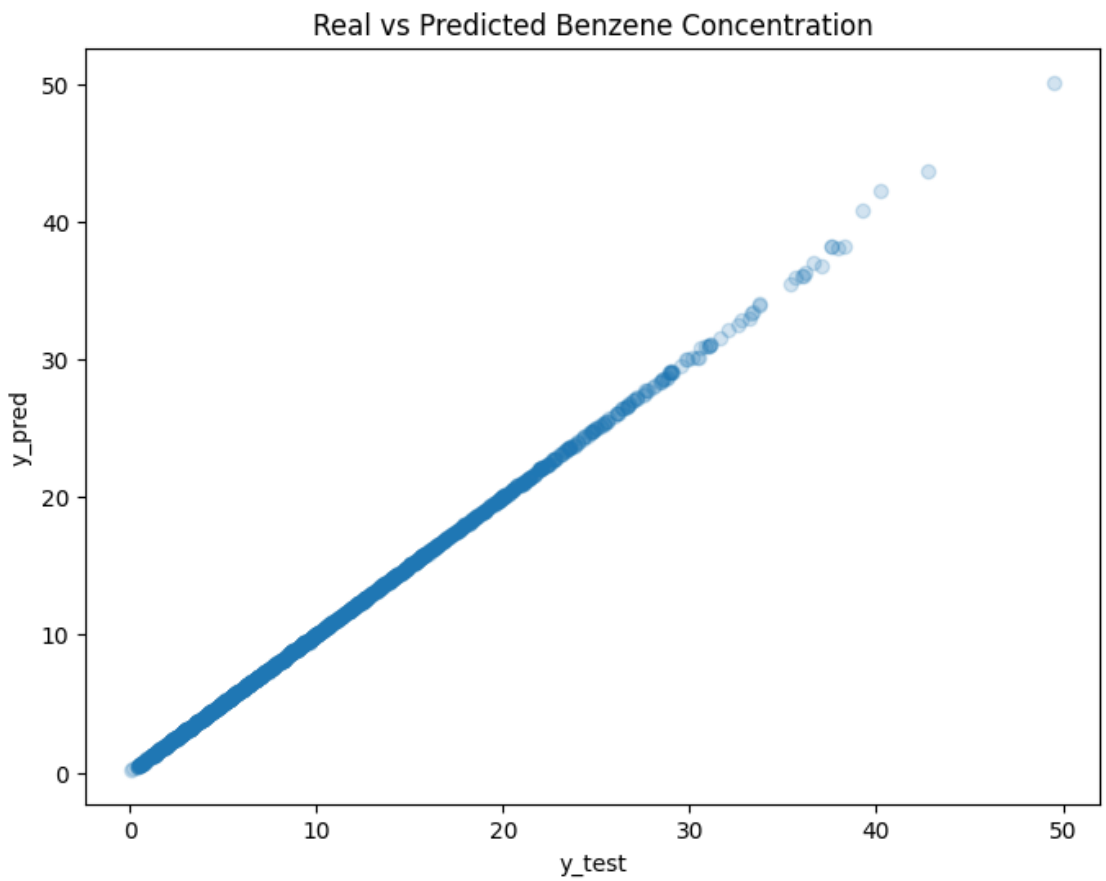
[230]:
```
# check with train data
y_train_pred = rf.predict(X_train)
r2_train = r2_score(y_train, y_train_pred)
mse_train = mean_squared_error(y_train, y_train_pred)
print("R-Squared:", r2_train)
print("MSE:", mse_train)
```

```
R-Squared: 0.9996910174328122
MSE: 0.017819365731462335
```

The training data shows a slightly larger MSE and slightly smaller R-squared, but still very consistent with the test data.

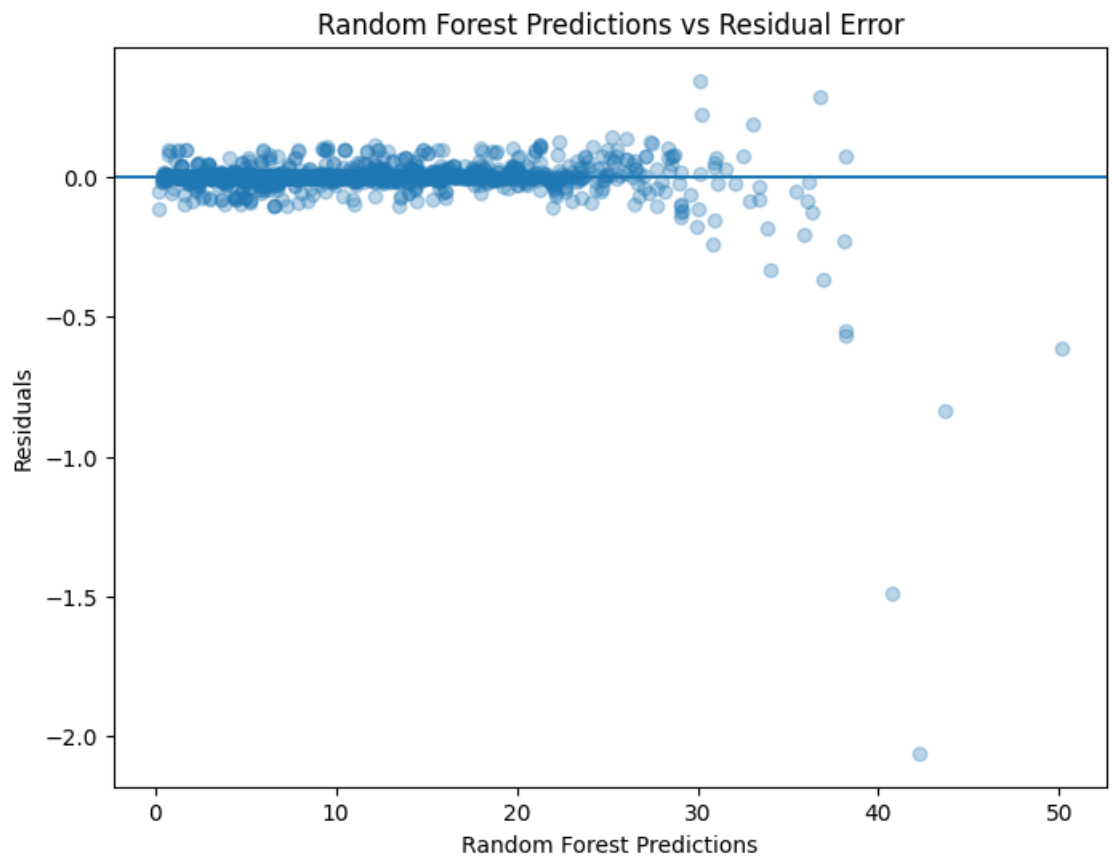
[231]:
```
# check preds vs real values
plt.figure(figsize=(8,6))
plt.scatter(y_test, y_pred_rf, alpha=0.2)
plt.xlabel('y_test')
plt.ylabel('y_pred')
plt.title('Real vs Predicted Benzene Concentration')
```

[231]:
```
Text(0.5, 1.0, 'Real vs Predicted Benzene Concentration')
```



[633]:
```
# lastly plot residuals to check for errors
res_rf = y_test - y_pred_rf
plt.figure(figsize=(8,6))
plt.scatter(y_pred_rf, res_rf, alpha=0.3)
plt.axhline(0)
plt.xlabel('Random Forest Predictions')
plt.ylabel('Residuals')
plt.title('Random Forest Predictions vs Residual Error')
```

[633]: Text(0.5, 1.0, 'Random Forest Predictions vs Residual Error')

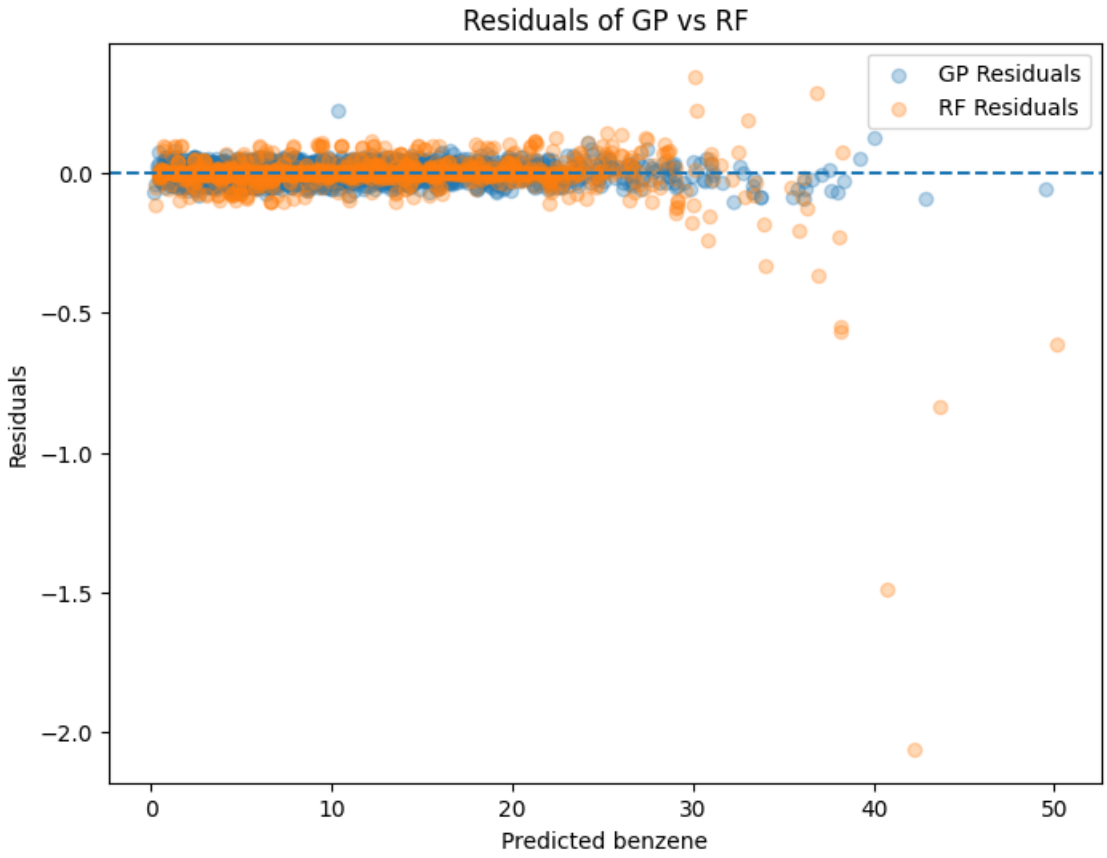


Other than a a few data points that go out of the -0.5 to 0.5 range, the plot indicates very low residual error. Random Forest Regression also predicts almost perfectly the benzene concentration. This is shown with its extremely low MSE (0.008) and extremely high R-Squared (0.9998) values.

Comparison

[634]:
```
# plotting residuals together
plt.figure(figsize=(8,6))
plt.scatter(y_pred_gp, res_gp, alpha=0.3, label='GP Residuals')
plt.scatter(y_pred_rf, res_rf, alpha=0.3, label='RF Residuals')

plt.axhline(0, linestyle='--')
plt.xlabel('Predicted benzene')
plt.ylabel('Residuals')
plt.title('Residuals of GP vs RF')
plt.legend()
```

[634]:
```
<matplotlib.legend.Legend at 0x228a82b9d10>
```



The Random Forest model appears to have its data points slightly more spread out from the axis than the Gaussian Process data points. It also has more points showing higher residual error.

There is a very small difference in the MSE and R-squared values for the test set with each model:

Gaussian Process:

R-Squared: 0.9999811632536668

MSE: 0.0010276774150365714

Random Forest:

R-Squared: 0.9998927843620261

MSE: 0.005849369510839171

Overall, both Gaussian Process and Random Forest models perform extremely well on this dataset, with minimal prediction error. The Gaussian Process model achieves slightly lower MSE, suggesting a better fit to the data. However, both models likely benefit from the strong correlations between features and the target variable.

Given the simplicity and interpretability of Random Forests compared to Gaussian Processes, either model could be appropriate depending on computational constraints and use case. However, due

to its slightly better performance, the Gaussian Process model appears to be the preferred choice for this dataset.